linear-algebra-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `linear_algebra_ai999x.about()` for the same answer.

Every capability linear-algebra-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import linear_algebra_ai999x

result = linear_algebra_ai999x.solve('vector_addition', u='1, 2, 3', v='4, 5, 6')
result.show()
# sum = [5.0, 7.0, 9.0]
```

By topic

topic	capabilities
Decompositions / Cholesky	Cholesky decomposition
Decompositions / LU	LU decomposition with partial pivoting
Decompositions / polar decomposition	Polar decomposition
Decompositions / QR	QR decomposition (Householder)
Decompositions / singular value decomposition	Singular values of a matrix, Best rank-k approximation (Eckart-Young)
Determinants / cofactor expansion	Minor and cofactor of an entry, Adjugate (classical adjoint) of a matrix
Determinants / general determinants	Determinant of an n x n matrix
Determinants / small determinants	Determinant of a 2 x 2 matrix, Determinant of a 3 x 3 matrix
Eigenvalues / characteristic polynomial	Characteristic polynomial of a matrix
Eigenvalues / definiteness	Positive definiteness test (Sylvester's criterion)
Eigenvalues / general matrices	Eigenvalues of a general square matrix (QR algorithm)
Eigenvalues / iterative methods	Dominant eigenvalue by power iteration, Smallest eigenvalue by inverse iteration
Eigenvalues / small matrices	Eigenvalues of a 2 x 2 matrix
Eigenvalues / symmetric matrices	Eigenvalues of a symmetric matrix (Jacobi method), Eigenvalues and eigenvectors of a symmetric matrix

topic	capabilities
Least squares / line fitting	Least-squares regression line
Least squares / orthogonal projections	Orthogonal projection matrix onto a column space
Least squares / overdetermined systems	Least-squares solution of an overdetermined system
Least squares / polynomial fitting	Least-squares polynomial fit
Least squares / pseudoinverse	Moore-Penrose pseudoinverse (full rank)
Linear systems / banded and sparse systems	Tridiagonal system (Thomas algorithm)
Linear systems / existence and uniqueness	Number of solutions of a linear system
Linear systems / Gaussian elimination	Solve a square linear system Ax = b
Linear systems / iterative methods	Jacobi iteration for Ax = b, Gauss-Seidel iteration for Ax = b, Conjugate gradient method for Ax = b
Linear systems / row reduction	Reduced row echelon form
Linear systems / small systems	Two linear equations in two unknowns, Three linear equations in three unknowns
Linear transformations / matrix-vector product	Matrix times vector (image under a linear map)
Linear transformations / transformations of the plane	Rotation of a plane vector
Matrices / inverses	Inverse of a square matrix
Matrices / matrix arithmetic	Sum of two matrices, Difference of two matrices, Scalar multiple of a matrix, Transpose of a matrix, Trace of a square matrix, Product of two matrices, Kronecker (tensor) product of two matrices, Power of a square matrix
Matrices / norms and conditioning	Norms of a matrix, Condition number of a matrix
Matrices / rank	Rank of a matrix
Matrix functions / exponential	Matrix exponential
Matrix functions / square root	Square root of a positive definite matrix
Numerical methods / differentiation	Derivative of equally spaced data (finite differences)
Numerical methods / interpolation	Lagrange interpolation, Newton divided differences, Coefficients of the interpolating polynomial, Piecewise linear interpolation
Numerical methods / polynomials	Evaluate a polynomial and its derivative (Horner's method)
Numerical methods / quadrature	Trapezoidal rule for tabulated data, Simpson's rule for tabulated data, Composite midpoint rule for a polynomial, Gauss-Legendre quadrature nodes and weights, Gauss-Legendre quadrature of a polynomial

topic	capabilities
Numerical methods / root finding	Root of a polynomial by bisection, Root of a polynomial by Newton's method, Root of a polynomial by the secant method
Numerical methods / splines	Natural cubic spline interpolation, Clamped cubic spline interpolation
Numerical PDE / discrete Poisson problems	1-D Poisson equation by finite differences, 2-D Poisson equation on a grid (5-point stencil)
Sparse matrices / sparse operations	Sparse matrix times vector (CSR)
Sparse matrices / storage formats	Compressed sparse row (CSR) form of a matrix, Compressed sparse column (CSC) form of a matrix, Dense matrix from CSR arrays
Vector spaces / bases and coordinates	Coordinates of a vector in a basis
Vector spaces / fundamental subspaces	Null space (kernel) of a matrix, Column space of a matrix
Vector spaces / independence and span	Linear independence of vectors
Vector spaces / orthogonality	Gram-Schmidt orthonormalization
Vectors / length and distance	Length (norm) of a vector, Unit vector in the direction of a vector, Distance between two points
Vectors / products of vectors	Dot product of two vectors, Cross product of two 3-D vectors, Angle between two vectors, Scalar triple product (volume of a parallelepiped), Area of the parallelogram (and triangle) spanned by two vectors, Outer product of two vectors
Vectors / projections	Projection of one vector onto another
Vectors / vector arithmetic	Sum of two vectors, Difference of two vectors, Scalar multiple of a vector

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
vector_addition	Sum of two vectors	u, v	sum
vector_subtraction	Difference of two vectors	u, v	difference
scalar_vector_multiplication	Scalar multiple of a vector	k, v	product
dot_product	Dot product of two vectors	u, v	dot
cross_product	Cross product of two 3-D vectors	u, v	cross
vector_norm	Length (norm) of a vector	v	norm, norm1, norm_inf
unit_vector	Unit vector in the direction of a vector	v	unit
angle_between_vectors	Angle between two vectors	u, v	angle_degrees, angle_radians
distance_between_points	Distance between two points	p, r	distance
triple_scalar_product	Scalar triple product (volume of a parallelepiped)	u, v, w	triple, volume
parallelogram_area	Area of the parallelogram (and triangle) spanned by two vectors	u, v	area, triangle_area
vector_projection	Projection of one vector onto another	u, v	projection, scalar_projection

id	operation	inputs	outputs
rotate_vector_2d	Rotation of a plane vector	v, angle	rotated
matrix_addition	Sum of two matrices	A, B	sum
matrix_subtraction	Difference of two matrices	A, B	difference
scalar_matrix_multiplication	Scalar multiple of a matrix	k, A	product
matrix_transpose	Transpose of a matrix	A	transpose
matrix_trace	Trace of a square matrix	A	trace
matrix_multiplication	Product of two matrices	A, B	product
matrix_vector_product	Matrix times vector (image under a linear map)	A, x	image
outer_product	Outer product of two vectors	u, v	outer
kronecker_product	Kronecker (tensor) product of two matrices	A, B	kron
matrix_power	Power of a square matrix	A, k	power
determinant_2x2	Determinant of a 2 x 2 matrix	A	det
determinant_3x3	Determinant of a 3 x 3 matrix	A	det
determinant	Determinant of an n x n matrix	A	det
matrix_inverse	Inverse of a square matrix	A	inverse
solve_2x2_system	Two linear equations in two unknowns	A, b	solution, det
solve_3x3_system	Three linear equations in three unknowns	A, b	solution, det
solve_linear_system	Solve a square linear system Ax = b	A, b	solution
matrix_rank	Rank of a matrix	A	rank
rref	Reduced row echelon form	A	rref, pivot_columns, rank
null_space	Null space (kernel) of a matrix	A	nullity, basis
column_space	Column space of a matrix	A	dimension, basis
linear_independence	Linear independence of vectors	vectors	independent, rank
linear_system_classification	Number of solutions of a linear system	A, b	solutions, rank_A, rank_augmented
minor_cofactor	Minor and cofactor of an entry	A, i, j	minor, cofactor
adjugate	Adjugate (classical adjoint) of a matrix	A	adjugate
characteristic_polynomial	Characteristic polynomial of a matrix	A	coefficients
eigenvalues_2x2	Eigenvalues of a 2 x 2 matrix	A	lambda1, lambda2
change_of_basis	Coordinates of a vector in a basis	basis, v	coordinates
least_squares_line	Least-squares regression line	x, y	slope, intercept, r_squared
least_squares_polynomial	Least-squares polynomial fit	x, y, degree	coefficients
least_squares_solution	Least-squares solution of an overdetermined system	A, b	solution, residual_norm
pseudoinverse	Moore-Penrose pseudoinverse (full rank)	A	pseudoinverse
projection_matrix	Orthogonal projection matrix onto a column space	A	projection
singular_values	Singular values of a matrix	A	singular_values
matrix_norms	Norms of a matrix	A	frobenius, norm1, norm_inf, norm2

id	operation	inputs	outputs
condition_number	Condition number of a matrix	A	cond2, cond1
low_rank_approximation	Best rank-k approximation (Eckart-Young)	A, k	approximation, error2, errorF
lu_decomposition	LU decomposition with partial pivoting	A	P, L, U
qr_decomposition	QR decomposition (Householder)	A	Q, R
gram_schmidt	Gram-Schmidt orthonormalization	vectors	orthonormal
cholesky	Cholesky decomposition	A	L
positive_definiteness	Positive definiteness test (Sylvester's criterion)	A	positive_definite, leading_minors
symmetric_eigenvalues	Eigenvalues of a symmetric matrix (Jacobi method)	A	eigenvalues
symmetric_eigenvectors	Eigenvalues and eigenvectors of a symmetric matrix	A	eigenvalues, eigenvectors
general_eigenvalues	Eigenvalues of a general square matrix (QR algorithm)	A	eigenvalues, spectral_radius
power_iteration	Dominant eigenvalue by power iteration	A	eigenvalue, eigenvector
inverse_iteration	Smallest eigenvalue by inverse iteration	A	eigenvalue, eigenvector
matrix_exponential	Matrix exponential	A	exponential
matrix_square_root	Square root of a positive definite matrix	A	sqrt
polar_decomposition	Polar decomposition	A	U, P
jacobi_iteration	Jacobi iteration for Ax = b	A, b	solution
gauss_seidel	Gauss-Seidel iteration for Ax = b	A, b	solution
conjugate_gradient	Conjugate gradient method for Ax = b	A, b	solution
tridiagonal_solve	Tridiagonal system (Thomas algorithm)	sub, diag, sup, d	solution
csr_format	Compressed sparse row (CSR) form of a matrix	A	data, indices, indptr, nnz, density
csc_format	Compressed sparse column (CSC) form of a matrix	A	data, indices, indptr, nnz, density
csr_to_dense	Dense matrix from CSR arrays	data, indices, indptr, ncols	dense
sparse_matvec	Sparse matrix times vector (CSR)	data, indices, indptr, x	product
poisson_1d	1-D Poisson equation by finite differences	f, L	u, h
poisson_2d	2-D Poisson equation on a grid (5-point stencil)	f, h	u
bisection_root	Root of a polynomial by bisection	coefficients, a, b	root
newton_root	Root of a polynomial by Newton's method	coefficients, x0	root
secant_root	Root of a polynomial by the secant method	coefficients, x0, x1	root
polynomial_evaluation	Evaluate a polynomial and its derivative (Horner's method)	coefficients, x	value, derivative
lagrange_interpolation	Lagrange interpolation	x, y, at	value
newton_divided_differences	Newton divided differences	x, y	coefficients

id	operation	inputs	outputs
interpolating_polynomial	Coefficients of the interpolating polynomial	x, y	coefficients
linear_interpolation	Piecewise linear interpolation	x, y, at	value
natural_cubic_spline	Natural cubic spline interpolation	x, y, at	value
clamped_cubic_spline	Clamped cubic spline interpolation	x, y, slope_start, slope_end, at	value
trapezoid_rule	Trapezoidal rule for tabulated data	x, y	integral
simpson_rule	Simpson's rule for tabulated data	x, y	integral
midpoint_rule	Composite midpoint rule for a polynomial	coefficients, a, b, n	integral
gauss_legendre_rule	Gauss-Legendre quadrature nodes and weights	n	nodes, weights
gauss_legendre_integral	Gauss-Legendre quadrature of a polynomial	coefficients, a, b, n	integral
numerical_derivative	Derivative of equally spaced data (finite differences)	y, h	derivative

Sum of two vectors — `vector_addition`

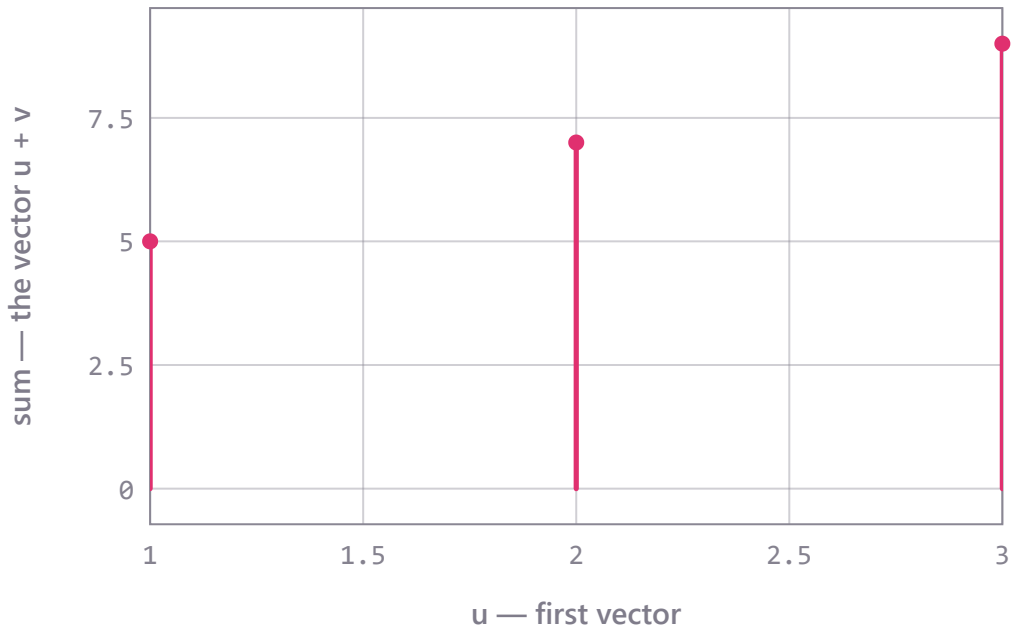
Add two vectors of the same length entry by entry.

$$\mathbf{u} + \mathbf{v} = (u_1 + v_1, \ldots, u_n + v_n)$$

input	meaning	condition
`u`	first vector	
`v`	second vector	

output	meaning
`sum`	the vector u + v

- **Method:** entry-by-entry addition (exact decimal arithmetic)
- **Assumes:** u and v have the same number of entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the vector u + v (sum) returned by linear-algebra-ai999x for the worked example, one value for each u (first vector) (3 values, first 5, last 9, largest 9). Inputs: u = 1, 2, 3, v = 4, 5, 6.

```
linear_algebra_ai999x.solve('vector_addition', u='1, 2, 3', v='4, 5, 6')
# -> sum = [5.0, 7.0, 9.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('vector_addition', u=[0.1, 0.2], v=[0.2, 0.1])
# -> sum = [0.3, 0.3]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Add the vectors (1, 2, 3) and (4, 5, 6).	SOLVED	sum = [5.0, 7.0, 9.0]
What is the sum of the vectors [2, -1] and [3, 4]?	SOLVED	sum = [5.0, 3.0]
Add the vectors (1, 2, 3) and (4, 5).	INVALID_INPUT	Sum of two vectors: u and v must have the same number of entries.

Difference of two vectors — `vector_subtraction`

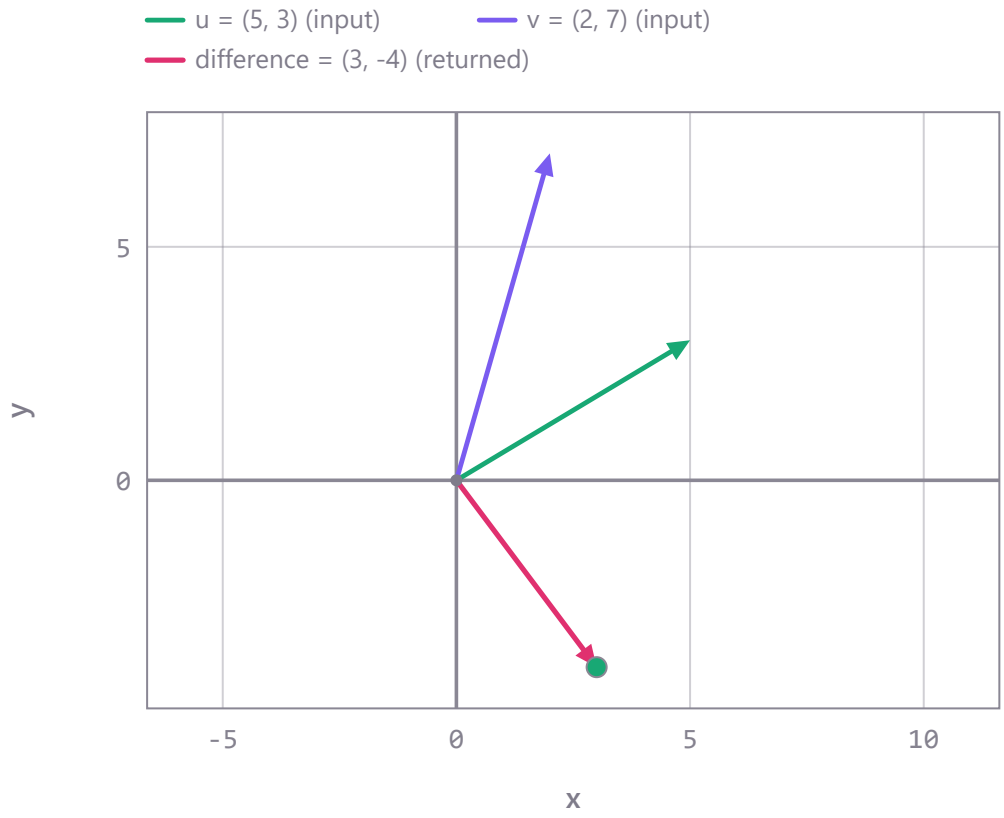
Subtract the second vector from the first, entry by entry.

$$\mathbf{u} - \mathbf{v} = (u_1 - v_1, \ldots, u_n - v_n)$$

input	meaning	condition
`u`	first vector	
`v`	vector subtracted	

output	meaning
`difference`	the vector u - v

- **Method:** entry-by-entry subtraction (exact decimal arithmetic)
- **Assumes:** u and v have the same number of entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input u = (5, 3), v = (2, 7); returned by linear-algebra-ai999x: difference = (3, -4). Marked: the tip of each returned vector.

```
linear_algebra_ai999x.solve('vector_subtraction', u='5, 3', v='2, 7')
# -> difference = [3.0, -4.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('vector_subtraction', u=[1, 2, 3], v=[4, 5, 6])
# -> difference = [-3.0, -3.0, -3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the difference of the vectors [5, 3] and [2, 7].	SOLVED	difference = [3.0, -4.0]
Compute u - v for the vectors u = (1, 2) and v = (3, 4).	SOLVED	difference = [-2.0, -2.0]
Subtract the vector (4, 5, 6) from (1, 2, 3).	SOLVED	difference = [-3.0, -3.0, -3.0]

Scalar multiple of a vector — `scalar_vector_multiplication`

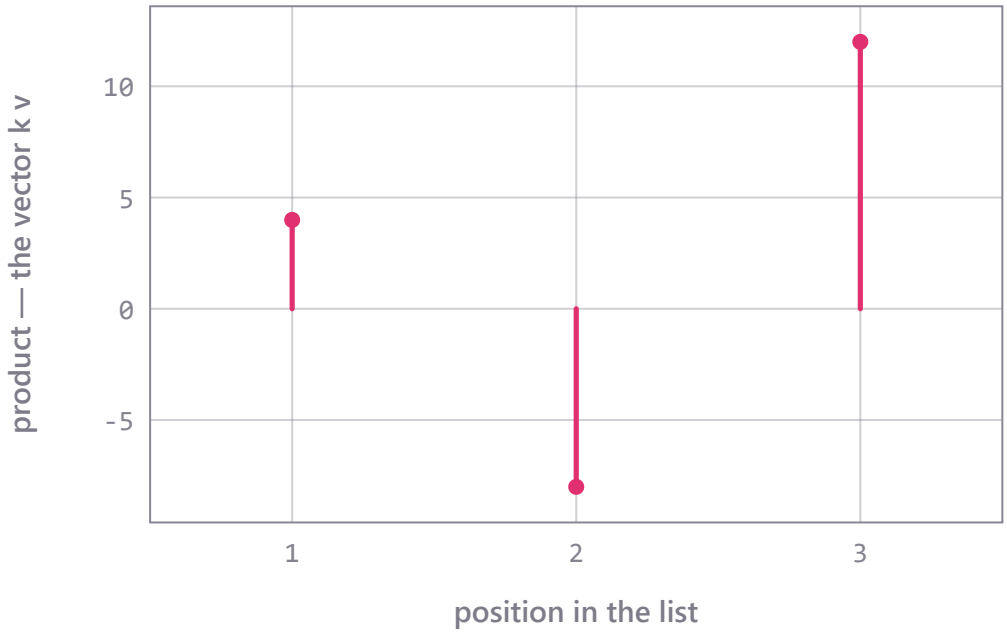
Multiply every entry of a vector by a number k.

$$k\mathbf{v} = (kv_1, \ldots, kv_n)$$

input	meaning	condition
`k`	the scalar	
`v`	the vector	

output	meaning
`product`	the vector k v

- **Method:** entry-by-entry multiplication (exact decimal arithmetic)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the vector k v (product) returned by linear-algebra-ai999x for the worked example, one value for each position in the list (3 values, first 4, last 12, largest 12). Inputs: k = 4, v = 1, -2, 3.

```
linear_algebra_ai999x.solve('scalar_vector_multiplication', k=4, v='1, -2, 3')
# -> product = [4.0, -8.0, 12.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('scalar_vector_multiplication', k=2.5, v=[2, 4, 6])
# -> product = [5.0, 10.0, 15.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiply the vector (1, -2, 3) by 4.	SOLVED	product = [4.0, -8.0, 12.0]
Compute 2.5 times the vector [2, 4, 6].	SOLVED	product = [5.0, 10.0, 15.0]
Scale the vector (3, 1) by k = -2.	SOLVED	product = [-6.0, -2.0]

Dot product of two vectors — `dot_product`

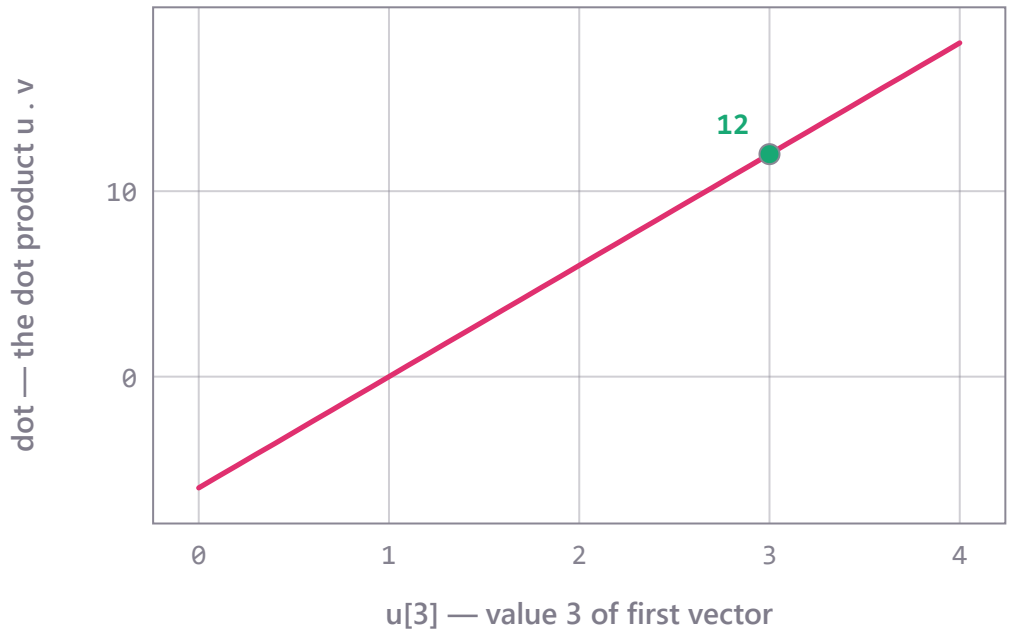
The dot (scalar, inner) product u . v = sum of u_i v_i.

$$\mathbf{u} \cdot \mathbf{v} = \sum_i u_i v_i$$

input	meaning	condition
`u`	first vector	
`v`	second vector	

output	meaning
`dot`	the dot product u . v

- **Method:** sum of entry-wise products (exact decimal arithmetic)
- **Assumes:** u and v have the same number of entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dot (the dot product u . v) as value 3 of u (first vector) moves from 0 to 4, the other values as given (1, 2, 3); 41 runs of linear-algebra-ai999x. Other inputs: v = 4, -5, 6. Marked: the worked example, u[3] = 3, dot = 12.

```
linear_algebra_ai999x.solve('dot_product', u='1, 2, 3', v='4, -5, 6')
# -> dot = 12   (SOLVED)
```

```
linear_algebra_ai999x.solve('dot_product', u=[0.1, 0.2], v=[3, 3])
# -> dot = 0.9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the dot product of (1, 2, 3) and (4, -5, 6).	SOLVED	dot = 12
What is the scalar product of the vectors [2, 0, 1] and [3, 4, -2]?	SOLVED	dot = 4

question	outcome	result
Find the dot product of (1, 2, 3) and (4, 5).	INVALID_INPUT	Dot product of two vectors: u and v must have the same number of entries.

Cross product of two 3-D vectors — `cross_product`

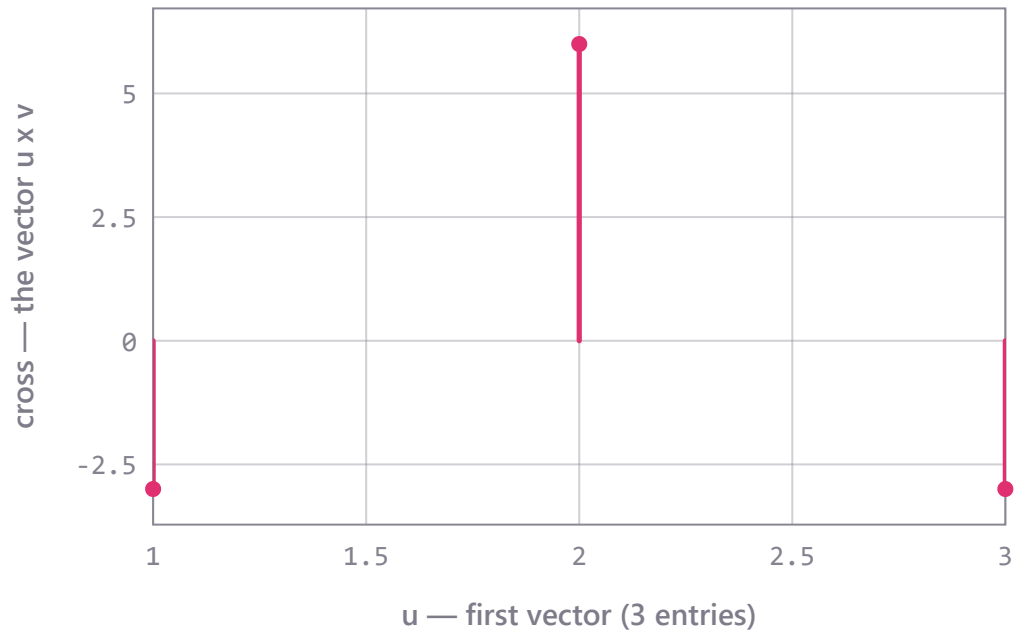
The cross product u x v of two vectors in three dimensions (right-hand rule).

$$\mathbf{u} \times \mathbf{v} = (u_2 v_3 - u_3 v_2,\ u_3 v_1 - u_1 v_3,\ u_1 v_2 - u_2 v_1)$$

input	meaning	condition
`u`	first vector (3 entries)	
`v`	second vector (3 entries)	

output	meaning
`cross`	the vector u x v

- **Method:** determinant formula (u2 $v_3 - u_3$ v2, u3 $v_1 - u_1$ v3, u1 $v_2 - u_2$ v1), exact decimal arithmetic
- **Assumes:** u and v have exactly 3 entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the vector u x v (cross) returned by linear-algebra-ai999x for the worked example, one value for each u (first vector (3 entries)) (3 values, first -3, last -3, largest 6). Inputs: u = 1, 2, 3, v = 4, 5, 6.

```
linear_algebra_ai999x.solve('cross_product', u='1, 2, 3', v='4, 5, 6')
# -> cross = [-3.0, 6.0, -3.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('cross_product', u=[1, 0, 0], v=[0, 1, 0])
# -> cross = [0.0, 0.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the cross product of (1, 2, 3) and (4, 5, 6).	SOLVED	cross = [-3.0, 6.0, -3.0]
Compute the vector product of [1, 0, 0] and [0, 1, 0].	SOLVED	cross = [0.0, 0.0, 1.0]
Find the cross product of (1, 2) and (3, 4).	INVALID_INPUT	u: needs at least 3 entries, got 2.

Length (norm) of a vector — `vector_norm`

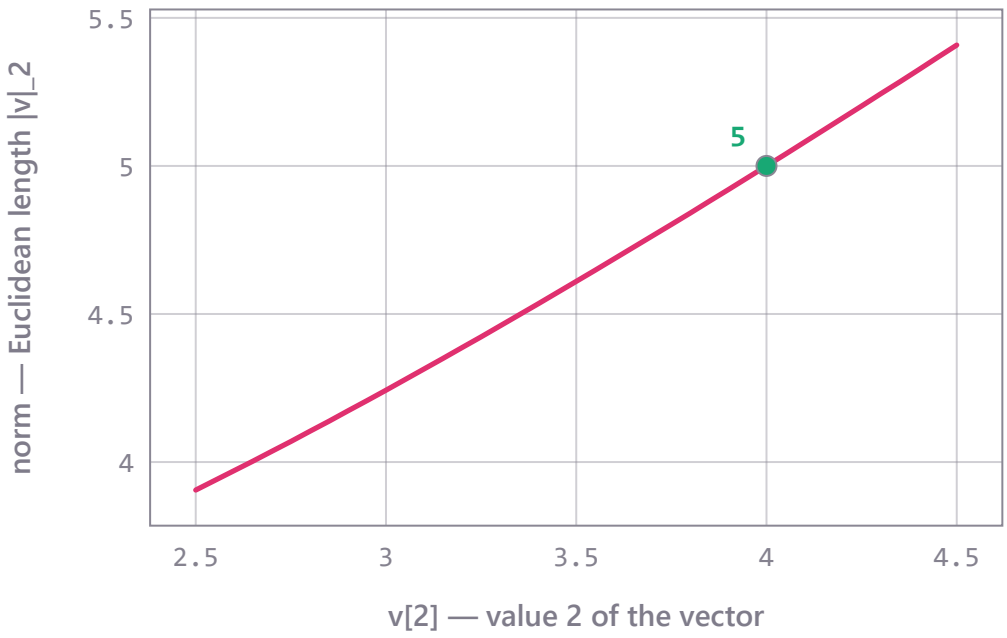
The Euclidean length |v| of a vector, with its 1-norm (sum of magnitudes) and max-norm.

$$\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}$$

input	meaning	condition
`v`	the vector	

output	meaning
`norm`	Euclidean length |v|_2
`norm1`	1-norm: sum of |v_i|
`norm_inf`	max-norm: largest |v_i|

- **Method:** square root of the sum of squares (the sum exact)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



norm (Euclidean length |v|_2) as value 2 of v (the vector) moves from 2.5 to 4.5, the other values as given (3, 4); 41 runs of linear-algebra-ai999x. Marked: the worked example, v[2] = 4, norm = 5.

```
linear_algebra_ai999x.solve('vector_norm', v='3, 4')
# -> norm = 5, norm1 = 7, norm_inf = 4   (SOLVED)
```

```
linear_algebra_ai999x.solve('vector_norm', v=[1, 2, 2])
# -> norm = 3, norm1 = 5, norm_inf = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the magnitude of the vector (3, 4).	SOLVED	norm = 5, norm1 = 7, norm_inf = 4
What is the norm of the vector [1, 2, 2]?	SOLVED	norm = 3, norm1 = 5, norm_inf = 2
Compute the length of the vector (1, -1, 1, -1).	SOLVED	norm = 2, norm1 = 4, norm_inf = 1

Unit vector in the direction of a vector — `unit_vector`

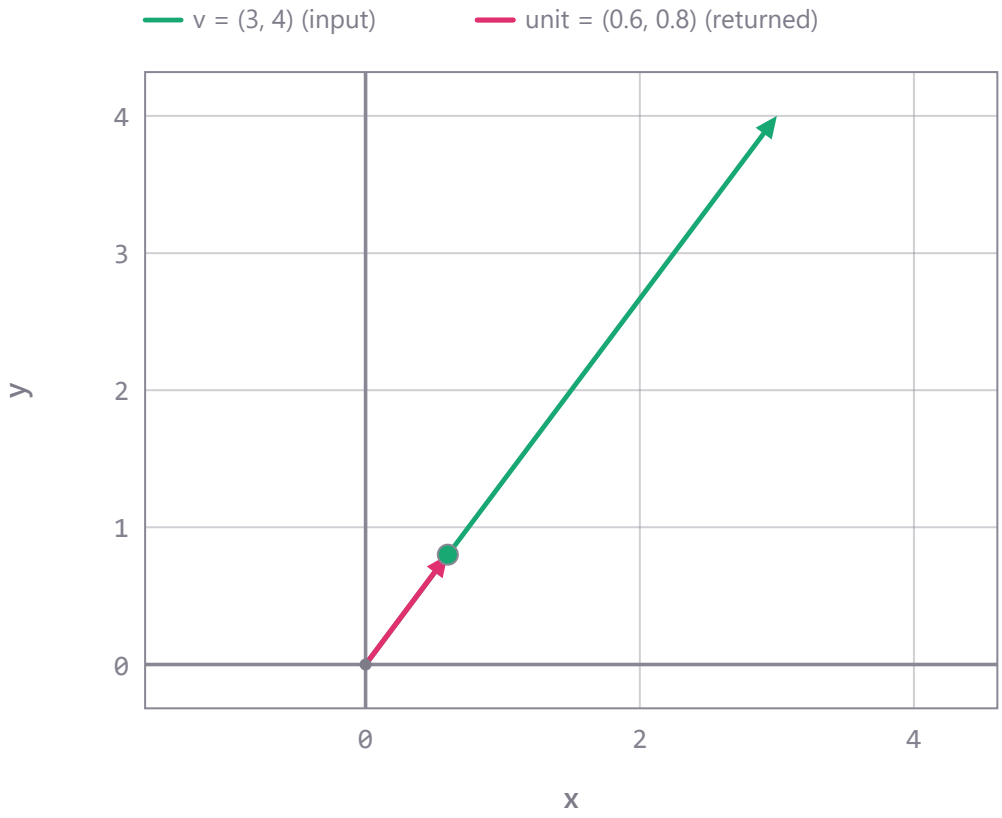
Normalize a nonzero vector: v / |v|, the vector of length 1 pointing the same way.

$$\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$$

input	meaning	condition
`v`	the vector (not all zero)	

output	meaning
`unit`	the unit vector v / |v|

- **Method:** divide each entry by the Euclidean length
- **Assumes:** v is not the zero vector
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input v = (3, 4); returned by linear-algebra-ai999x: unit = (0.6, 0.8). Marked: the tip of each returned vector.

```
linear_algebra_ai999x.solve('unit_vector', v='3, 4')
# -> unit = [0.6, 0.8]   (SOLVED)
```

```
linear_algebra_ai999x.solve('unit_vector', v=[0, 0, 5])
# -> unit = [0.0, 0.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the unit vector in the direction of (3, 4).	SOLVED	unit = [0.6, 0.8]
Normalize the vector [1, 2, 2].	SOLVED	unit = [0.3333333333333333, 0.6666666666666666, 0.6666666666666666]
Normalize the vector (0, 0, 0).	INVALID_INPUT	Unit vector in the direction of a vector: the zero vector has no direction, so it has no unit vector.

Angle between two vectors — `angle_between_vectors`

The angle between two nonzero vectors, in degrees and radians (between 0 and 180 degrees).

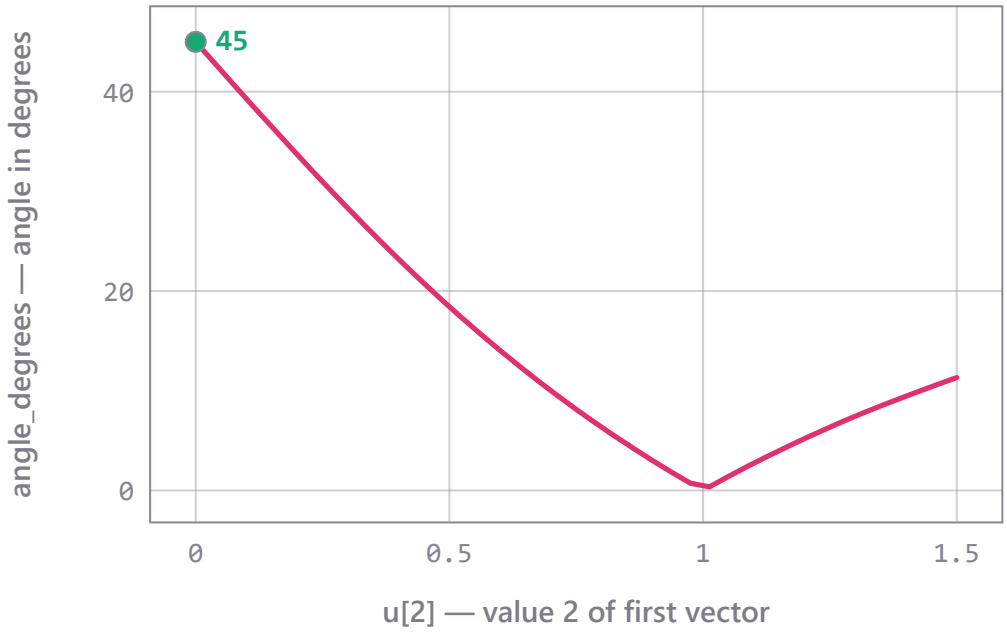
$$\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}$$

input	meaning	condition
u	first vector	

input	meaning	condition
v	second vector	

output	meaning
angle_degrees	angle in degrees
angle_radians	angle in radians

- **Method:** $\theta = 2$ atan2(|a - b|, |a + b|) with a, b the unit vectors (accurate also near 0 and 180 degrees)
- **Assumes:** u and v have the same number of entries; neither vector is zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



angle_degrees (angle in degrees) as value 2 of u (first vector) moves from 0 to 1.5, the other values as given (1, 0); 41 runs of linear-algebra-ai999x. Other inputs: v = 1, 1. Marked: the worked example, u[2] = 0, angle_degrees = 45.

```
linear_algebra_ai999x.solve('angle_between_vectors', u='1, 0', v='1, 1')
# -> angle_degrees = 45, angle_radians = 0.785398   (SOLVED)
```

```
linear_algebra_ai999x.solve('angle_between_vectors', u=[1, 0, 0], v=[0, 1, 0])
# -> angle_degrees = 90, angle_radians = 1.5708   (SOLVED)
```

```
linear_algebra_ai999x.solve('angle_between_vectors', u=[1, 2], v=[-2, -4])
# -> angle_degrees = 180, angle_radians = 3.14159   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the angle between the vectors (1, 0) and (1, 1).	SOLVED	angle_degrees = 45, angle_radians = 0.785398
What is the angle between [1, 0, 0] and [0, 0, 2]?	SOLVED	angle_degrees = 90, angle_radians = 1.5708

question	outcome	result
Find the angle between the vectors (0, 0) and (1, 1).	INVALID_INPUT	Angle between two vectors: the angle with the zero vector is undefined.

Distance between two points — `distance_between_points`

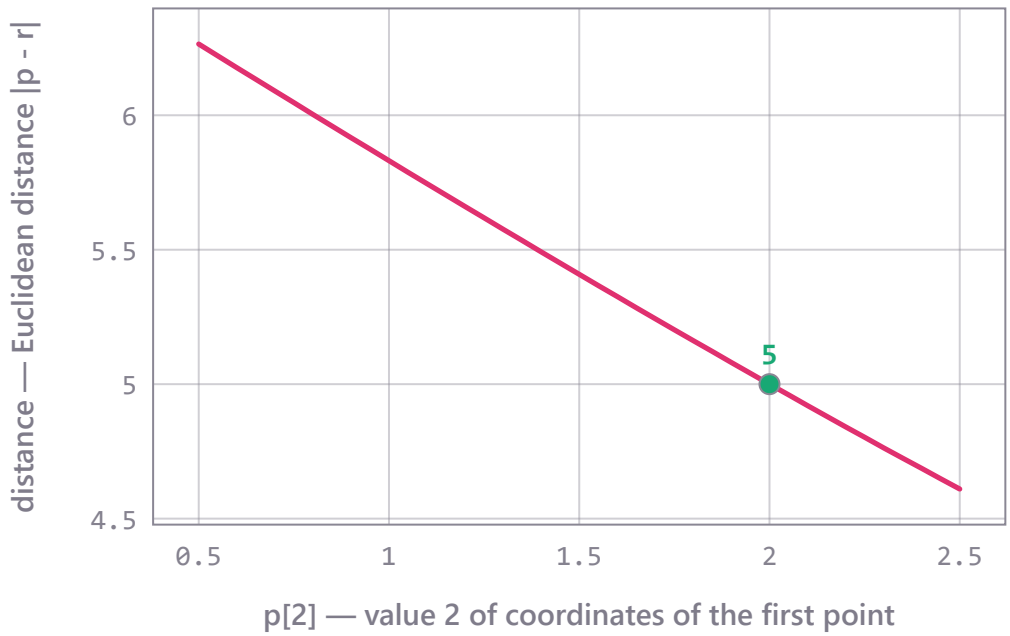
The Euclidean distance between two points given by their coordinates (any dimension).

$$d = \sqrt{\sum_i (p_i - r_i)^2}$$

input	meaning	condition
`p`	coordinates of the first point	
`r`	coordinates of the second point	

output	meaning
`distance`	Euclidean distance |p - r|

- **Method:** square root of the sum of squared coordinate differences (the sum exact)
- **Assumes:** the points have the same number of coordinates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



distance (Euclidean distance |p - r|) as value 2 of p (coordinates of the first point) moves from 0.5 to 2.5, the other values as given (1, 2); 41 runs of linear-algebra-ai999x. Other inputs: r = 4, 6. Marked: the worked example, p[2] = 2, distance = 5.

```
linear_algebra_ai999x.solve('distance_between_points', p='1, 2', r='4, 6')
# -> distance = 5   (SOLVED)
```

```
linear_algebra_ai999x.solve('distance_between_points', p=[0, 0, 0], r=[2, 3, 6])
# -> distance = 7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the distance between the points (1, 2) and (4, 6).	SOLVED	distance = 5
What is the distance between (0, 0, 0) and (2, 3, 6)?	SOLVED	distance = 7

Scalar triple product (volume of a parallelepiped) — `triple_scalar_product`

u . (v x w), the signed volume of the parallelepiped spanned by three $3-D$ vectors, and its absolute value.

$$\mathbf{u}\cdot(\mathbf{v}\times\mathbf{w})=\det[\mathbf{u};\mathbf{v};\mathbf{w}]$$

input	meaning	condition
`u`	first vector (3 entries)	
`v`	second vector (3 entries)	
`w`	third vector (3 entries)	

output	meaning
`triple`	u . (v x w)
`volume`	volume |u . (v x w)|

- **Method:** the determinant of the matrix with rows u, v, w (exact)
- **Assumes:** each vector has exactly 3 entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Scalar triple product (volume of a parallelepiped) as inputs and results: stated u = 1, 0, 0, v = 0, 1, 0, w = 0, 0, 1; returned by linear-algebra-ai999x: triple = 1, volume = 1.

```
linear_algebra_ai999x.solve('triple_scalar_product', u='1, 0, 0', v='0, 1, 0', w='0, 0, 1')
# -> triple = 1, volume = 1   (SOLVED)
```

```
linear_algebra_ai999x.solve('triple_scalar_product', u=[1, 2, 3], v=[0, 1, 4], w=[5, 6, 0])
# -> triple = 1, volume = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the scalar triple product of (1, 0, 0), (0, 1, 0) and (0, 0, 1).	SOLVED	triple = 1, volume = 1
What is the volume of the parallelepiped spanned by (1, 2, 3), (0, 1, 4) and (5, 6, 0)?	SOLVED	triple = 1, volume = 1

Area of the parallelogram (and triangle) spanned by two vectors — `parallelogram_area`

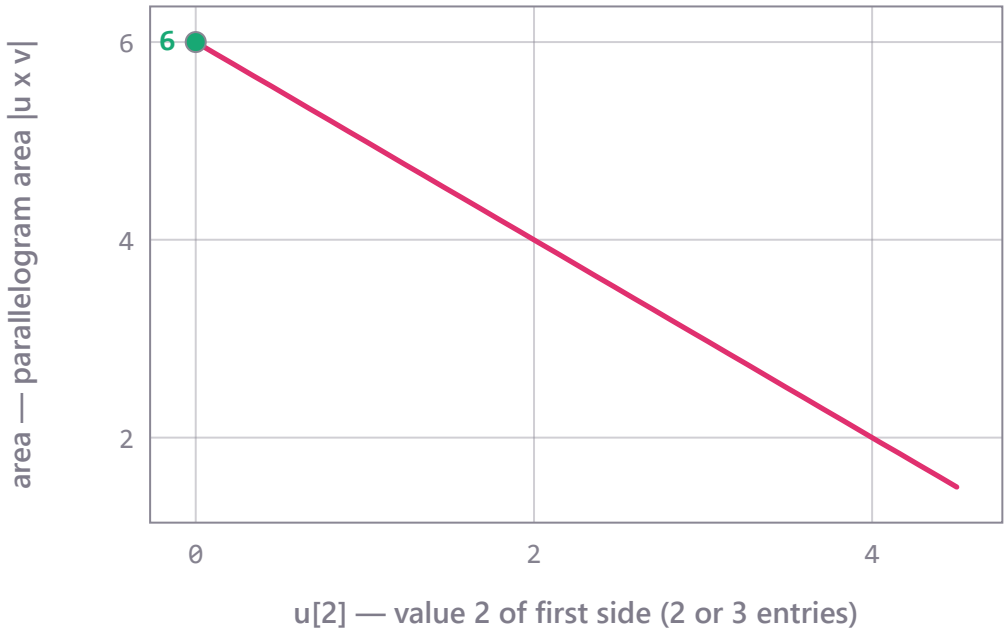
|u x v|: the area of the parallelogram with sides u and v (2-D or 3-D), and half of it for the triangle.

$$A = \|\mathbf{u} \times \mathbf{v}\| = \sqrt{\|\mathbf{u}\|^2\|\mathbf{v}\|^2 - (\mathbf{u} \cdot \mathbf{v})^2}$$

input	meaning	condition
`u`	first side (2 or 3 entries)	
`v`	second side (2 or 3 entries)	

output	meaning
`area`	parallelogram area |u x v|
`triangle_area`	triangle area |u x v| / 2

- **Method:** Lagrange's identity |u x v|^2 = |u|^2 |v|^2 - (u . v)^2 (exact under the root)
- **Assumes:** u and v have the same number of entries, 2 or 3
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (parallelogram area |u x v|) as value 2 of u (first side (2 or 3 entries)) moves from 0 to 4.5, the other values as given (3, 0); 41 runs of linear-algebra-ai999x. Other inputs: v = 1, 2. Marked: the worked example, u[2] = 0, area = 6.

```
linear_algebra_ai999x.solve('parallelogram_area', u='3, 0', v='1, 2')
# -> area = 6, triangle_area = 3   (SOLVED)
```

```
linear_algebra_ai999x.solve('parallelogram_area', u=[1, 0, 0], v=[0, 2, 0])
# -> area = 2, triangle_area = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of the parallelogram spanned by (3, 0) and (1, 2).	SOLVED	area = 6, triangle_area = 3
What is the area of the triangle with sides (1, 0, 0) and (0, 2, 0)?	SOLVED	area = 2, triangle_area = 1

Projection of one vector onto another — `vector_projection`

The orthogonal projection of u onto v, (u.v / v.v) v, and the scalar projection (component) u.v / |v|.

$$\text{proj}_{\mathbf{v}}\mathbf{u} = \frac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}\,\mathbf{v}$$

input	meaning	condition
`u`	vector to project	
`v`	direction projected onto (not zero)	

output	meaning
`projection`	vector projection of u onto v
`scalar_projection`	scalar projection u.v / |v|

- **Method:** proj = (u.v / v.v) v in exact decimal arithmetic
- **Assumes:** u and v have the same number of entries; v is not the zero vector
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input u = (2, 3), v = (1, 0); returned by linear-algebra-ai999x: projection = (2, 0); also returned scalar_projection = 2. Marked: the tip of each returned vector.

```
linear_algebra_ai999x.solve('vector_projection', u='2, 3', v='1, 0')
# -> projection = [2.0, 0.0], scalar_projection = 2   (SOLVED)
```

```
linear_algebra_ai999x.solve('vector_projection', u=[1, 2, 3], v=[1, 1, 1])
# -> projection = [2.0, 2.0, 2.0], scalar_projection = 3.4641   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the projection of (2, 3) onto (1, 0).	SOLVED	projection = [2.0, 0.0], scalar_projection = 2
Project the vector [1, 2, 3] onto [1, 1, 1].	SOLVED	projection = [2.0, 2.0, 2.0], scalar_projection = 3.4641
Project the vector [1, 2] onto [0, 0].	INVALID_INPUT	Projection of one vector onto another: cannot project onto the zero vector.

Rotation of a plane vector — `rotate_vector_2d`

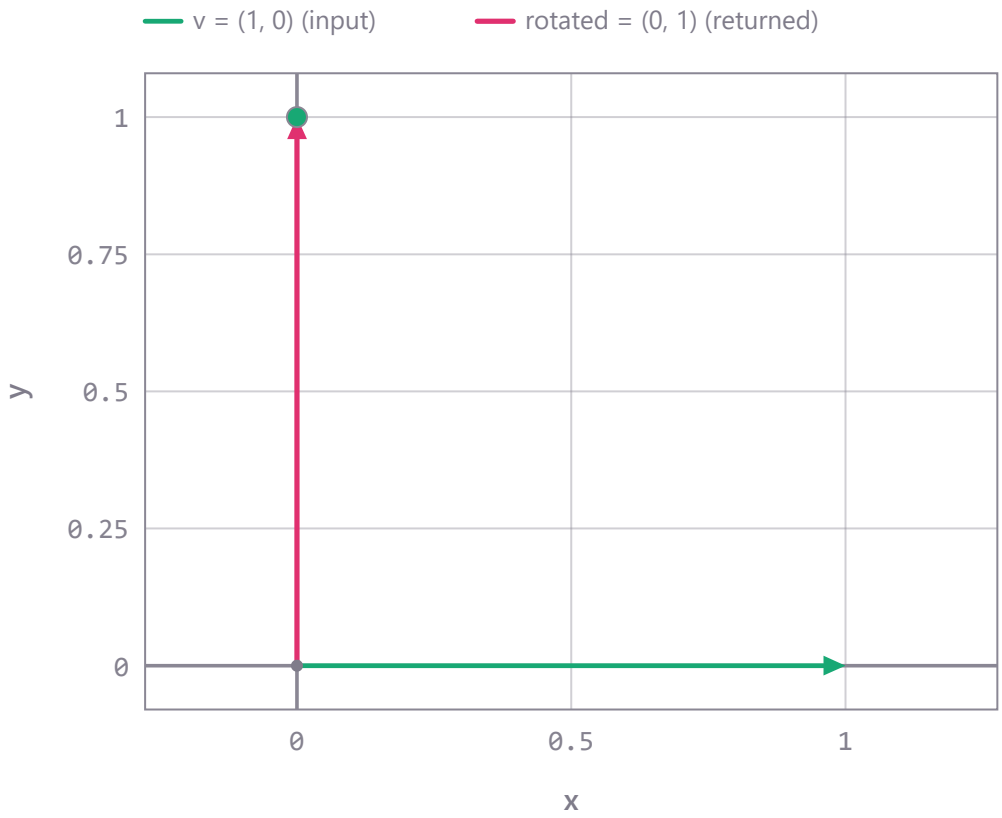
Rotate a $2-D$ vector counterclockwise about the origin by an angle given in degrees.

$$R(\theta)\mathbf{v} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}\mathbf{v}$$

input	meaning	condition
`v`	the vector (2 entries)	
`angle`	rotation angle in degrees (counterclockwise)	

output	meaning
`rotated`	the rotated vector R(angle) v

- **Method:** multiply by the rotation matrix [[cos t, -sin t], [sin t, cos t]] (exact at multiples of 90 degrees)
- **Assumes:** v has 2 entries; a positive angle turns counterclockwise
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input v = (1, 0); returned by linear-algebra-ai999x: rotated = (0, 1). Other inputs: angle = 90. Marked: the tip of each returned vector.

```
linear_algebra_ai999x.solve('rotate_vector_2d', v='1, 0', angle=90)
# -> rotated = [0.0, 1.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('rotate_vector_2d', v=[1, 1], angle=-45)
# -> rotated = [1.414213562373095, -3.3306690738754696e-16]   (SOLVED)
```

```
linear_algebra_ai999x.solve('rotate_vector_2d', v=[3, 4], angle=180)
# -> rotated = [-3.0, -4.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rotate the vector (1, 0) by 90 degrees.	SOLVED	rotated = [0.0, 1.0]

question	outcome	result
Rotate the vector (1, 1) clockwise by 45 degrees.	SOLVED	rotated = [1.414213562373095, -3.3306690738754696e-16]
Rotate the vector (1, 2, 3) by 90 degrees.	INVALID_INPUT	v: allows at most 2 entries, got 3.

Sum of two matrices — `matrix_addition`

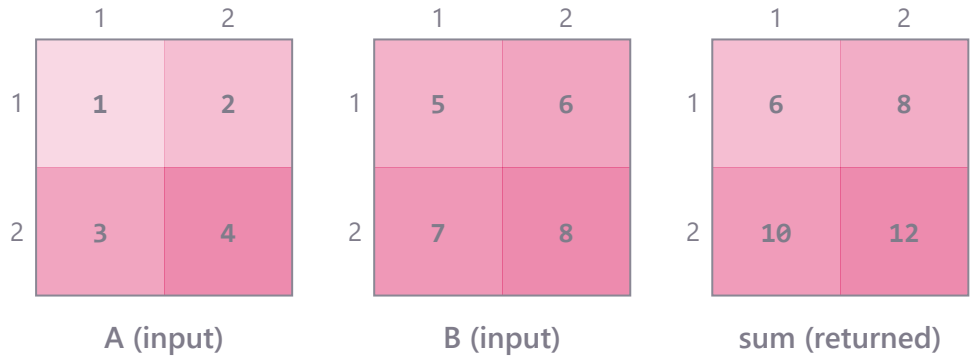
Add two matrices of the same shape entry by entry.

$$(A + B)_{ij} = a_{ij} + b_{ij}$$

input	meaning	condition
`A`	first matrix	
`B`	second matrix	

output	meaning
`sum`	the matrix A + B

- **Method:** entry-by-entry addition (exact decimal arithmetic)
- **Assumes:** A and B have the same shape
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); B (input); sum (returned).

```
linear_algebra_ai999x.solve('matrix_addition', A='[[1, 2], [3, 4]]', B='[[5, 6], [7, 8]]')
# -> sum = [[6.0, 8.0], [10.0, 12.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_addition', A=[[0.1, 0.2, 0.3]], B=[[0.2, 0.1, 0.0]])
# -> sum = [[0.3, 0.3, 0.3]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Add the matrices [[1, 2], [3, 4]] and [[5, 6], [7, 8]].	SOLVED	sum = [[6.0, 8.0], [10.0, 12.0]]

question	outcome	result
What is the sum of the matrices [1 2; 3 4] and [0 1; 1 0]?	SOLVED	sum = [[1.0, 3.0], [4.0, 4.0]]
Add the matrices [[1, 2], [3, 4]] and [[1, 2, 3]].	INVALID_INPUT	Sum of two matrices: A and B must have the same shape.

Difference of two matrices — `matrix_subtraction`

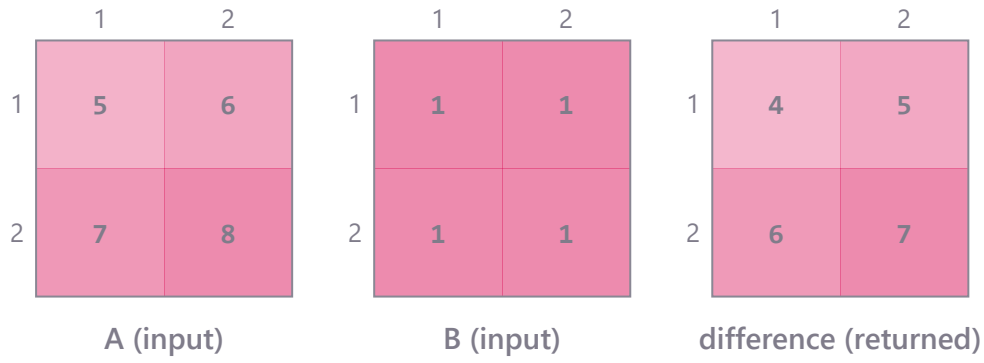
Subtract the second matrix from the first, entry by entry.

$$(A - B)_{ij} = a_{ij} - b_{ij}$$

input	meaning	condition
`A`	first matrix	
`B`	matrix subtracted	

output	meaning
`difference`	the matrix A - B

- **Method:** entry-by-entry subtraction (exact decimal arithmetic)
- **Assumes:** A and B have the same shape
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); B (input); difference (returned).

```
linear_algebra_ai999x.solve('matrix_subtraction', A='[[5, 6], [7, 8]]', B='[[1, 1], [1, 1]]')
# -> difference = [[4.0, 5.0], [6.0, 7.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_subtraction', A=[[4, 3], [2, 1]], B=[[1, 2], [3, 4]])
# -> difference = [[3.0, 1.0], [-1.0, -3.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Subtract the matrix [[1, 1], [1, 1]] from [[5, 6], [7, 8]].	SOLVED	difference = [[4.0, 5.0], [6.0, 7.0]]

question	outcome	result
Compute A - B for A = [[4, 3], [2, 1]] and B = [[1, 2], [3, 4]].	SOLVED	difference = [[3.0, 1.0], [-1.0, -3.0]]

Scalar multiple of a matrix — `scalar_matrix_multiplication`

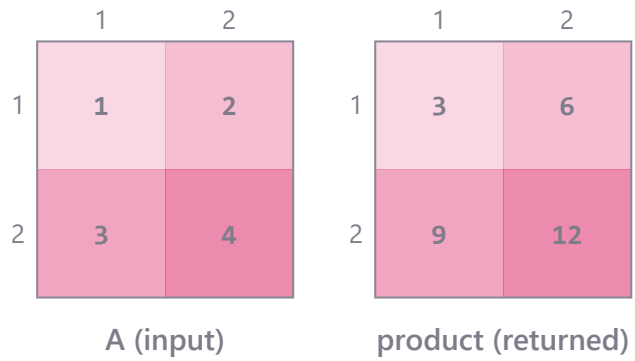
Multiply every entry of a matrix by a number k.

$$(kA)_{ij} = k\,a_{ij}$$

input	meaning	condition
`k`	the scalar	
`A`	the matrix	

output	meaning
`product`	the matrix k A

- **Method:** entry-by-entry multiplication (exact decimal arithmetic)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); product (returned). Other inputs: k = 3.

```
linear_algebra_ai999x.solve('scalar_matrix_multiplication', k=3, A='[[1, 2], [3, 4]]')
# -> product = [[3.0, 6.0], [9.0, 12.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('scalar_matrix_multiplication', k=0.5, A=[[2, 4], [6, 8]])
# -> product = [[1.0, 2.0], [3.0, 4.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiply the matrix [[1, 2], [3, 4]] by 3.	SOLVED	product = [[3.0, 6.0], [9.0, 12.0]]
Compute 0.5 times the matrix [2 4; 6 8].	SOLVED	product = [[1.0, 2.0], [3.0, 4.0]]

Transpose of a matrix — `matrix_transpose`

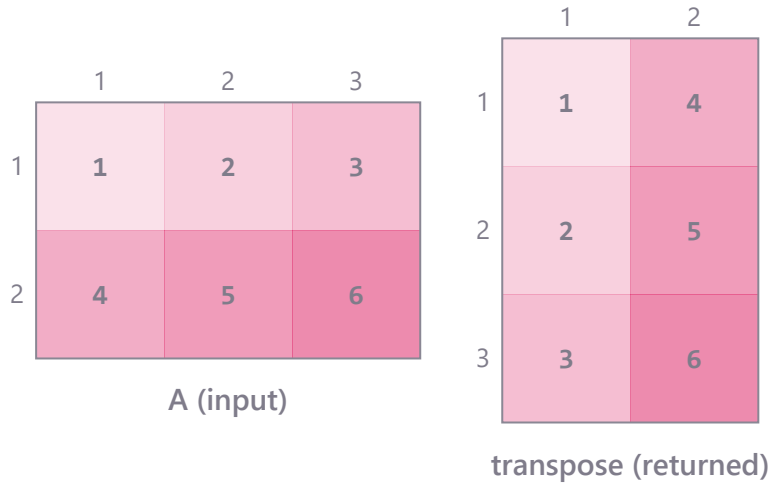
Swap rows and columns: the transpose A^T with (A^T)$_{ij} = a_{ji}$.

$$(A^T)_{ij} = a_{ji}$$

input	meaning	condition
`A`	the matrix	

output	meaning
`transpose`	the matrix A^T

- **Method:** row i of A becomes column i
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); transpose (returned).

```
linear_algebra_ai999x.solve('matrix_transpose', A='[[1, 2, 3], [4, 5, 6]]')
# -> transpose = [[1.0, 4.0], [2.0, 5.0], [3.0, 6.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_transpose', A=[[1, 2], [3, 4]])
# -> transpose = [[1.0, 3.0], [2.0, 4.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the transpose of [[1, 2, 3], [4, 5, 6]].	SOLVED	transpose = [[1.0, 4.0], [2.0, 5.0], [3.0, 6.0]]
Transpose the matrix [1 2; 3 4].	SOLVED	transpose = [[1.0, 3.0], [2.0, 4.0]]

Trace of a square matrix — `matrix_trace`

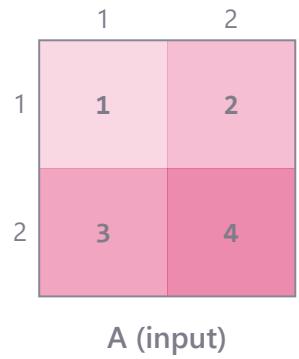
The trace: the sum of the diagonal entries of a square matrix.

$$\operatorname{tr} A = \sum_i a_{ii}$$

input	meaning	condition
`A`	a square matrix	

output	meaning
`trace`	tr(A) = sum of a_ii

- **Method:** sum of the diagonal (exact)
- **Assumes:** A is square
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: trace = 5.

```
linear_algebra_ai999x.solve('matrix_trace', A='[[1, 2], [3, 4]]')
# -> trace = 5   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_trace', A=[[2, 0, 1], [3, 5, 0], [1, 1, -4]])
# -> trace = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the trace of [[1, 2], [3, 4]].	SOLVED	trace = 5
What is the trace of the matrix [2 0 1; 3 5 0; 1 1 -4]?	SOLVED	trace = 3
Find the trace of [[1, 2, 3], [4, 5, 6]].	INVALID_INPUT	Trace of a square matrix: A must be square.

Product of two matrices — `matrix_multiplication`

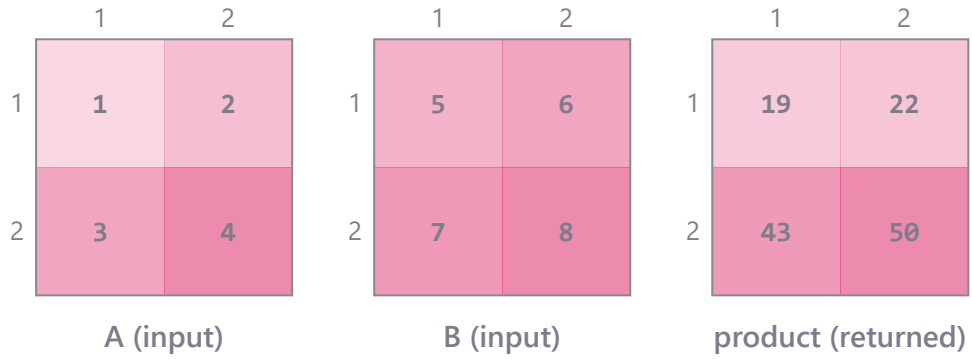
The matrix product AB (rows of A times columns of B).

$$(AB)_{ij} = \sum_k a_{ik} b_{kj}$$

input	meaning	condition
`A`	left matrix	
`B`	right matrix	

output	meaning
`product`	the matrix AB

- **Method:** row-by-column sums in exact decimal arithmetic
- **Assumes:** columns of A = rows of B; at most 10 rows and columns
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); B (input); product (returned).

```
linear_algebra_ai999x.solve('matrix_multiplication', A='[[1, 2], [3, 4]]', B='[[5, 6], [7, 8]]')
# -> product = [[19.0, 22.0], [43.0, 50.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_multiplication', A=[[1, 0, 2]], B=[[1], [2], [3]])
# -> product = [[7.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiply the matrices [[1, 2], [3, 4]] and [[5, 6], [7, 8]].	SOLVED	product = [[19.0, 22.0], [43.0, 50.0]]
What is the product of [1 2; 3 4] and [0 1; 1 0]?	SOLVED	product = [[2.0, 1.0], [4.0, 3.0]]
Multiply the matrices [[1, 2], [3, 4]] and [[1, 2, 3]].	INVALID_INPUT	Product of two matrices: the number of columns of A must equal the number of rows of B.

Matrix times vector (image under a linear map) — `matrix_vector_product`

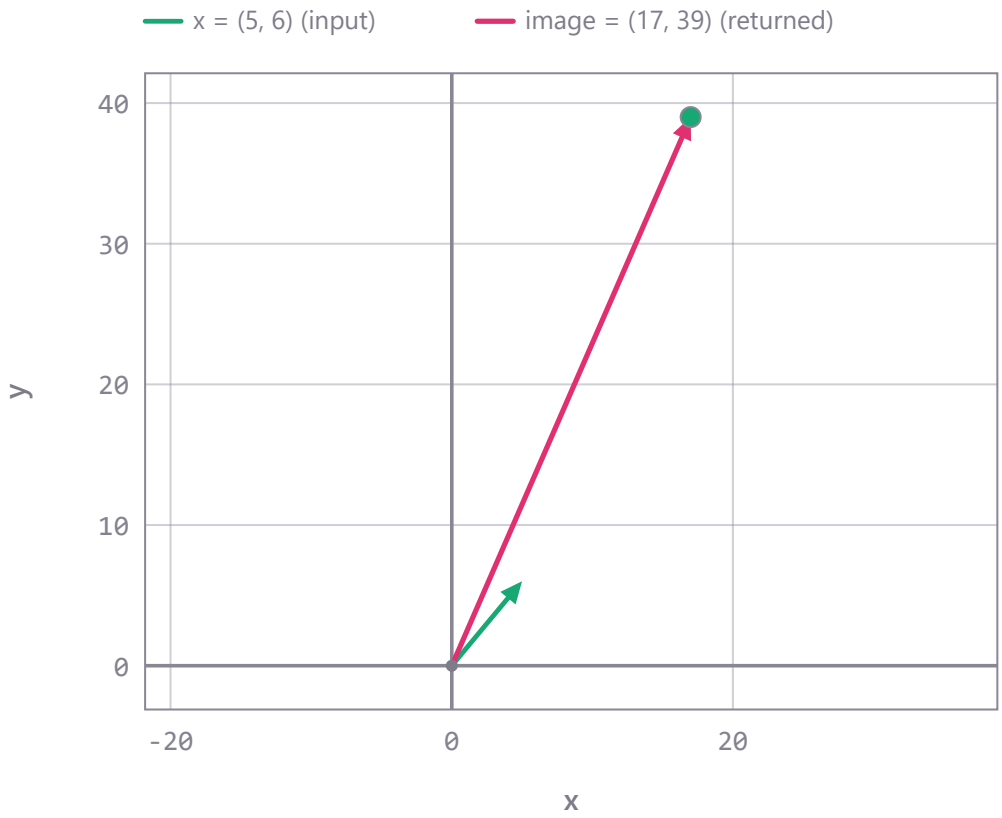
The vector Ax: the image of x under the linear transformation with matrix A.

$$(A\mathbf{x})_i = \sum_j a_{ij} x_j$$

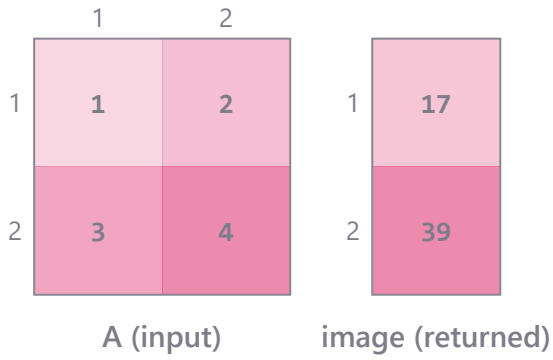
input	meaning	condition
`A`	the matrix	
`x`	the vector	

output	meaning
`image`	the vector Ax

- **Method:** dot product of each row with x (exact decimal arithmetic)
- **Assumes:** columns of A = length of x
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input x = (5, 6); returned by linear-algebra-ai999x: image = (17, 39). Other inputs: A = [[1, 2], [3, 4]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); image (returned). Other inputs: x = 5, 6.

```
linear_algebra_ai999x.solve('matrix_vector_product', A='[[1, 2], [3, 4]]', x='5, 6')
# -> image = [17.0, 39.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_vector_product', A=[[1, 2, 3], [4, 5, 6]], x=[1, 0, -1])
# -> image = [-2.0, -2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiply the matrix [[1, 2], [3, 4]] by the vector (5, 6).	SOLVED	image = [17.0, 39.0]
Find the image of the vector [1, 0, -1] under the matrix [1 2 3; 4 5 6].	SOLVED	image = [-2.0, -2.0]
Multiply the matrix [[1, 2], [3, 4]] by the vector (5, 6, 7).	INVALID_INPUT	Matrix times vector (image under a linear map): the number of columns of A must equal the length of x.

Outer product of two vectors — `outer_product`

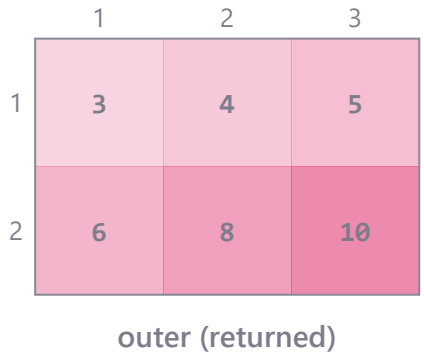
The matrix u v^T with entries u_i v_j.

$$(\mathbf{u}\mathbf{v}^T)_{ij} = u_i v_j$$

input	meaning	condition
`u`	first vector (rows)	
`v`	second vector (columns)	

output	meaning
`outer`	the matrix u v^T

- **Method:** entry (i, j) = u_i v_j (exact)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



outer (returned)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): outer (returned). Other inputs: u = 1, 2, v = 3, 4, 5.

```
linear_algebra_ai999x.solve('outer_product', u='1, 2', v='3, 4, 5')
# -> outer = [[3.0, 4.0, 5.0], [6.0, 8.0, 10.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('outer_product', u=[1, -1], v=[2, 3])
# -> outer = [[2.0, 3.0], [-2.0, -3.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the outer product of (1, 2) and (3, 4, 5).	SOLVED	outer = [[3.0, 4.0, 5.0], [6.0, 8.0, 10.0]]

Kronecker (tensor) product of two matrices — `kronecker_product`

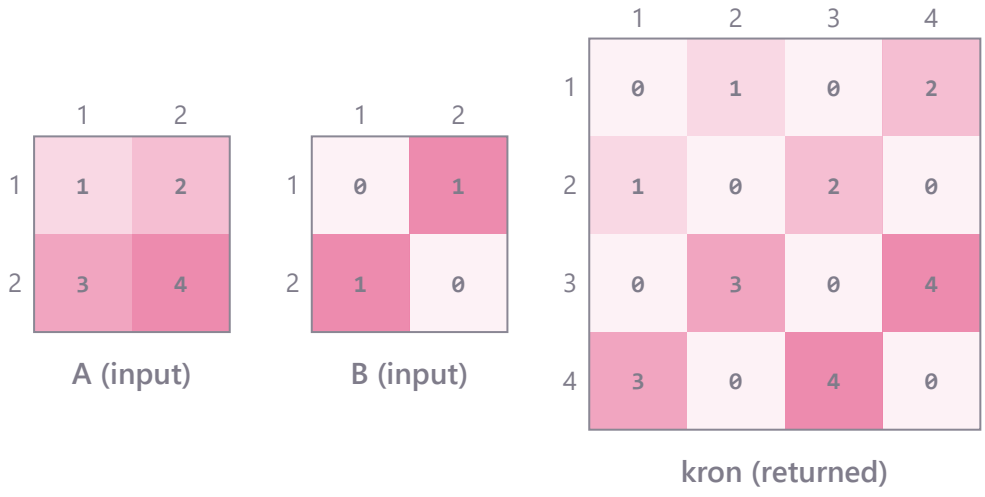
The block matrix A (x) B whose (i, j) block is a_ij B.

$$A \otimes B = [a_{ij}B]$$

input	meaning	condition
`A`	first matrix	
`B`	second matrix	

output	meaning
`kron`	the Kronecker product A (x) B

- **Method:** block (i, j) = a_ij B (exact)
- **Assumes:** each matrix at most 6 x 6
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); B (input); kron (returned).

```
linear_algebra_ai999x.solve('kronecker_product', A='[[1, 2], [3, 4]]', B='[[0, 1], [1, 0]]')
# -> kron = [[0.0, 1.0, 0.0, 2.0], [1.0, 0.0, 2.0, 0.0], [0.0, 3.0, 0.0, 4.0], [3.0, 0.0, 4.0,
0.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('kronecker_product', A=[[1, 2]], B=[[1], [1]])
# -> kron = [[1.0, 2.0], [1.0, 2.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Kronecker product of [[1, 2], [3, 4]] and [[0, 1], [1, 0]].	SOLVED	kron = [[0.0, 1.0, 0.0, 2.0], [1.0, 0.0, 2.0, 0.0], [0.0, 3.0, 0.0, 4.0], [3.0, 0.0, 4.0, 0.0]]

Power of a square matrix — `matrix_power`

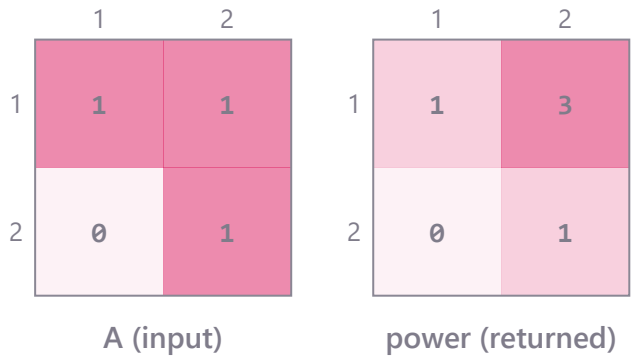
A^k for a square matrix A and a whole number k $(A^0 = I)$.

$$A^k = \underbrace{AA\cdots A}_{k}$$

input	meaning	condition
`A`	a square matrix (at most 6 x 6)	
`k`	the exponent (0 to 64)	

output	meaning
`power`	the matrix A^k

- **Method:** binary powering (repeated squaring) in exact decimal arithmetic
- **Assumes:** A is square; k is a whole number, $0 \le k \le 64$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); power (returned). Other inputs: k = 3.

```
linear_algebra_ai999x.solve('matrix_power', A='[[1, 1], [0, 1]]', k=3)
# -> power = [[1.0, 3.0], [0.0, 1.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_power', A=[[1, 2], [3, 4]], k=2)
# -> power = [[7.0, 10.0], [15.0, 22.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_power', A=[[1, 2], [3, 4]], k=0)
# -> power = [[1.0, 0.0], [0.0, 1.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute A^3 for A = [[1, 1], [0, 1]].	SOLVED	power = [[1.0, 3.0], [0.0, 1.0]]
Find the square of the matrix [1 2; 3 4].	SOLVED	power = [[7.0, 10.0], [15.0, 22.0]]
Raise the matrix [[2, 0], [1, 1]] to the 5th power.	SOLVED	power = [[32.0, 0.0], [31.0, 1.0]]
Compute A^2 for A = [[1, 2, 3], [4, 5, 6]].	INVALID_INPUT	Power of a square matrix: A must be square.

Determinant of a 2 x 2 matrix — `determinant_2x2`

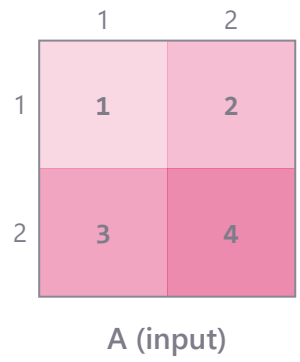
det [[a, b], [c, d]] = ad - bc.

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

input	meaning	condition
`A`	a 2 x 2 matrix	

output	meaning
`det`	the determinant ad - bc

- **Method:** ad - bc (exact decimal arithmetic)
- **Assumes:** A is 2 x 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: det = -2.

```
linear_algebra_ai999x.solve('determinant_2x2', A='[[1, 2], [3, 4]]')
# -> det = -2   (SOLVED)
```

```
linear_algebra_ai999x.solve('determinant_2x2', A=[[3, 8], [4, 6]])
# -> det = -14   (SOLVED)
```

```
linear_algebra_ai999x.solve('determinant_2x2', A=[[0.1, 0.2], [0.3, 0.4]])
# -> det = -0.02   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the determinant of [[1, 2], [3, 4]].	SOLVED	det = -2
What is det [3 8; 4 6]?	SOLVED	det = -14

Determinant of a 3 x 3 matrix — `determinant_3x3`

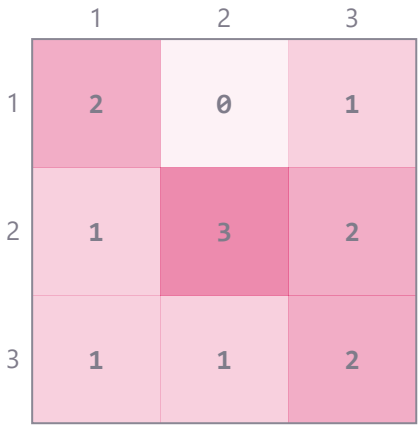
The determinant of a 3 x 3 matrix by the rule of Sarrus.

$$\det A = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{13}a_{22}a_{31} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33}$$

input	meaning	condition
`A`	a 3 x 3 matrix	

output	meaning
`det`	the determinant

- **Method:** rule of Sarrus: three down-diagonal products minus three up-diagonal products (exact)
- **Assumes:** A is 3 x 3
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: det = 6.

```
linear_algebra_ai999x.solve('determinant_3x3', A='[[2, 0, 1], [1, 3, 2], [1, 1, 2]]')
# -> det = 6   (SOLVED)
```

```
linear_algebra_ai999x.solve('determinant_3x3', A=[[1, 2, 3], [4, 5, 6], [7, 8, 10]])
# -> det = -3   (SOLVED)
```

```
linear_algebra_ai999x.solve('determinant_3x3', A=[[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# -> det = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the determinant of [[2, 0, 1], [1, 3, 2], [1, 1, 2]].	SOLVED	det = 6
Compute det [1 2 3; 4 5 6; 7 8 10].	SOLVED	det = -3

Determinant of an n x n matrix — `determinant`

The determinant of a square matrix of any size up to 10 x 10, by row reduction.

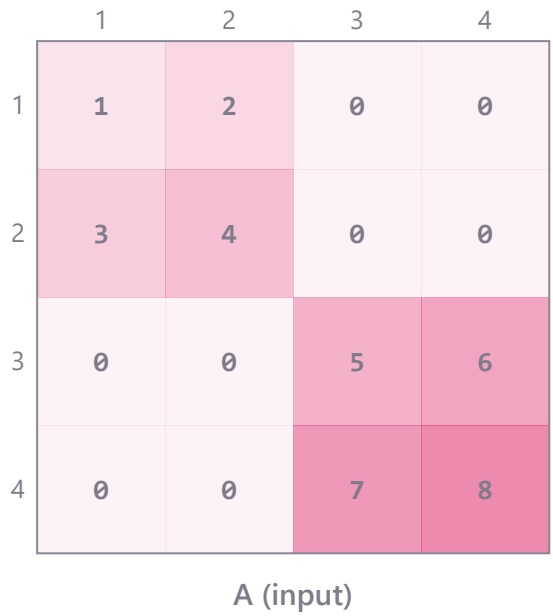
$$\det A = (-1)^s \prod_i u_{ii}$$

input	meaning	condition
`A`	a square matrix (up to 10 x 10)	

output	meaning
`det`	the determinant

- **Method:** Gaussian elimination with partial pivoting in exact rational arithmetic: det = (-1)^swaps x product of pivots
- **Assumes:** A is square; at most 10 x 10

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: det = 4.

```
linear_algebra_ai999x.solve('determinant', A='[[1, 2, 0, 0], [3, 4, 0, 0], [0, 0, 5, 6], [0, 0, 7, 8]]')
# -> det = 4   (SOLVED)
```

```
linear_algebra_ai999x.solve('determinant', A=[[2, 1, 0, 0], [1, 2, 1, 0], [0, 1, 2, 1], [0, 0, 1, 2]])
# -> det = 5   (SOLVED)
```

```
linear_algebra_ai999x.solve('determinant', A=[[1, 2], [3, 4]])
# -> det = -2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the determinant of [[1, 2, 0, 0], [3, 4, 0, 0], [0, 0, 5, 6], [0, 0, 7, 8]].	SOLVED	det = 4
Compute det [2 1 0 0; 1 2 1 0; 0 1 2 1; 0 0 1 2].	SOLVED	det = 5
Find the determinant of [[1, 2, 3], [4, 5, 6]].	INVALID_INPUT	Determinant of an n x n matrix: A must be square.

Inverse of a square matrix — `matrix_inverse`

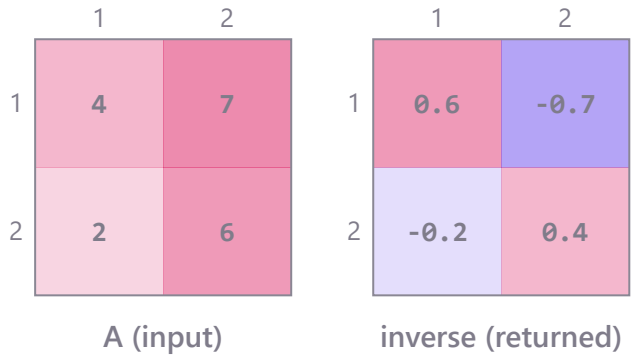
The inverse A^{-1} of a nonsingular square matrix (any size up to 10 x 10).

$$AA^{-1} = A^{-1}A = I$$

input	meaning	condition
`A`	a nonsingular square matrix	

output	meaning
`inverse`	the matrix A^{-1}

- **Method:** Gauss-Jordan elimination on [A | I] in exact rational arithmetic
- **Assumes:** A is square; det $A \neq 0$ (a singular matrix has no inverse); at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); inverse (returned).

```
linear_algebra_ai999x.solve('matrix_inverse', A='[[4, 7], [2, 6]]')
# -> inverse = [[0.6, -0.7], [-0.2, 0.4]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_inverse', A=[[1, 2, 3], [0, 1, 4], [5, 6, 0]])
# -> inverse = [[-24.0, 18.0, 5.0], [20.0, -15.0, -4.0], [-5.0, 4.0, 1.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the inverse of [[4, 7], [2, 6]].	SOLVED	inverse = [[0.6, -0.7], [-0.2, 0.4]]
Compute A^-1 for A = [[2, 1], [1, 1]].	SOLVED	inverse = [[1.0, -1.0], [-1.0, 2.0]]
Find the inverse of [[1, 2], [2, 4]].	INVALID_INPUT	Inverse of a square matrix: the matrix is singular (its determinant is 0), so it has no inverse.

Two linear equations in two unknowns — `solve_2x2_system`

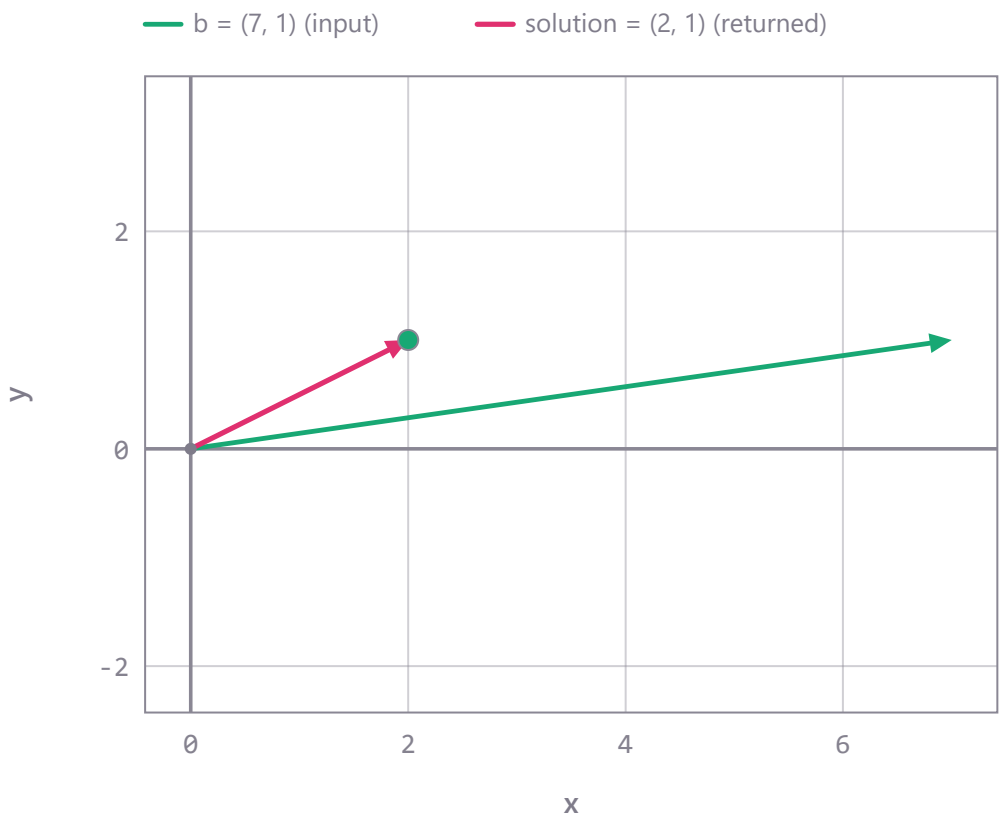
Solve a x + b $y = e$, c x + d $y = f$ for (x, y) by Cramer's rule.

$$x = \frac{\begin{vmatrix} e & b \\ f & d \end{vmatrix}}{\begin{vmatrix} a & b \\ c & d \end{vmatrix}}, \quad y = \frac{\begin{vmatrix} a & e \\ c & f \end{vmatrix}}{\begin{vmatrix} a & b \\ c & d \end{vmatrix}}$$

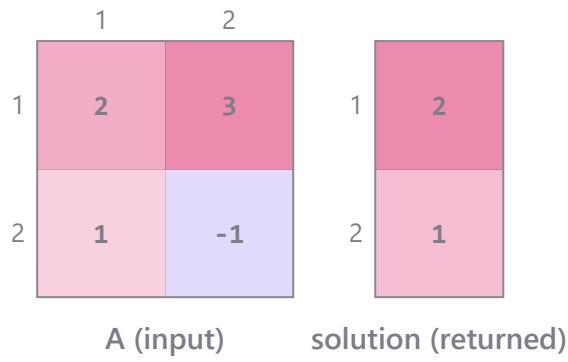
input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

output	meaning
`solution`	the solution (x, y)
`det`	determinant of the coefficient matrix

- **Method:** Cramer's rule x_j = det(A_j)/det(A) in exact decimal arithmetic
- **Assumes:** A is 2 x 2; det $A \neq 0$ (otherwise there is no unique solution)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input b = (7, 1); returned by linear-algebra-ai999x: solution = (2, 1); also returned det = -5. Other inputs: A = 2 3; 1 -1. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); solution (returned). Returned with them by linear-algebra-ai999x: det = -5. Other inputs: b = 7, 1.

```
linear_algebra_ai999x.solve('solve_2x2_system', A='2 3; 1 -1', b='7, 1')
# -> solution = [2.0, 1.0], det = -5   (SOLVED)
```

```
linear_algebra_ai999x.solve('solve_2x2_system', A=[[3, -1], [1, 2]], b=[5, 4])
# -> solution = [2.0, 1.0], det = 7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the system 2x + 3y = 7, x - y = 1.	SOLVED	solution = [2.0, 1.0], det = -5
Solve 3x - y = 5 and x + 2y = 4.	SOLVED	solution = [2.0, 1.0], det = 7
Use Cramer's rule to solve A = [[2, 3], [1, -1]], b = (7, 1).	SOLVED	solution = [2.0, 1.0], det = -5
Solve the system x + y = 2, 2x + 2y = 4.	INVALID_INPUT	Two linear equations in two unknowns: the determinant of the coefficient matrix is 0: the system has no unique solution.

Three linear equations in three unknowns — `solve_3x3_system`

Solve three linear equations in x, y, z by Cramer's rule.

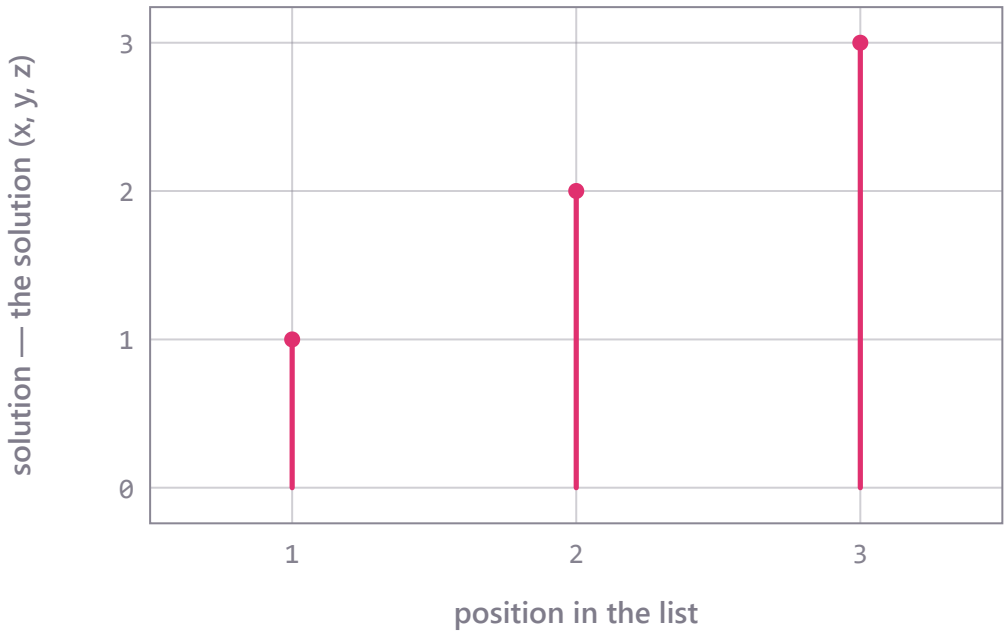
$$x_j = \frac{\det A_j}{\det A}$$

input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

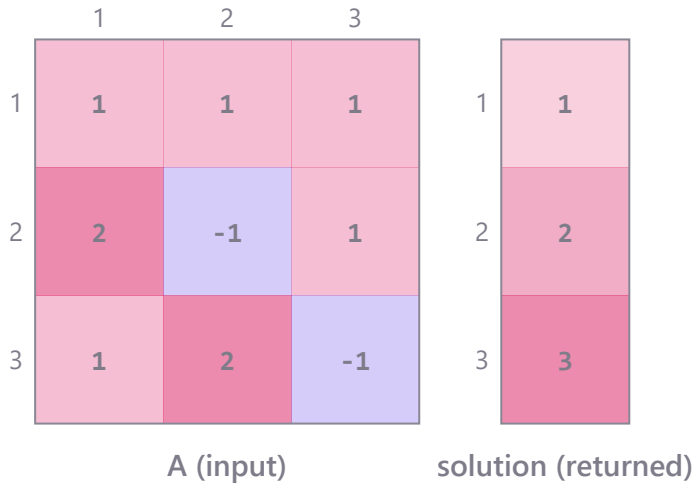
output	meaning
`solution`	the solution (x, y, z)
`det`	determinant of the coefficient matrix

- **Method:** Cramer's rule x_j = det(A_j)/det(A) in exact decimal arithmetic

- **Assumes:** A is 3 x 3; det $A \neq 0$ (otherwise there is no unique solution)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the solution (x, y, z) (solution) returned by linear-algebra-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 3, largest 3). Inputs: A = 1 1 1; 2 -1 1; 1 2 -1, b = 6, 3, 2.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); solution (returned). Returned with them by linear-algebra-ai999x: det = 7. Other inputs: b = 6, 3, 2.

```
linear_algebra_ai999x.solve('solve_3x3_system', A='1 1 1; 2 -1 1; 1 2 -1', b='6, 3, 2')
# -> solution = [1.0, 2.0, 3.0], det = 7   (SOLVED)
```

```
linear_algebra_ai999x.solve('solve_3x3_system', A=[[1, 1, 1], [0, 2, 5], [2, 5, -1]], b=[6, -4,
27])
# -> solution = [5.0, 3.0, -2.0], det = -21   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the system x + y + z = 6, 2x - y + z = 3, x + 2y - z = 2.	SOLVED	solution = [1.0, 2.0, 3.0], det = 7
Use Cramer's rule to solve A = [[1, 1, 1], [0, 2, 5], [2, 5, -1]], b = (6, -4, 27).	SOLVED	solution = [5.0, 3.0, -2.0], det = -21
Solve the system x + y + z = 1, x + y + z = 2, x - y = 0.	INVALID_INPUT	Three linear equations in three unknowns: the determinant of the coefficient matrix is 0: the system has no unique solut

Solve a square linear system Ax = b — `solve_linear_system`

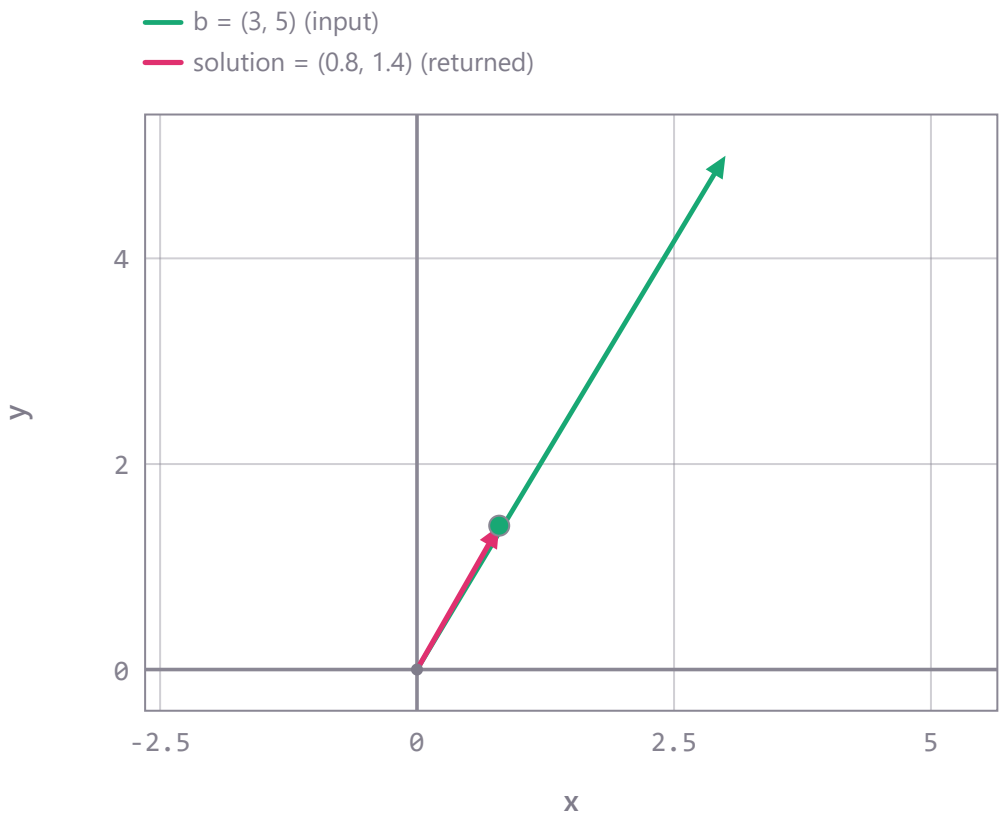
The unique solution of $Ax = b$ for a nonsingular n x n matrix A (n up to 10).

$$\mathbf{Ax = b}$$

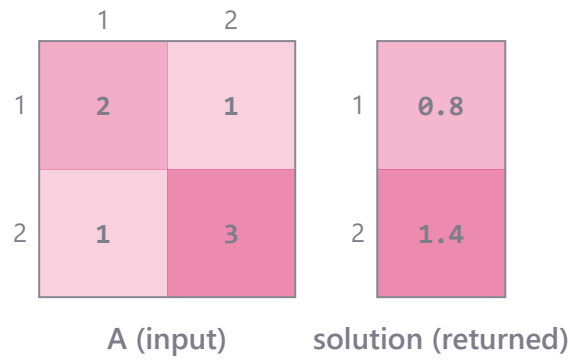
input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

output	meaning
`solution`	the solution vector x

- **Method:** Gaussian elimination with partial pivoting and back substitution, in exact rational arithmetic
- **Assumes:** A is square and nonsingular; b has one entry per row of A; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input b = (3, 5); returned by linear-algebra-ai999x: solution = (0.8, 1.4). Other inputs: A = [[2, 1], [1, 3]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); solution (returned). Other inputs: b = 3, 5.

```
linear_algebra_ai999x.solve('solve_linear_system', A='[[2, 1], [1, 3]]', b='3, 5')
# -> solution = [0.8, 1.4]   (SOLVED)
```

```
linear_algebra_ai999x.solve('solve_linear_system', A=[[1, 2, 0, 0], [0, 1, 2, 0], [0, 0, 1, 2], [2,
0, 0, 1]], b=[3, 3, 3, 3])
# -> solution = [1.0, 1.0, 1.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve Ax = b for A = [[2, 1], [1, 3]] and b = (3, 5).	SOLVED	solution = [0.8, 1.4]
Solve Ax = b for A = [[1, 2], [2, 4]] and b = (3, 6).	INVALID_INPUT	Solve a square linear system Ax = b: the system has no unique solution (the coefficient matrix is singular).
Solve Ax = b for A = [[1, 2], [3, 4]] and b = (3, 6, 9).	INVALID_INPUT	Solve a square linear system Ax = b: b must have exactly 2 entries.

Rank of a matrix — `matrix_rank`

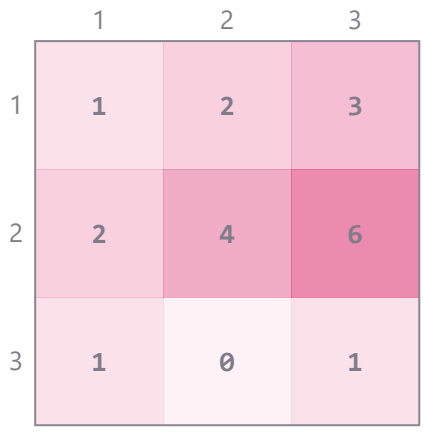
The rank: the number of linearly independent rows (equivalently columns) of a matrix.

$$\operatorname{rank} A = \dim \operatorname{col} A$$

input	meaning	condition
`A`	the matrix (up to 10 x 10)	

output	meaning
`rank`	rank of A

- **Method:** count of pivots of the reduced row echelon form, exact rational arithmetic
- **Assumes:** at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: rank = 2.

```
linear_algebra_ai999x.solve('matrix_rank', A='[[1, 2, 3], [2, 4, 6], [1, 0, 1]]')
# -> rank = 2   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_rank', A=[[1, 2], [2, 4]])
# -> rank = 1   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_rank', A=[[0, 0], [0, 0]])
# -> rank = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the rank of [[1, 2, 3], [2, 4, 6], [1, 0, 1]].	SOLVED	rank = 2
What is the rank of the matrix [1 2; 2 4]?	SOLVED	rank = 1

Reduced row echelon form — `rref`

The reduced row echelon form (RREF) of a matrix, its pivot columns (1-based) and its rank.

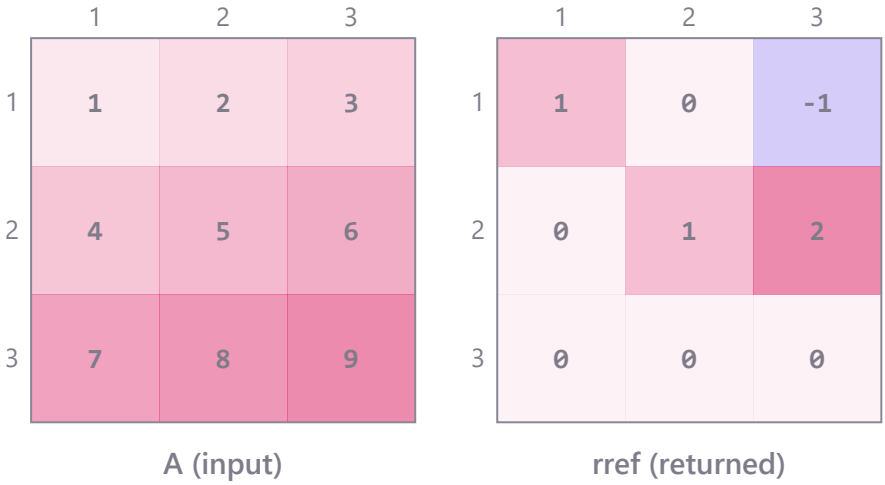
$$A \sim \mathrm{rref}(A)$$

input	meaning	condition
`A`	the matrix (up to 10 x 10)	

output	meaning
`rref`	the reduced row echelon form
`pivot_columns`	pivot columns, 1-based
`rank`	number of pivots

- **Method:** Gauss-Jordan elimination in exact rational arithmetic (the RREF is unique)

- **Assumes:** at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); rref (returned). Returned with them by linear-algebra-ai999x: rank = 2.

```
linear_algebra_ai999x.solve('rref', A='[[1, 2, 3], [4, 5, 6], [7, 8, 9]]')
# -> rref = [[1.0, 0.0, -1.0], [0.0, 1.0, 2.0], [0.0, 0.0, 0.0]], pivot_columns = [1.0, 2.0], rank
= 2   (SOLVED)
```

```
linear_algebra_ai999x.solve('rref', A=[[2, 4, 6], [1, 3, 5]])
# -> rref = [[1.0, 0.0, -1.0], [0.0, 1.0, 2.0]], pivot_columns = [1.0, 2.0], rank = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the reduced row echelon form of [[1, 2, 3], [4, 5, 6], [7, 8, 9]].	SOLVED	rref = [[1.0, 0.0, -1.0], [0.0, 1.0, 2.0], [0.0, 0.0, 0.0]], pivot_columns = [1.0, 2.0], rank = 2
Row reduce the matrix [2 4 6; 1 3 5].	SOLVED	rref = [[1.0, 0.0, -1.0], [0.0, 1.0, 2.0]], pivot_columns = [1.0, 2.0], rank = 2

Null space (kernel) of a matrix — `null_space`

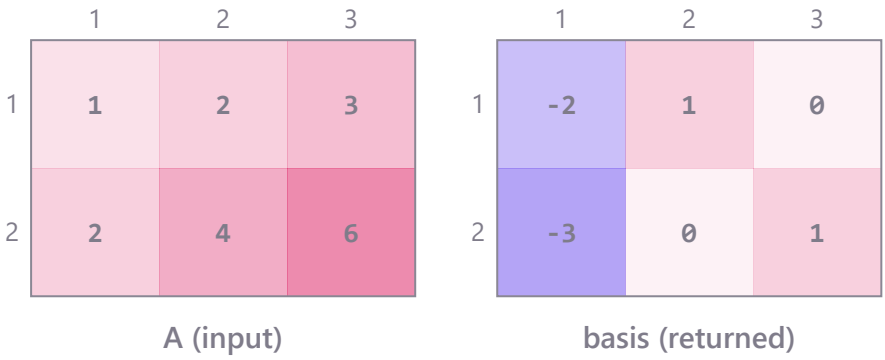
The dimension of the null space {x : $\boldsymbol{Ax} = \mathbf{0}$} (the nullity) and a basis of it.

$$\operatorname{null} A = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}, \quad \dim = n - \operatorname{rank} A$$

input	meaning	condition
`A`	the matrix (up to 10 x 10)	

output	meaning
`nullity`	dimension of the null space = columns - rank
`basis`	basis vectors: one per free variable, that variable 1, the other free variables 0

- **Method:** solve $Ax = 0$ from the exact reduced row echelon form; each free variable gives one basis vector
- **Assumes:** at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); basis (returned). Returned with them by linear-algebra-ai999x: nullity = 2.

```
linear_algebra_ai999x.solve('null_space', A='[[1, 2, 3], [2, 4, 6]]')
# -> nullity = 2, basis = [[-2.0, 1.0, 0.0], [-3.0, 0.0, 1.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('null_space', A=[[1, 0, -1], [0, 1, 2]])
# -> nullity = 1, basis = [[1.0, -2.0, 1.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('null_space', A=[[1, 0], [0, 1]])
# -> nullity = 0, basis = []   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the null space of [[1, 2, 3], [2, 4, 6]].	SOLVED	nullity = 2, basis = [[-2.0, 1.0, 0.0], [-3.0, 0.0, 1.0]]
What is the nullity of the matrix [1 0 -1; 0 1 2]?	SOLVED	nullity = 1, basis = [[1.0, -2.0, 1.0]]
Find a basis for the kernel of [[1, 1], [1, 1]].	SOLVED	nullity = 1, basis = [[-1.0, 1.0]]

Column space of a matrix — `column_space`

A basis of the column space (range) of A: the pivot columns of A itself, and its dimension.

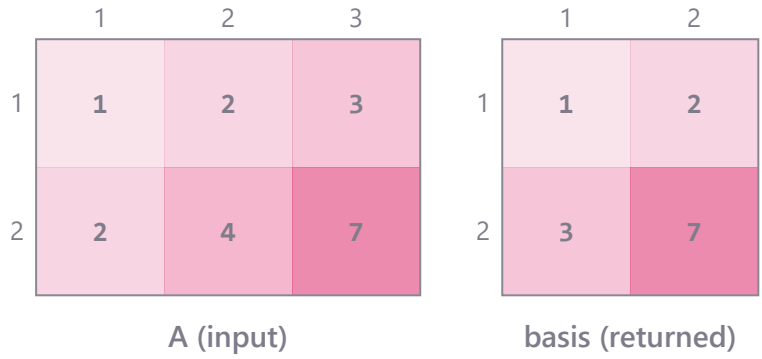
$$\operatorname{col} A = \operatorname{span}\{\mathbf{a}_j : j \text{ a pivot column}\}$$

input	meaning	condition
`A`	the matrix (up to 10 x 10)	

output	meaning
`dimension`	dimension of the column space (the rank)

output	meaning
`basis`	the pivot columns of A, left to right

- **Method:** the pivot positions of the exact reduced row echelon form pick the columns of the ORIGINAL matrix
- **Assumes:** at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); basis (returned). Returned with them by linear-algebra-ai999x: dimension = 2.

```
linear_algebra_ai999x.solve('column_space', A='[[1, 2, 3], [2, 4, 7]]')
# -> dimension = 2, basis = [[1.0, 2.0], [3.0, 7.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('column_space', A=[[1, 2], [2, 4]])
# -> dimension = 1, basis = [[1.0, 2.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find a basis for the column space of [[1, 2, 3], [2, 4, 7]].	SOLVED	dimension = 2, basis = [[1.0, 2.0], [3.0, 7.0]]
What is the column space of [1 2; 2 4]?	SOLVED	dimension = 1, basis = [[1.0, 2.0]]

Linear independence of vectors — `linear_independence`

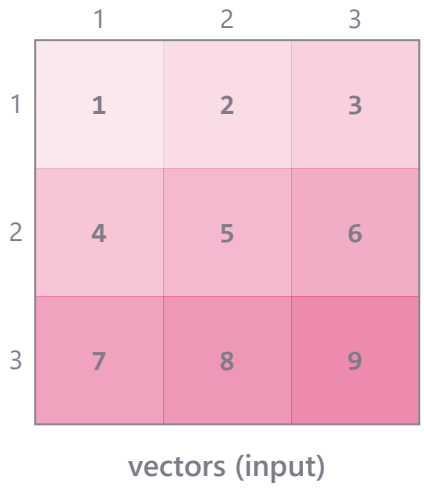
Whether a list of vectors is linearly independent, with the rank of the list.

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0} \Rightarrow c_1 = \cdots = c_k = 0$$

input	meaning	condition
`vectors`	the vectors, one per row (equal lengths)	

output	meaning
`independent`	true when no vector is a combination of the others
`rank`	dimension of their span

- **Method:** the vectors are independent exactly when the rank of the matrix of rows equals their number (exact)
- **Assumes:** all vectors have the same length; at most 10 vectors of at most 10 entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): vectors (input). Returned with them by linear-algebra-ai999x: rank = 2.

```
linear_algebra_ai999x.solve('linear_independence', vectors='1 2 3; 4 5 6; 7 8 9')
# -> independent = False, rank = 2   (SOLVED)
```

```
linear_algebra_ai999x.solve('linear_independence', vectors=[[1, 0], [1, 1]])
# -> independent = True, rank = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Are the vectors (1, 2, 3), (4, 5, 6) and (7, 8, 9) linearly independent?	SOLVED	independent = False, rank = 2
Are the vectors [1, 0] and [1, 1] linearly independent?	SOLVED	independent = True, rank = 2
Are the vectors (1, 2) and (1, 2, 3) linearly independent?	INVALID_INPUT	vectors: expected a rectangular matrix (rows of equal length).

Number of solutions of a linear system — `linear_system_classification`

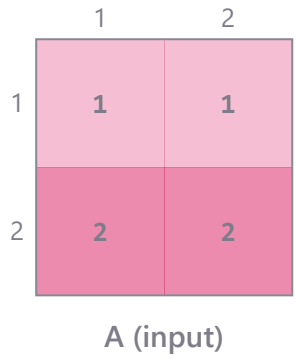
Whether $\boldsymbol{Ax} = \boldsymbol{b}$ has a unique solution, infinitely many, or none (Rouche-Capelli), with the two ranks.

$$\mathbf{rank}\, A < \mathbf{rank}[A \,|\, \mathbf{b}] \Rightarrow \mathbf{none};\;\; \mathbf{rank}\, A = \mathbf{rank}[A \,|\, \mathbf{b}] = n \Rightarrow \mathbf{unique}$$

input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

output	meaning
`solutions`	unique / infinitely many / none
`rank_A`	rank of A
`rank_augmented`	rank of the augmented matrix [A | b]

- **Method:** compare rank A, rank [A | b] and the number of unknowns (exact row reduction)
- **Assumes:** b has one entry per row of A; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: rank_A = 1, rank_augmented = 2. Other inputs: b = 2, 5.

```
linear_algebra_ai999x.solve('linear_system_classification', A='1 1; 2 2', b='2, 5')
# -> solutions = none, rank_A = 1, rank_augmented = 2   (SOLVED)
```

```
linear_algebra_ai999x.solve('linear_system_classification', A=[[1, 1, 1], [1, -1, 0]], b=[1, 0])
# -> solutions = infinitely many, rank_A = 2, rank_augmented = 2   (SOLVED)
```

```
linear_algebra_ai999x.solve('linear_system_classification', A=[[2, 1], [1, 3]], b=[3, 5])
# -> solutions = unique, rank_A = 2, rank_augmented = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many solutions does the system x + y = 2, 2x + 2y = 5 have?	SOLVED	solutions = none, rank_A = 1, rank_augmented = 2
Is the system x + y + z = 1, x - y = 0 consistent?	SOLVED	solutions = infinitely many, rank_A = 2, rank_augmented = 2
How many solutions does Ax = b have for A = [[1, 2], [2, 4]] and b = (3, 6)?	SOLVED	solutions = infinitely many, rank_A = 1, rank_augmented = 1

Minor and cofactor of an entry — `minor_cofactor`

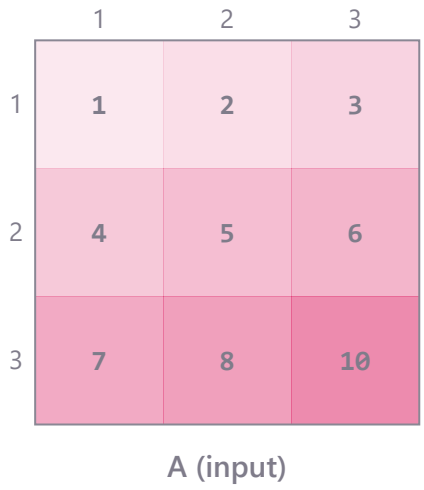
The minor M_ij (determinant with row i and column j deleted) and the cofactor $C_ij = (-1)^{i+j}$ M_ij.

$$C_{ij} = (-1)^{i+j} M_{ij}$$

input	meaning	condition
`A`	a square matrix	
`i`	row of the entry (1-based)	
`j`	column of the entry (1-based)	

output	meaning
`minor`	the minor M_ij
`cofactor`	the cofactor C_ij

- **Method:** delete row i and column j, take the determinant exactly; attach the sign $(-1)^{i+j}$
- **Assumes:** A is square, at least 2 x 2; $1 \le i, j \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: minor = -6, cofactor = 6. Other inputs: i = 2, j = 3.

```
linear_algebra_ai999x.solve('minor_cofactor', A='[[1, 2, 3], [4, 5, 6], [7, 8, 10]]', i=2, j=3)
# -> minor = -6, cofactor = 6   (SOLVED)
```

```
linear_algebra_ai999x.solve('minor_cofactor', A=[[1, 2, 3], [4, 5, 6], [7, 8, 10]], i=1, j=2)
# -> minor = -2, cofactor = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the cofactor C23 of [[1, 2, 3], [4, 5, 6], [7, 8, 10]].	SOLVED	minor = -6, cofactor = 6
Find the minor of the entry in row 1, column 2 of [[1, 2, 3], [4, 5, 6], [7, 8, 10]].	SOLVED	minor = -2, cofactor = 2

question	outcome	result
Find the cofactor C35 of [[1, 2, 3], [4, 5, 6], [7, 8, 10]].	INVALID_INPUT	Minor and cofactor of an entry: j must be a whole number from 1 to 3.

Adjugate (classical adjoint) of a matrix — `adjugate`

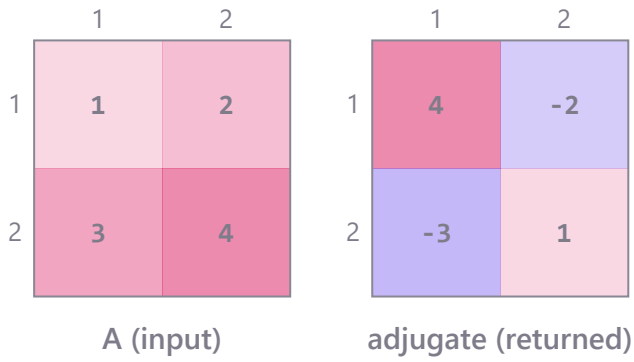
adj(A): the transpose of the matrix of cofactors; A adj(A) = det(A) I.

$$\operatorname{adj}(A)_{ij} = C_{ji}$$

input	meaning	condition
`A`	a square matrix (up to 7 x 7)	

output	meaning
`adjugate`	the adjugate matrix adj(A)

- **Method:** entry (i, j) = cofactor C_ji, every minor computed exactly
- **Assumes:** A is square; at most 7 x 7
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); adjugate (returned).

```
linear_algebra_ai999x.solve('adjugate', A='[[1, 2], [3, 4]]')
# -> adjugate = [[4.0, -2.0], [-3.0, 1.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('adjugate', A=[[1, 2, 3], [0, 1, 4], [5, 6, 0]])
# -> adjugate = [[-24.0, 18.0, 5.0], [20.0, -15.0, -4.0], [-5.0, 4.0, 1.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the adjugate of [[1, 2], [3, 4]].	SOLVED	adjugate = [[4.0, -2.0], [-3.0, 1.0]]

Characteristic polynomial of a matrix — `characteristic_polynomial`

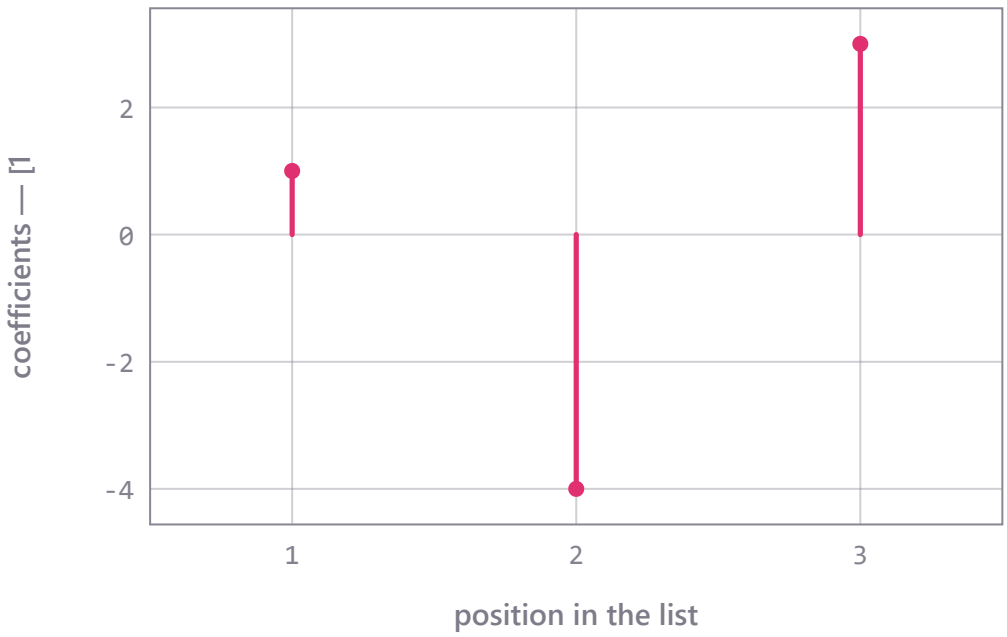
The coefficients of det(lambda I - A) = lambda^n + c1 $\lambda^{n-1} + \ldots + cn$, highest power first.

$$p(\lambda) = \det(\lambda I - A)$$

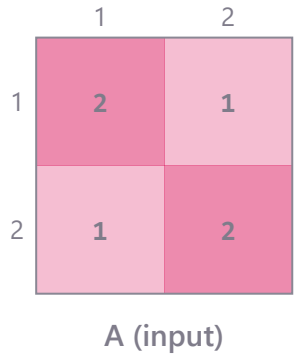
input	meaning	condition
`A`	a square matrix (up to 10 x 10)	

output	meaning
`coefficients`	[1, c1, ..., cn] of det(lambda I - A), highest power first

- **Method:** Faddeev-LeVerrier recursion in exact rational arithmetic
- **Assumes:** A is square; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



[1, c1, ..., cn] of det(lambda I - A), highest power first (coefficients) returned by linear-algebra-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 3, largest 3). Inputs: A = [[2, 1], [1, 2]].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input).

```
linear_algebra_ai999x.solve('characteristic_polynomial', A='[[2, 1], [1, 2]]')
# -> coefficients = [1.0, -4.0, 3.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('characteristic_polynomial', A=[[1, 2, 0], [0, 3, 0], [2, -4, 2]])
# -> coefficients = [1.0, -6.0, 11.0, -6.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the characteristic polynomial of [[2, 1], [1, 2]].	SOLVED	coefficients = [1.0, -4.0, 3.0]
Find the characteristic polynomial of [[1, 2, 3], [4, 5, 6]].	INVALID_INPUT	Characteristic polynomial of a matrix: A must be square.

Eigenvalues of a 2 x 2 matrix — `eigenvalues_2x2`

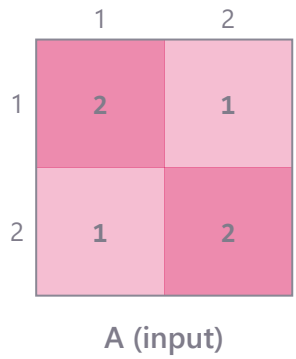
Both eigenvalues of a 2 x 2 matrix from its characteristic equation $\lambda^2 - tr$(A) lambda + det(A) = 0.

$$\lambda = \frac{\operatorname{tr} A \pm \sqrt{(\operatorname{tr} A)^2 - 4 \det A}}{2}$$

input	meaning	condition
`A`	a 2 x 2 matrix	

output	meaning
`lambda1`	first eigenvalue (the smaller real one, or the one with positive imaginary part)
`lambda2`	second eigenvalue

- **Method:** quadratic formula on the characteristic polynomial (trace and determinant exact, stable form)
- **Assumes:** A is 2 x 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: lambda1 = 1, lambda2 = 3.

```
linear_algebra_ai999x.solve('eigenvalues_2x2', A='[[2, 1], [1, 2]]')
# -> lambda1 = 1, lambda2 = 3   (SOLVED)
```

```
linear_algebra_ai999x.solve('eigenvalues_2x2', A=[[0, -1], [1, 0]])
# -> lambda1 = 0 + 1i, lambda2 = 0 - 1i   (SOLVED)
```

```
linear_algebra_ai999x.solve('eigenvalues_2x2', A=[[4, 1], [2, 3]])
# -> lambda1 = 2, lambda2 = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the eigenvalues of [[2, 1], [1, 2]].	SOLVED	lambda1 = 1, lambda2 = 3
What are the eigenvalues of the matrix [0 -1; 1 0]?	SOLVED	lambda1 = 0 + 1i, lambda2 = 0 - 1i

Coordinates of a vector in a basis — `change_of_basis`

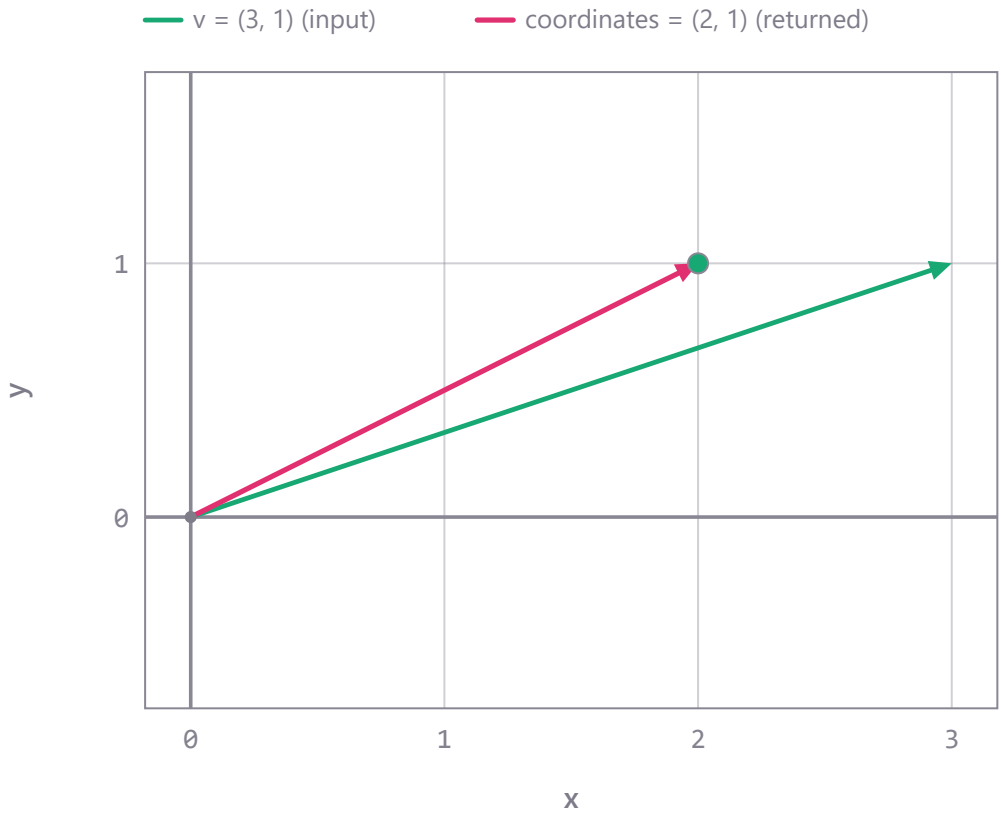
The coordinates c of v with respect to a basis b1, ..., bn: $v = c_1 b_1 + \ldots + cn$ bn.

$$\mathbf{v} = \sum_i c_i \mathbf{b}_i$$

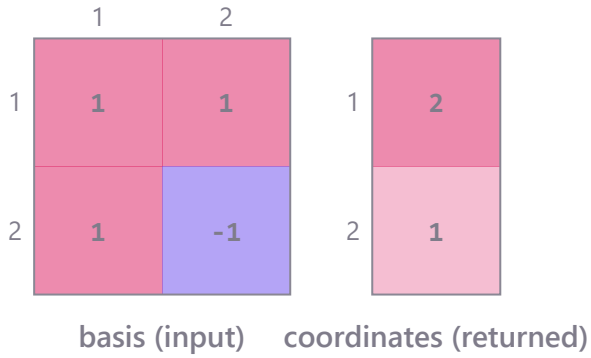
input	meaning	condition
`basis`	the basis vectors, one per row	
`v`	the vector	

output	meaning
`coordinates`	the coordinate vector [v]_B

- **Method:** solve B $c = v$ with the basis vectors as the columns of B (exact)
- **Assumes:** n basis vectors of length n that are linearly independent; v has n entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input v = (3, 1); returned by linear-algebra-ai999x: coordinates = (2, 1). Other inputs: basis = 1 1; 1 -1. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): basis (input); coordinates (returned). Other inputs: v = 3, 1.

```
linear_algebra_ai999x.solve('change_of_basis', basis='1 1; 1 -1', v='3, 1')
# -> coordinates = [2.0, 1.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('change_of_basis', basis=[[1, 0, 0], [1, 1, 0], [1, 1, 1]], v=[2, 3, 4])
# -> coordinates = [-1.0, -1.0, 4.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the coordinates of (3, 1) in the basis (1, 1), (1, -1).	SOLVED	coordinates = [2.0, 1.0]
Find the coordinates of (3, 1) in the basis (1, 2), (2, 4).	INVALID_INPUT	Coordinates of a vector in a basis: the basis vectors are linearly dependent, so they are not a basis.

Least-squares regression line — `least_squares_line`

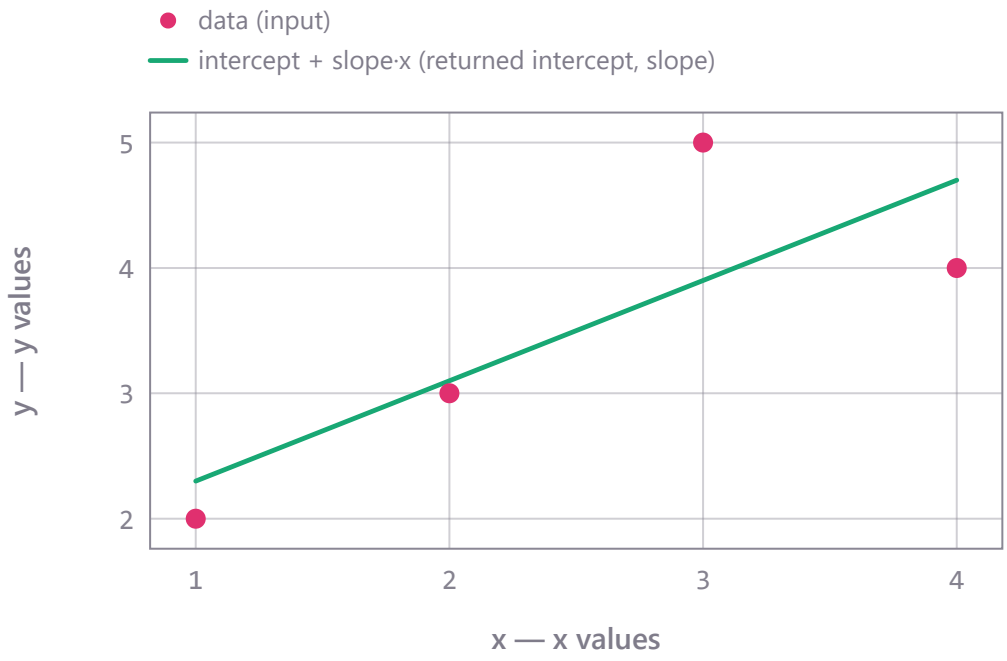
The straight line $y = a + b$ x that best fits data points in the least-squares sense, with R^2.

$$b = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}, \quad a = \bar{y} - b\bar{x}$$

input	meaning	condition
`x`	x values	
`y`	y values	

output	meaning
`slope`	slope b
`intercept`	intercept a
`r_squared`	coefficient of determination R^2

- **Method:** b = Sxy / Sxx, a = mean(y) - b mean(x), R^2 = Sxy^2 / (Sxx Syy), exact sums
- **Assumes:** x and y have the same number of values, at least 2; the x values are not all equal; the y values are not all equal (otherwise R^2 is undefined)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y values) against x (x values) as given in the worked example (4 points); drawn on it from what linear-algebra-ai999x returned: the line y = 1.5 + 0.8·x from the returned intercept and slope. Also returned: r_squared = 0.64.

```
linear_algebra_ai999x.solve('least_squares_line', x='1, 2, 3, 4', y='2, 3, 5, 4')
# -> slope = 0.8, intercept = 1.5, r_squared = 0.64   (SOLVED)
```

```
linear_algebra_ai999x.solve('least_squares_line', x=[0, 1, 2, 3], y=[1, 3, 5, 7])
# -> slope = 2, intercept = 1, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the least squares line for the points (1, 2), (2, 3), (3, 5), (4, 4).	SOLVED	slope = 0.8, intercept = 1.5, r_squared = 0.64
Fit a line to the data x = (0, 1, 2, 3), y = (1, 3, 5, 7) by least squares.	SOLVED	slope = 2, intercept = 1, r_squared = 1
Find the least squares line for the points (1, 2), (1, 3), (1, 5).	INVALID_INPUT	Least-squares regression line: all x values are equal: the line is vertical and has no slope.

Least-squares polynomial fit — `least_squares_polynomial`

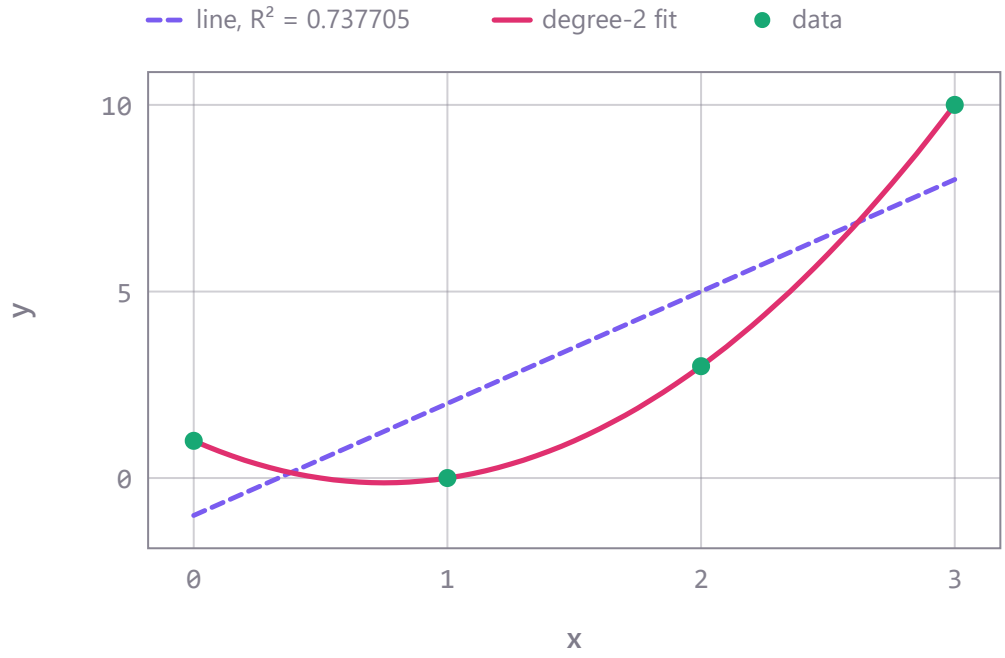
The polynomial $c_0 + c_1$ x + ... + cd x^d of a given degree that best fits data in the least-squares sense.

$$\min_{\mathbf{c}} \sum_i \left(y_i - \sum_{k=0}^{d} c_k x_i^k \right)^2$$

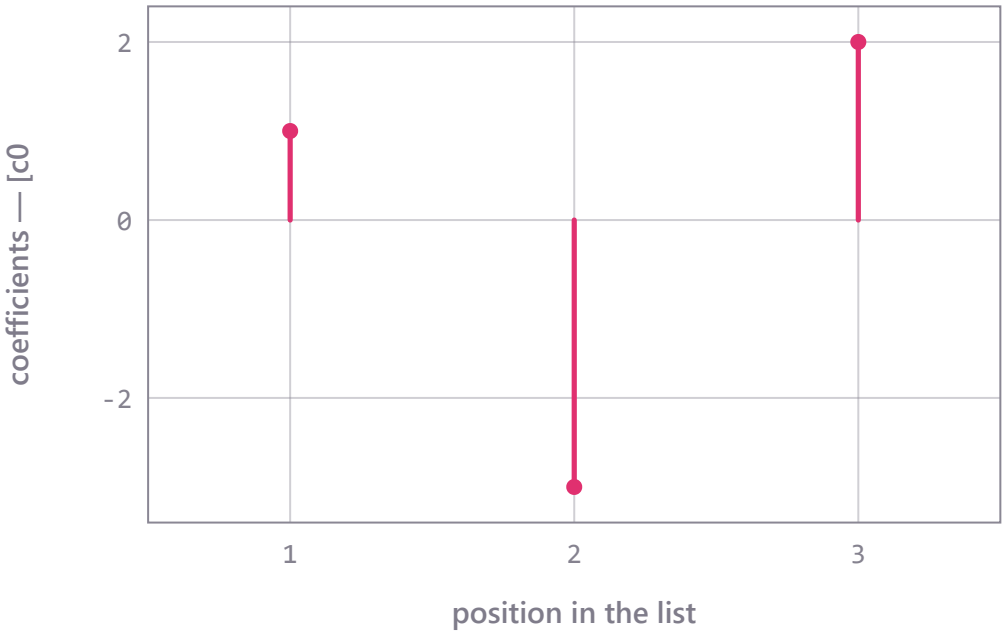
input	meaning	condition
`x`	x values	
`y`	y values	
`degree`	degree d of the polynomial (0 to 8)	

output	meaning
`coefficients`	[c0, c1, ..., cd], lowest power first

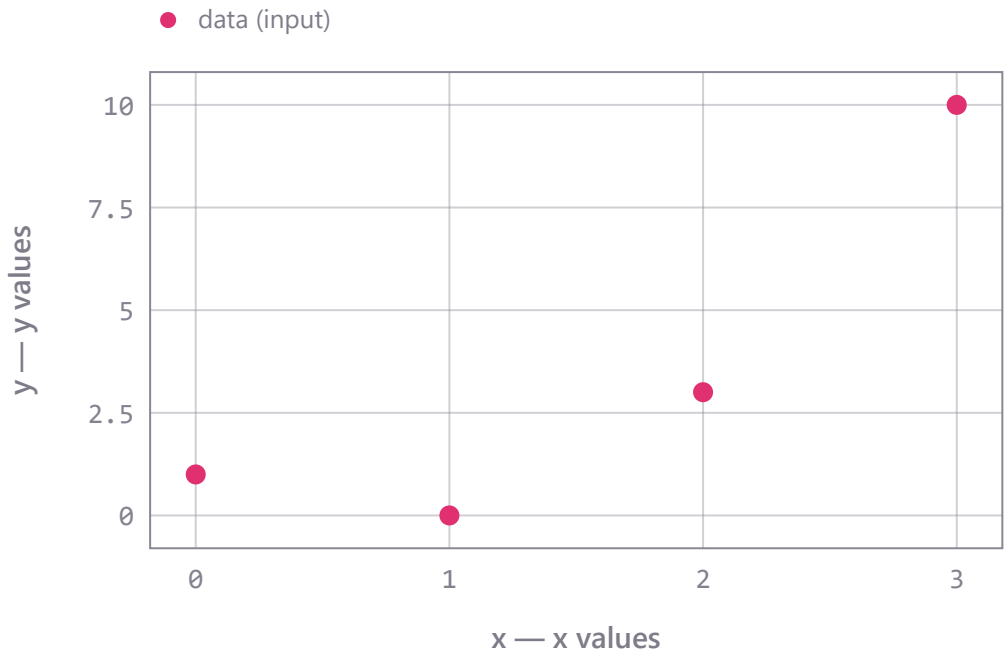
- **Method:** normal equations V^T V $c = V^T$ y of the Vandermonde matrix, solved in exact rational arithmetic
- **Assumes:** x and y have the same number of values; more distinct x values than the degree; degree at most 8
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Least-squares fits to the points x = 0, 1, 2, 3, y = 1, 0, 3, 10: the degree-2 polynomial $y = 1 - 3x + 2x^2$ (coefficients from the package, the curve evaluated by it at 61 points) and the straight line $y = -1 + 3x$ with $R^2 = 0.737705$.



[c0, c1, ..., cd], lowest power first (coefficients) returned by linear-algebra-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 2, largest 2). Inputs: x = 0, 1, 2, 3, y = 1, 0, 3, 10, degree = 2.



The data y (y values) against x (x values) as given in the worked example (4 points). Other inputs: degree = 2.

```
linear_algebra_ai999x.solve('least_squares_polynomial', x='0, 1, 2, 3', y='1, 0, 3, 10', degree=2)
# -> coefficients = [1.0, -3.0, 2.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('least_squares_polynomial', x=[-1, 0, 1, 2], y=[2, 1, 2, 5], degree=2)
# -> coefficients = [1.0, 0.0, 1.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('least_squares_polynomial', x=[1, 2, 3, 4], y=[2, 3, 5, 4], degree=1)
# -> coefficients = [1.5, 0.8]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a quadratic to the points (0, 1), (1, 0), (2, 3), (3, 10) by least squares.	SOLVED	coefficients = [1.0, -3.0, 2.0]
Find the least squares polynomial of degree 2 for x = (-1, 0, 1, 2), y = (2, 1, 2, 5).	SOLVED	coefficients = [1.0, 0.0, 1.0]
Fit a cubic to the points (0, 1), (1, 0), (2, 3) by least squares.	INVALID_INPUT	Least-squares polynomial fit: a degree-3 fit needs at least 4 distinct x values.

Least-squares solution of an overdetermined system — `least_squares_solution`

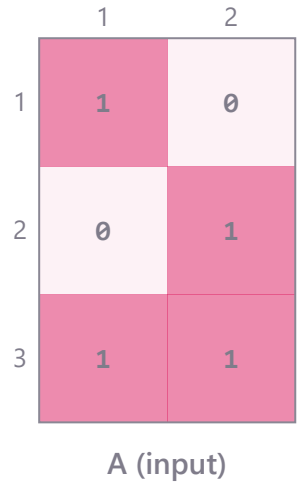
The x minimizing |Ax - b| for a tall matrix A with independent columns, and the residual length.

$$A^T A\,\hat{\mathbf{x}} = A^T \mathbf{b}$$

input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

output	meaning
`solution`	least-squares solution x
`residual_norm`	|Ax - b| at the solution

- **Method:** normal equations A^T A $x = A^T$ b in exact rational arithmetic
- **Assumes:** at least as many rows as columns; the columns of A are linearly independent; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: residual_norm = 1.1547. Other inputs: b = 1, 1, 0.

```
linear_algebra_ai999x.solve('least_squares_solution', A='[[1, 0], [0, 1], [1, 1]]', b='1, 1, 0')
# -> solution = [0.3333333333333333, 0.3333333333333333], residual_norm = 1.1547   (SOLVED)
```

```
linear_algebra_ai999x.solve('least_squares_solution', A=[[1, 1], [1, 2], [1, 3]], b=[1, 2, 2])
# -> solution = [0.6666666666666666, 0.5], residual_norm = 0.408248   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the least squares solution of Ax = b for A = [[1, 0], [0, 1], [1, 1]] and b = (1, 1, 0).	SOLVED	solution = [0.3333333333333333, 0.3333333333333333], residual_norm = 1.1547
Find the least squares solution of Ax = b for A = [[1, 2], [2, 4], [3, 6]] and b = (1, 1, 0).	INVALID_INPUT	Least-squares solution of an overdetermined system: the columns are linearly dependent: the least-squares solution is no

Moore-Penrose pseudoinverse (full rank) — `pseudoinverse`

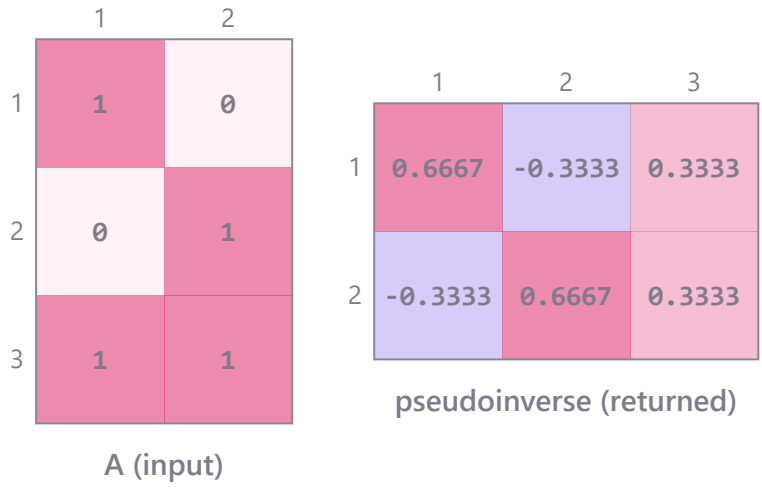
The pseudoinverse A+ of a matrix with independent columns, (A^T A)^-1 A^T, or independent rows, A^T (A A^T)^-1.

$$A^+ = (A^T A)^{-1} A^T \text{ or } A^T (A A^T)^{-1}$$

input	meaning	condition
`A`	a matrix of full rank (up to 10 x 10)	

output	meaning
`pseudoinverse`	the matrix A+

- **Method:** the full-rank formulas in exact rational arithmetic
- **Assumes:** A has full rank (independent rows or independent columns); at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); pseudoinverse (returned).

```
linear_algebra_ai999x.solve('pseudoinverse', A='[[1, 0], [0, 1], [1, 1]]')
# -> pseudoinverse = [[0.6666666666666666, -0.3333333333333333, 0.3333333333333333],
[-0.3333333333333333, 0.6666666666666666, 0.3333333333333333]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('pseudoinverse', A=[[1, 2, 3]])
# -> pseudoinverse = [[0.07142857142857142], [0.14285714285714285], [0.21428571428571427]]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the pseudoinverse of [[1, 0], [0, 1], [1, 1]].	SOLVED	pseudoinverse = [[0.6666666666666666, -0.3333333333333333, 0.3333333333333333], [-0.3333333333333333, 0.6666666666666666, 0.3333333333333333]]
Find the pseudoinverse of [[1, 2], [2, 4], [3, 6]].	INVALID_INPUT	Moore-Penrose pseudoinverse (full rank): this version computes the pseudoinverse of a matrix of full rank (independent r

Orthogonal projection matrix onto a column space — `projection_matrix`

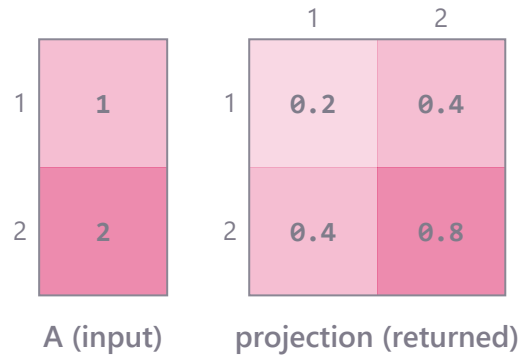
The matrix $P = A$ (A^T A)^-1 A^T that projects orthogonally onto the column space of A.

$$P = A(A^T A)^{-1} A^T$$

input	meaning	condition
`A`	a matrix with independent columns	

output	meaning
`projection`	the projection matrix P

- **Method:** $P = A$ (A^T A)^-1 A^T in exact rational arithmetic
- **Assumes:** the columns of A are linearly independent; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); projection (returned).

```
linear_algebra_ai999x.solve('projection_matrix', A='1; 2')
# -> projection = [[0.2, 0.4], [0.4, 0.8]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('projection_matrix', A=[[1, 0], [0, 1], [0, 0]])
# -> projection = [[1.0, 0.0, 0.0], [0.0, 1.0, 0.0], [0.0, 0.0, 0.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the projection matrix onto the column space of [1; 2]?	SOLVED	projection = [[0.2, 0.4], [0.4, 0.8]]
Find the projection matrix onto the column space of [[1, 2], [2, 4]].	INVALID_INPUT	Orthogonal projection matrix onto a column space: the columns of A must be linearly independent.

Singular values of a matrix — `singular_values`

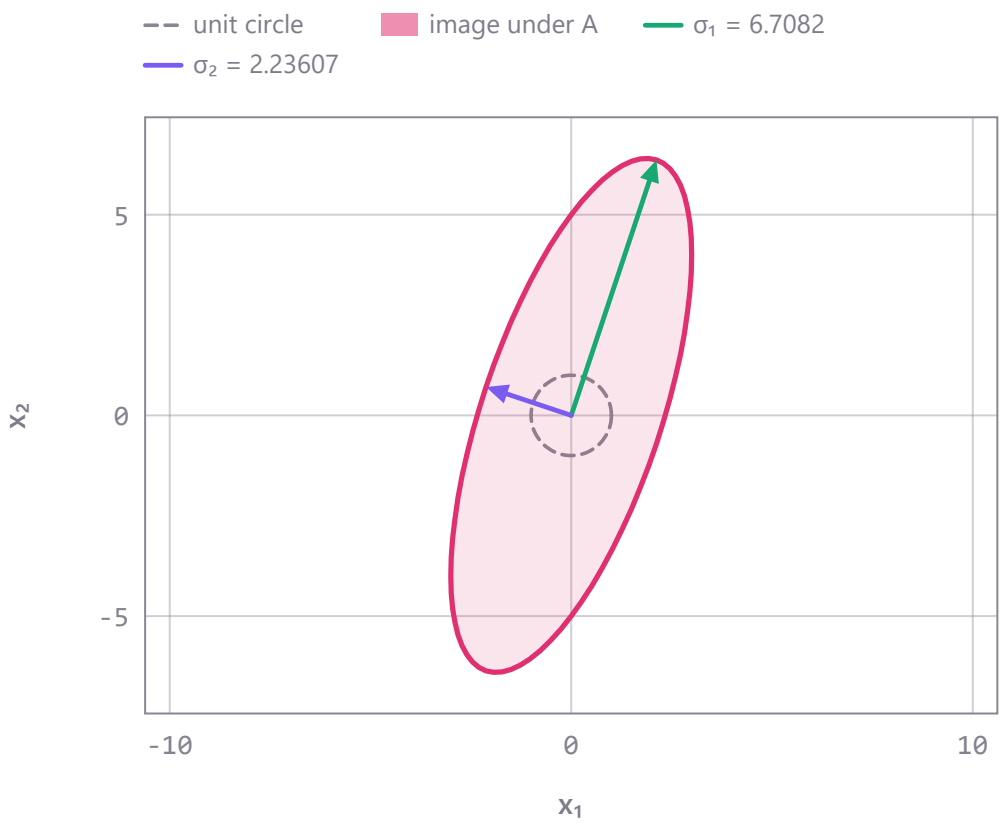
The singular values of a matrix (any shape up to 10 x 10), largest first.

$$A = U\Sigma V^T, \quad \sigma_i = \sqrt{\lambda_i(A^T A)}$$

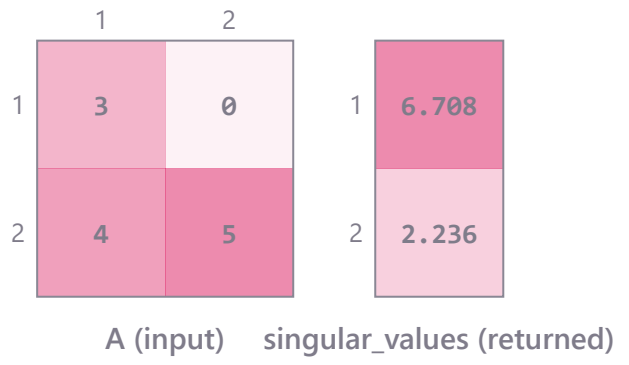
input	meaning	condition
`A`	the matrix	

output	meaning
`singular_values`	singular values, descending

- **Method:** one-sided Jacobi (Hestenes) orthogonalization of the columns; the column lengths are the singular values
- **Assumes:** at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



72 points of the unit circle mapped by A = [[3, 0], [4, 5]]: an ellipse whose semi-axes are the singular values $\sigma_1 = 6.7082$ and $\sigma_2 = 2.23607$ returned by the package (the longest and shortest mapped radii drawn: 6.7082 and 2.23607).



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); singular_values (returned).

```
linear_algebra_ai999x.solve('singular_values', A='[[3, 0], [4, 5]]')
# -> singular_values = [6.708203932499369, 2.2360679774997894]   (SOLVED)
```

```
linear_algebra_ai999x.solve('singular_values', A=[[1, 0, 0], [0, 2, 0]])
# -> singular_values = [2.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the singular values of [[3, 0], [4, 5]].	SOLVED	singular_values = [6.708203932499369, 2.2360679774997894]
What are the singular values of the matrix [1 0 0; 0 2 0]?	SOLVED	singular_values = [2.0, 1.0]

Norms of a matrix — `matrix_norms`

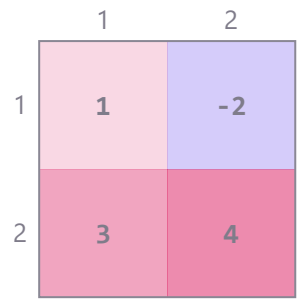
The Frobenius norm, the 1-norm (largest column sum), the infinity-norm (largest row sum) and the spectral 2-norm.

$$\|A\|_F = \sqrt{\sum a_{ij}^2},\ \|A\|_1 = \max_j \sum_i |a_{ij}|,\ \|A\|_\infty = \max_i \sum_j |a_{ij}|,\ \|A\|_2 = \sigma_{\max}$$

input	meaning	condition
`A`	the matrix	

output	meaning
`frobenius`	Frobenius norm
`norm1`	1-norm: largest absolute column sum
`norm_inf`	infinity-norm: largest absolute row sum
`norm2`	spectral norm: the largest singular value

- **Method:** exact sums for the Frobenius, 1- and infinity-norms; one-sided Jacobi SVD for the 2-norm
- **Assumes:** at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: frobenius = 5.47723, norm1 = 6, norm_inf = 7, norm2 = 5.11667.

```
linear_algebra_ai999x.solve('matrix_norms', A='[[1, -2], [3, 4]]')
# -> frobenius = 5.47723, norm1 = 6, norm_inf = 7, norm2 = 5.11667   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_norms', A=[[3, 0], [0, -4]])
# -> frobenius = 5, norm1 = 4, norm_inf = 4, norm2 = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the norms of the matrix [[1, -2], [3, 4]].	SOLVED	frobenius = 5.47723, norm1 = 6, norm_inf = 7, norm2 = 5.11667
What is the Frobenius norm of [1 2; 3 4]?	SOLVED	frobenius = 5.47723, norm1 = 6, norm_inf = 7, norm2 = 5.46499

Condition number of a matrix — `condition_number`

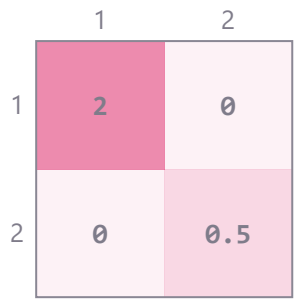
How sensitive $Ax = b$ is to errors: the 2-norm condition number sigma_max/sigma_min and the 1-norm one.

$$\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}, \quad \kappa_1(A) = \|A\|_1 \|A^{-1}\|_1$$

input	meaning	condition
`A`	a nonsingular square matrix	

output	meaning
`cond2`	2-norm condition number sigma_max / sigma_min
`cond1`	1-norm condition number |A|_1 $\|A^{-1}\|$_1

- **Method:** singular values by one-sided Jacobi; the 1-norm one from the exact inverse
- **Assumes:** A is square and nonsingular; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: cond2 = 4, cond1 = 4.

```
linear_algebra_ai999x.solve('condition_number', A='[[2, 0], [0, 0.5]]')
# -> cond2 = 4, cond1 = 4   (SOLVED)
```

```
linear_algebra_ai999x.solve('condition_number', A=[[1, 2], [3, 4]])
# -> cond2 = 14.933, cond1 = 21   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the condition number of the matrix [2 0; 0 0.5]?	SOLVED	cond2 = 4, cond1 = 4
Find the condition number of [[1, 2], [2, 4]].	INVALID_INPUT	Condition number of a matrix: the matrix is singular: its condition number is infinite.

Best rank-k approximation (Eckart-Young) — `low_rank_approximation`

The rank-k matrix closest to A in the 2- and Frobenius norms (truncated SVD), with both approximation errors.

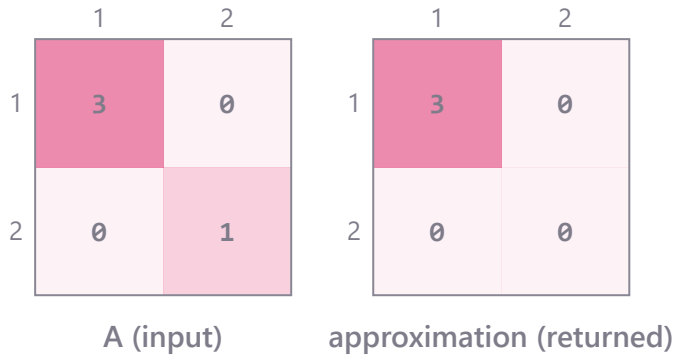
$$A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

input	meaning	condition
`A`	the matrix	
`k`	target rank k	

output	meaning
`approximation`	the best rank-k approximation A_k
`error2`	|A - A_k|Missing superscript or subscript argument(k+1)
`errorF`	|A - A_k|$_F$ = *sqrt*(sigma_(k+1)^2 + ...)

- **Method:** one-sided Jacobi SVD, keep the k largest singular triplets
- **Assumes:** 1 <= k <= min(rows, columns); Missing superscript or subscript argument(k+1) (otherwise A_k is not unique); at most 10 x 10

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); approximation (returned). Returned with them by linear-algebra-ai999x: error2 = 1, errorF = 1. Other inputs: k = 1.

```
linear_algebra_ai999x.solve('low_rank_approximation', A='[[3, 0], [0, 1]]', k=1)
# -> approximation = [[3.0, 0.0], [0.0, 0.0]], error2 = 1, errorF = 1   (SOLVED)
```

```
linear_algebra_ai999x.solve('low_rank_approximation', A=[[2, 0, 0], [0, 3, 0], [0, 0, 1]], k=2)
# -> approximation = [[2.0, 0.0, 0.0], [0.0, 3.0, 0.0], [0.0, 0.0, 0.0]], error2 = 1, errorF = 1
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the best rank-1 approximation of [[3, 0], [0, 1]].	SOLVED	approximation = [[3.0, 0.0], [0.0, 0.0]], error2 = 1, errorF = 1
Find the best rank-1 approximation of [[1, 0], [0, 1]].	INVALID_INPUT	Best rank-k approximation (Eckart-Young): sigma_k = sigma_(k+1): the best rank-k approximation is not unique.

LU decomposition with partial pivoting — `lu_decomposition`

Factor a nonsingular square matrix as $A = P$ L U (permutation, unit lower triangular, upper triangular).

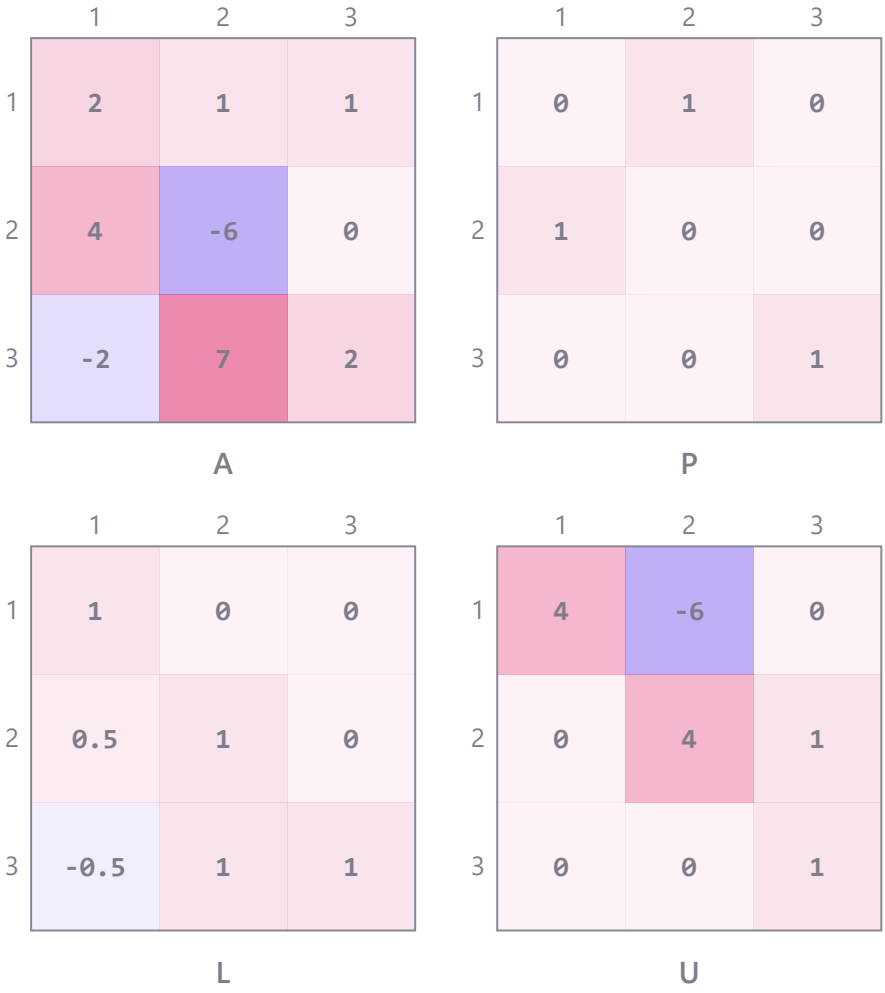
$$A = PLU$$

input	meaning	condition
A	a nonsingular square matrix	

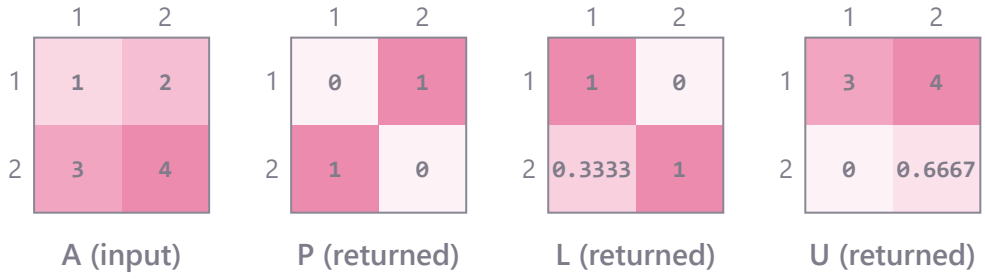
output	meaning
P	permutation matrix (A = P L U)
L	unit lower triangular factor, |l_ij| <= 1
U	upper triangular factor

- **Method:** Gaussian elimination with partial pivoting (Doolittle form) in floating point

- **Assumes:** A is square and nonsingular; at most 10 x 10; pivot: the first row of largest magnitude in the column (the scipy.linalg.lu convention $A = P$ L U)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The LU factorisation with partial pivoting returned by the package for A = [[2, 1, 1], [4, -6, 0], [-2, 7, 2]]: A = P L U with P a permutation, L unit lower triangular and U upper triangular (cells shaded by magnitude; negative values in the second colour). The product P L U, also computed by the package, equals A.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); P (returned); L (returned); U (returned).

```
linear_algebra_ai999x.solve('lu_decomposition', A='[[1, 2], [3, 4]]')
# -> P = [[0.0, 1.0], [1.0, 0.0]], L = [[1.0, 0.0], [0.3333333333333333, 1.0]], U = [[3.0, 4.0],
[0.0, 0.6666666666666667]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('lu_decomposition', A=[[2, 1, 1], [4, 3, 3], [8, 7, 9]])
# -> P = [[0.0, 1.0, 0.0], [0.0, 0.0, 1.0], [1.0, 0.0, 0.0]], L = [[1.0, 0.0, 0.0], [0.25, 1.0,
0.0], [0.5, 0.6666666666666666, 1.0]], U = [[8.0, 7.0, 9.0], [0.0, -0.75, -1.25], [0.0, 0.0,
-0.6666666666666667]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the LU decomposition of [[1, 2], [3, 4]].	SOLVED	P = [[0.0, 1.0], [1.0, 0.0]], L = [[1.0, 0.0], [0.3333333333333333, 1.0]], U = [[3.0, 4.0], [0.0, 0.6666666666666667]]
Find the LU decomposition of [[1, 2], [2, 4]].	INVALID_INPUT	LU decomposition with partial pivoting: A is singular: this capability factors nonsingular matrices.

QR decomposition (Householder) — `qr_decomposition`

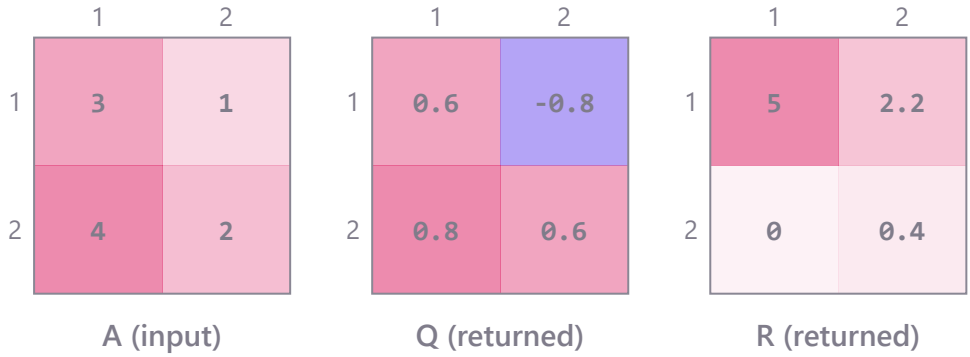
Factor $A = Q$ R with orthonormal columns Q and upper triangular R (reduced form, positive diagonal of R).

$$A = QR,\ Q^TQ = I$$

input	meaning	condition
`A`	a matrix with at least as many rows as columns, independent columns	

output	meaning
`Q`	m x n matrix with orthonormal columns
`R`	n x n upper triangular, positive diagonal

- **Method:** Householder reflections, then signs fixed so that diag(R) > 0
- **Assumes:** rows >= columns; independent columns; at most 10 x 10; convention: the diagonal of R is positive (unique)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Q (returned); R (returned).

```
linear_algebra_ai999x.solve('qr_decomposition', A='[[3, 1], [4, 2]]')
# -> Q = [[0.5999999999999999, -0.7999999999999999], [0.7999999999999999, 0.6000000000000001]], R =
[[5.0, 2.1999999999999997], [-0.0, 0.40000000000000013]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('qr_decomposition', A=[[1, 0], [0, 1], [0, 0]])
# -> Q = [[1.0, -0.0], [-0.0, 1.0], [-0.0, -0.0]], R = [[1.0, -0.0], [-0.0, 1.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the QR decomposition of [[3, 1], [4, 2]].	SOLVED	Q = [[0.5999999999999999, -0.7999999999999999], [0.7999999999999999, 0.6000000000000001]], R = [[5.0, 2.1999999999999997], [-0.0, 0.40000000000000013]]
Find the QR decomposition of [[1, 2], [2, 4]].	INVALID_INPUT	QR decomposition (Householder): the columns of A are linearly dependent.

Gram-Schmidt orthonormalization — `gram_schmidt`

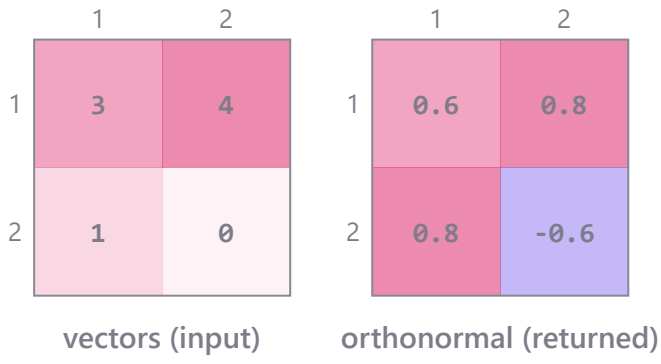
Turn linearly independent vectors v1, ..., vk into orthonormal vectors e1, ..., ek spanning the same nested subspaces.

$$\mathbf{w}_j = \mathbf{v}_j - \sum_{i<j} (\mathbf{e}_i \cdot \mathbf{v}_j)\, \mathbf{e}_i, \quad \mathbf{e}_j = \mathbf{w}_j / \|\mathbf{w}_j\|$$

input	meaning	condition
`vectors`	the vectors, one per row (independent)	

output	meaning
`orthonormal`	orthonormal vectors e1, ..., ek, one per row

- **Method:** modified Gram-Schmidt with re-orthogonalization (e_j . $v_j > 0$ by construction)
- **Assumes:** the vectors are linearly independent; at most 10 vectors of at most 10 entries
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): vectors (input); orthonormal (returned).

```
linear_algebra_ai999x.solve('gram_schmidt', vectors='3 4; 1 0')
# -> orthonormal = [[0.6, 0.8], [0.7999999999999999, -0.6]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('gram_schmidt', vectors=[[1, 0, 0], [1, 1, 0]])
# -> orthonormal = [[1.0, 0.0, 0.0], [0.0, 1.0, 0.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply Gram-Schmidt to the vectors (3, 4) and (1, 0).	SOLVED	orthonormal = [[0.6, 0.8], [0.7999999999999999, -0.6]]
Apply Gram-Schmidt to the vectors (1, 2) and (2, 4).	INVALID_INPUT	Gram-Schmidt orthonormalization: the vectors must be linearly independent.

Cholesky decomposition — `cholesky`

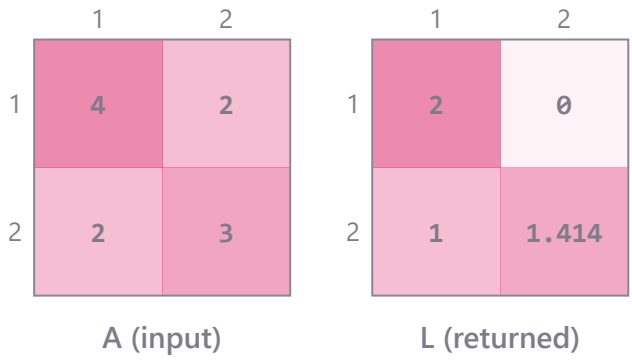
Factor a symmetric positive definite matrix as $A = L\,L^T$ with L lower triangular, positive diagonal.

$$A = LL^T$$

input	meaning	condition
A	a symmetric positive definite matrix	

output	meaning
L	lower triangular factor with positive diagonal

- **Method:** the Cholesky-Banachiewicz recursion $l_{jj} = sqrt$(a_jj - sum l_jk^2), l_ij = (a_ij - sum l_ik l_jk)/l_jj
- **Assumes:** A is symmetric; A is positive definite (checked exactly by its leading minors); at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); L (returned).

```
linear_algebra_ai999x.solve('cholesky', A='[[4, 2], [2, 3]]')
# -> L = [[2.0, 0.0], [1.0, 1.4142135623730951]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('cholesky', A=[[4, 12, -16], [12, 37, -43], [-16, -43, 98]])
# -> L = [[2.0, 0.0, 0.0], [6.0, 1.0, 0.0], [-8.0, 5.0, 3.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Cholesky factor of the matrix [4 12 -16; 12 37 -43; -16 -43 98].	SOLVED	L = [[2.0, 0.0, 0.0], [6.0, 1.0, 0.0], [-8.0, 5.0, 3.0]]
Find the Cholesky decomposition of [[1, 2], [2, 1]].	INVALID_INPUT	Cholesky decomposition: A is not positive definite (a leading principal minor is not positive).
Find the Cholesky decomposition of [[4, 1], [2, 3]].	INVALID_INPUT	Cholesky decomposition: A must be symmetric.

Positive definiteness test (Sylvester's criterion) — `positive_definiteness`

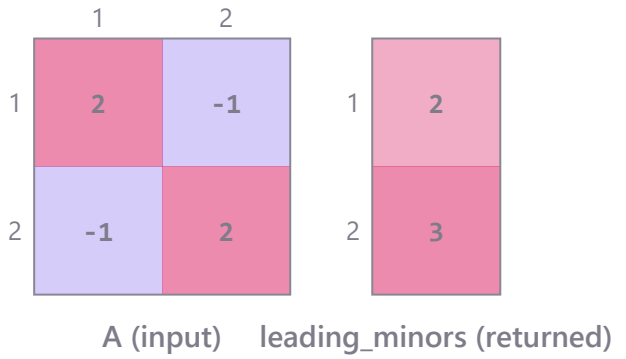
Whether a symmetric matrix is positive definite, with its leading principal minors.

$$A \succ 0 \iff D_k = \det A_{[1..k],[1..k]} > 0,\ k = 1..n$$

input	meaning	condition
`A`	a symmetric matrix	

output	meaning
`positive_definite`	true when x^T A $x > 0$ for every $x \neq 0$
`leading_minors`	the leading principal minors D1, ..., Dn

- **Method:** Sylvester's criterion: positive definite exactly when every leading principal minor is positive (exact determinants)
- **Assumes:** A is symmetric; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); leading_minors (returned).

```
linear_algebra_ai999x.solve('positive_definiteness', A='[[2, -1], [-1, 2]]')
# -> positive_definite = True, leading_minors = [2.0, 3.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('positive_definiteness', A=[[1, 2], [2, 1]])
# -> positive_definite = False, leading_minors = [1.0, -3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the matrix [[2, -1], [-1, 2]] positive definite?	SOLVED	positive_definite = True, leading_minors = [2.0, 3.0]
Is [1 2; 2 1] positive definite?	SOLVED	positive_definite = False, leading_minors = [1.0, -3.0]
Is the matrix [[2, 1], [0, 2]] positive definite?	INVALID_INPUT	Positive definiteness test (Sylvester's criterion): A must be symmetric.

Eigenvalues of a symmetric matrix (Jacobi method) — `symmetric_eigenvalues`

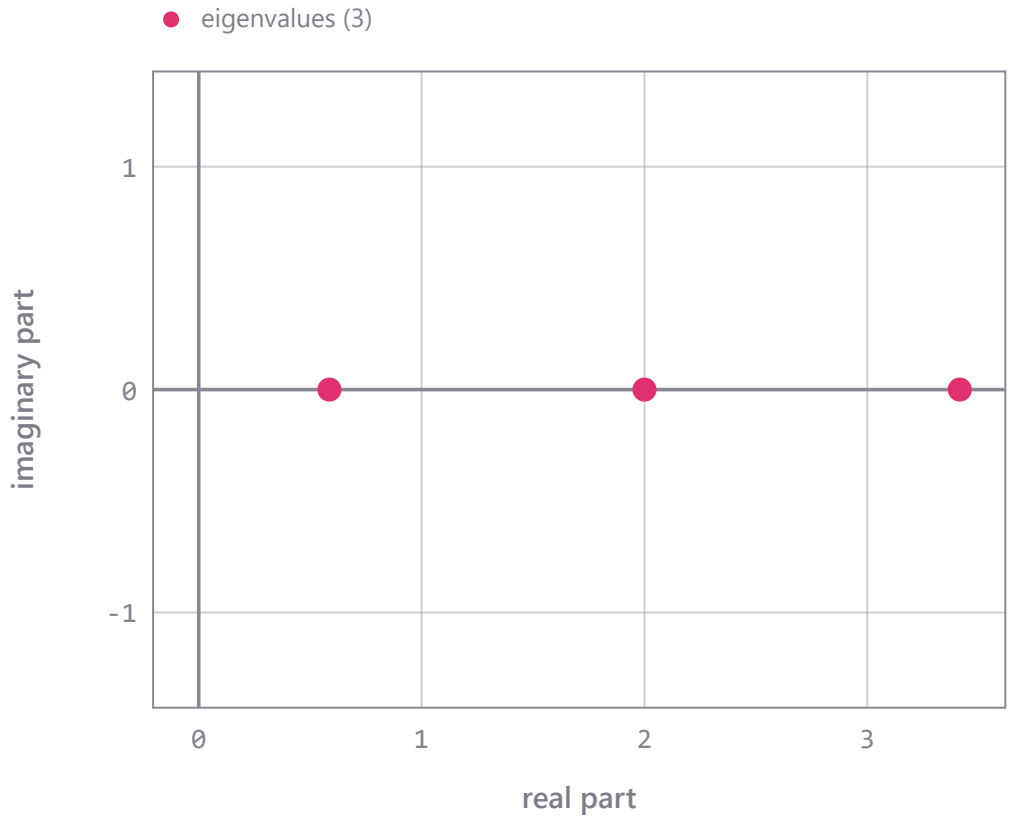
All eigenvalues of a real symmetric matrix (they are real), in ascending order.

$$A = Q \Lambda Q^T$$

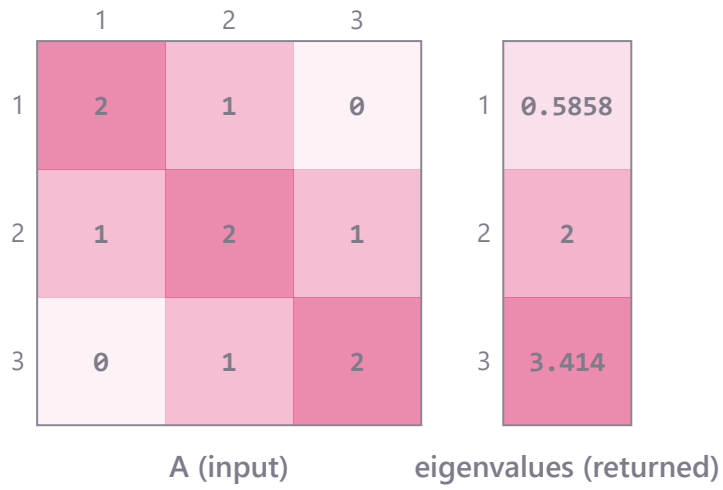
input	meaning	condition
`A`	a symmetric matrix (up to 10 x 10)	

output	meaning
`eigenvalues`	eigenvalues, ascending

- **Method:** cyclic Jacobi rotations until the off-diagonal part vanishes
- **Assumes:** A is symmetric; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



eigenvalues, ascending (eigenvalues) returned by linear-algebra-ai999x for the worked example (A = [[2, 1, 0], [1, 2, 1], [0, 1, 2]]), drawn in the complex plane: 0.585786 + 0i, 2 + 0i, 3.41421 + 0i.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); eigenvalues (returned).

```
linear_algebra_ai999x.solve('symmetric_eigenvalues', A='[[2, 1, 0], [1, 2, 1], [0, 1, 2]]')
# -> eigenvalues = [0.585786437626905, 1.999999999999999, 3.4142135623730936]   (SOLVED)
```

```
linear_algebra_ai999x.solve('symmetric_eigenvalues', A=[[2, 0, 0], [0, 3, 4], [0, 4, 9]])
# -> eigenvalues = [1.0, 2.0, 10.999999999999998]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the eigenvalues of [[2, 1, 0], [1, 2, 1], [0, 1, 2]].	SOLVED	eigenvalues = [0.585786437626905, 1.999999999999999, 3.4142135623730936]

Eigenvalues and eigenvectors of a symmetric matrix — `symmetric_eigenvectors`

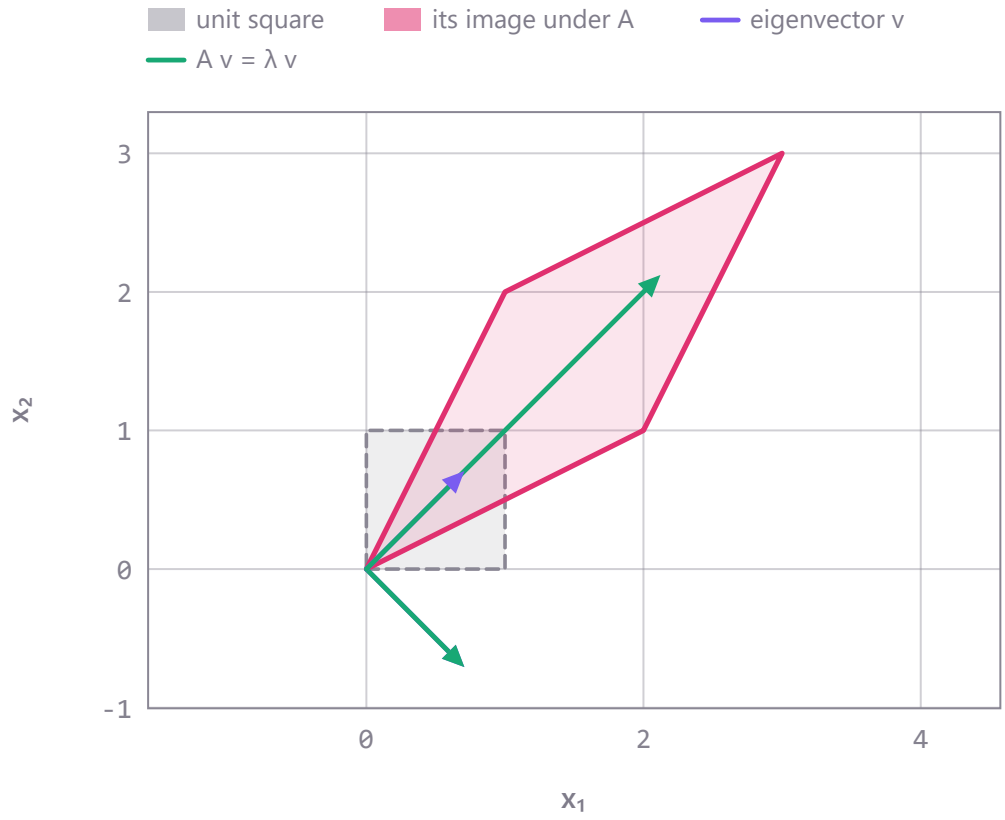
The orthonormal eigenvectors of a real symmetric matrix with distinct eigenvalues (spectral decomposition).

$$A\mathbf{q}_i = \lambda_i \mathbf{q}_i,\ Q^T Q = I$$

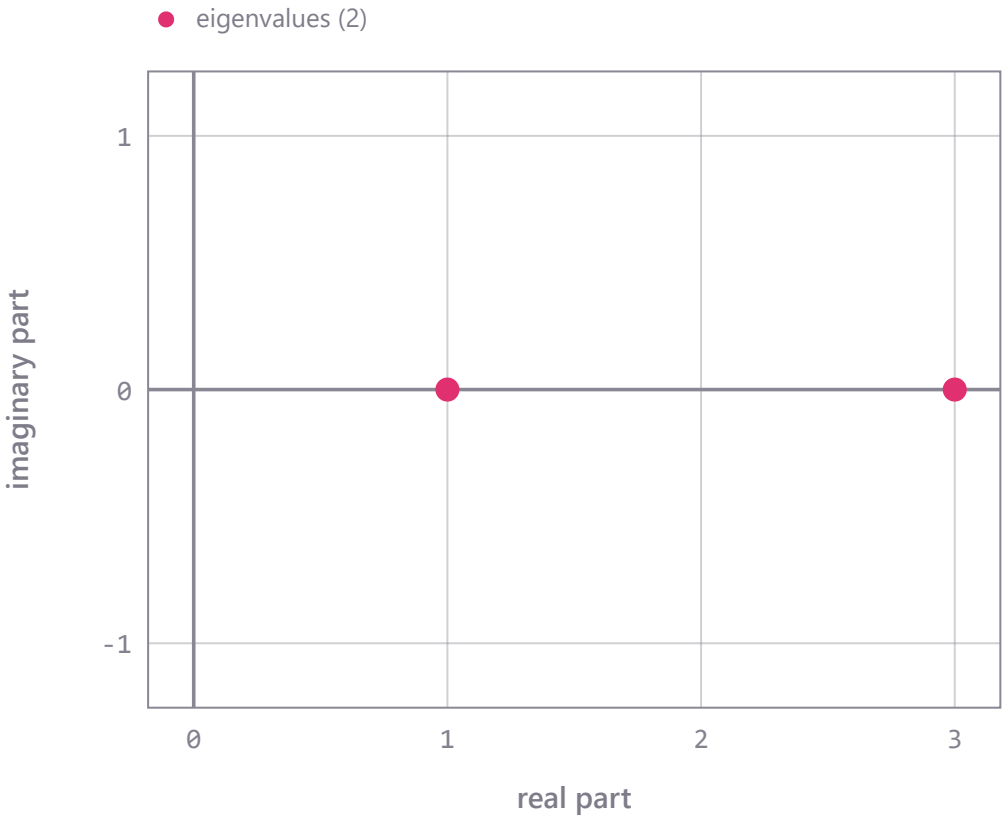
input	meaning	condition
`A`	a symmetric matrix with distinct eigenvalues	

output	meaning
`eigenvalues`	eigenvalues, ascending
`eigenvectors`	unit eigenvectors in the same order, one per row

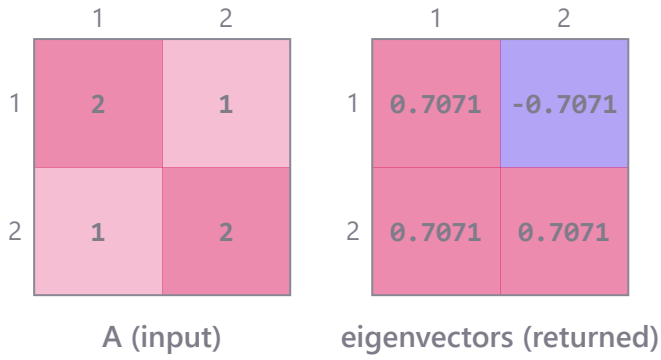
- **Method:** cyclic Jacobi rotations; the accumulated rotations are the eigenvectors
- **Assumes:** A is symmetric; the eigenvalues are distinct; at most 10 x 10; sign convention: the first entry of largest magnitude of each eigenvector is positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The unit square and its image under A = [[2, 1], [1, 2]] (each corner mapped by the package's matrix–vector product), with the eigenvectors it returned and their images: λ1 = 1 along (0.707107, -0.707107); λ2 = 3 along (0.707107, 0.707107).



eigenvalues, ascending (eigenvalues) returned by linear-algebra-ai999x for the worked example (A = [[2, 1], [1, 2]]), drawn in the complex plane: 1 + 0i, 3 + 0i.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); eigenvectors (returned).

```
linear_algebra_ai999x.solve('symmetric_eigenvectors', A='[[2, 1], [1, 2]]')
# -> eigenvalues = [0.9999999999999998, 2.9999999999999996], eigenvectors = [[0.7071067811865476,
-0.7071067811865476], [0.7071067811865476, 0.7071067811865476]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('symmetric_eigenvectors', A=[[2, 0, 0], [0, 3, 4], [0, 4, 9]])
# -> eigenvalues = [1.0, 2.0, 10.999999999999998], eigenvectors = [[0.0, 0.894427190999916,
-0.447213595499958], [1.0, 0.0, 0.0], [0.0, 0.447213595499958, 0.894427190999916]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the eigenvectors of [[2, 1], [1, 2]].	SOLVED	eigenvalues = [0.9999999999999998, 2.9999999999999996], eigenvectors = [[0.7071067811865476, -0.7071067811865476], [0.7071067811865476, 0.7071067811865476]]
Find the eigenvectors of [[1, 0], [0, 1]].	INVALID_INPUT	Eigenvalues and eigenvectors of a symmetric matrix: A has a repeated eigenvalue, so its eigenvectors are not determined
Find the eigenvectors of [[1, 2], [3, 4]].	INVALID_INPUT	Eigenvalues and eigenvectors of a symmetric matrix: A must be symmetric.

Eigenvalues of a general square matrix (QR algorithm) — `general_eigenvalues`

All eigenvalues (real or complex) of a real square matrix, and its spectral radius.

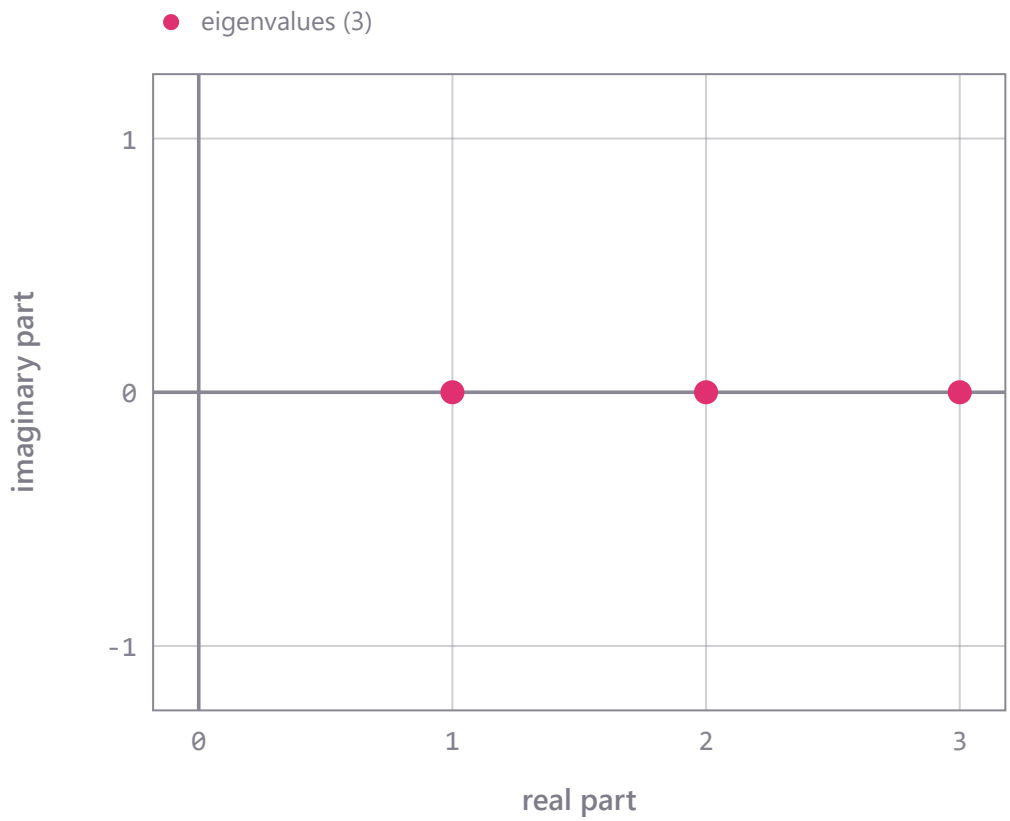
$$\det(A - \lambda I) = 0, \quad \rho(A) = \max_i |\lambda_i|$$

input	meaning	condition
`A`	a square matrix (up to 10 x 10)	

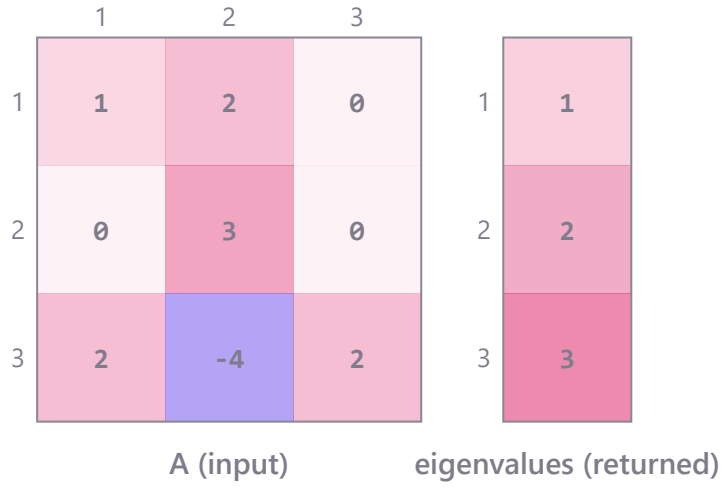
output	meaning
`eigenvalues`	eigenvalues: real ones ascending, then complex pairs by real part, positive imaginary part first
`spectral_radius`	largest |eigenvalue|

- **Method:** Householder reduction to Hessenberg form, then the shifted QR algorithm (Wilkinson shifts, Givens rotations, deflation)

- **Assumes:** A is square; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



eigenvalues: real ones ascending, then complex pairs by real part, positive imaginary part first (eigenvalues) returned by linear-algebra-ai999x for the worked example (A = [[1, 2, 0], [0, 3, 0], [2, -4, 2]]), drawn in the complex plane: 1 + 0i, 2 + 0i, 3 + 0i.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); eigenvalues (returned). Returned with them by linear-algebra-ai999x: spectral_radius = 3.

```
linear_algebra_ai999x.solve('general_eigenvalues', A='[[1, 2, 0], [0, 3, 0], [2, -4, 2]]')
# -> eigenvalues = [0.999999999999998, 1.9999999999999991, 3.0000000000000018], spectral_radius = 3
(SOLVED)
```

```
linear_algebra_ai999x.solve('general_eigenvalues', A=[[0, -1, 0], [1, 0, 0], [0, 0, 0.5]])
# -> eigenvalues = [0.5, 0.9999999999999998j, -0.9999999999999998j], spectral_radius = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the eigenvalues of [[1, 2, 0], [0, 3, 0], [2, -4, 2]].	SOLVED	eigenvalues = [0.999999999999998, 1.9999999999999991, 3.0000000000000018], spectral_radius = 3
What is the spectral radius of [[0, -1, 0], [1, 0, 0], [0, 0, 0.5]]?	SOLVED	eigenvalues = [0.5, 0.9999999999999998j, -0.9999999999999998j], spectral_radius = 1
Find the eigenvalues of [[1, 2, 3], [4, 5, 6]].	INVALID_INPUT	Eigenvalues of a general square matrix (QR algorithm): A must be square.

Dominant eigenvalue by power iteration — `power_iteration`

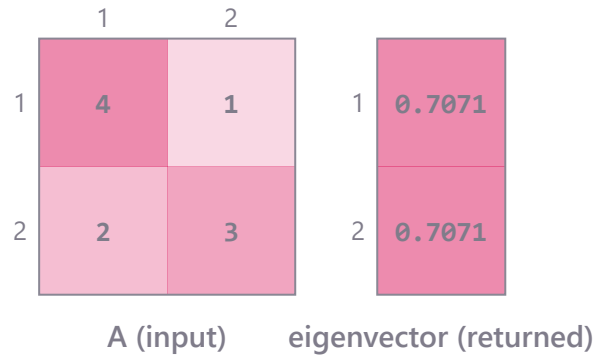
The eigenvalue of largest magnitude and its unit eigenvector, by the power method.

$$\mathbf{x}_{k+1} = \frac{A\mathbf{x}_k}{\|A\mathbf{x}_k\|}, \quad \lambda \approx \mathbf{x}_k^T A \mathbf{x}_k$$

input	meaning	condition
`A`	a square matrix with a single real dominant eigenvalue	

output	meaning
`eigenvalue`	the dominant eigenvalue
`eigenvector`	its unit eigenvector (first largest entry positive)

- **Method:** power iteration x <- A x / |A x| to convergence, polished by two steps of shifted inverse iteration; the dominance is checked against the QR-algorithm spectrum
- **Assumes:** A is square, at most 10 x 10; one real eigenvalue is strictly largest in magnitude
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); eigenvector (returned). Returned with them by linear-algebra-ai999x: eigenvalue = 5.

```
linear_algebra_ai999x.solve('power_iteration', A='[[4, 1], [2, 3]]')
# -> eigenvalue = 5, eigenvector = [0.7071067811865475, 0.7071067811865475]   (SOLVED)
```

```
linear_algebra_ai999x.solve('power_iteration', A=[[2, 0, 0], [0, -5, 0], [0, 0, 1]])
# -> eigenvalue = -5, eigenvector = [-4.646422177581031e-71, 1.0, -8.551349954144621e-79]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the dominant eigenvalue of [4 1; 2 3] by the power method.	SOLVED	eigenvalue = 5, eigenvector = [0.7071067811865475, 0.7071067811865475]
Use power iteration to find the dominant eigenvalue of [[0, 1], [-1, 0]].	INVALID_INPUT	Dominant eigenvalue by power iteration: there is no single real eigenvalue of largest magnitude, so the iteration does n

Smallest eigenvalue by inverse iteration — `inverse_iteration`

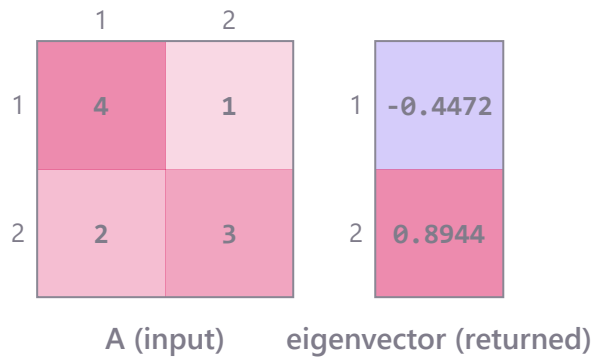
The eigenvalue of smallest magnitude and its unit eigenvector, by inverse power iteration.

$$A\mathbf{y}_{k+1} = \mathbf{x}_k, \quad \mathbf{x}_{k+1} = \mathbf{y}_{k+1}/\|\mathbf{y}_{k+1}\|$$

input	meaning	condition
`A`	a nonsingular square matrix with a single real eigenvalue of smallest magnitude	

output	meaning
`eigenvalue`	the eigenvalue of smallest magnitude
`eigenvector`	its unit eigenvector (first largest entry positive)

- **Method:** power iteration with A^{-1} (one LU factorization, a solve per step), polished by shifted inverse iteration; checked against the QR-algorithm spectrum
- **Assumes:** A is square and nonsingular, at most 10 x 10; one real eigenvalue is strictly smallest in magnitude
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); eigenvector (returned). Returned with them by linear-algebra-ai999x: eigenvalue = 2.

```
linear_algebra_ai999x.solve('inverse_iteration', A='[[4, 1], [2, 3]]')
# -> eigenvalue = 2, eigenvector = [-0.447213595499958, 0.894427190999916]   (SOLVED)
```

```
linear_algebra_ai999x.solve('inverse_iteration', A=[[2, 0, 0], [0, -5, 0], [0, 0, 1]])
# -> eigenvalue = 1, eigenvector = [1.195294316858053e-67, -7.939558564345683e-84, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the smallest eigenvalue of [4 1; 2 3] by inverse power iteration.	SOLVED	eigenvalue = 2, eigenvector = [-0.447213595499958, 0.894427190999916]
Use inverse iteration to find the smallest eigenvalue of [[1, 2], [2, 4]].	INVALID_INPUT	Smallest eigenvalue by inverse iteration: A is singular: its smallest eigenvalue is 0 and A cannot be inverted.

Matrix exponential — `matrix_exponential`

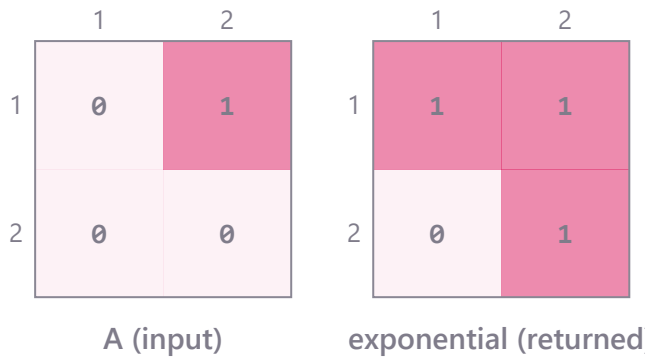
exp(A) = I + A + A^2/2! + ..., the solution operator of x' = A x.

$$e^{A} = \sum_{k=0}^{\infty} \frac{A^{k}}{k!} = \left(e^{A/2^{s}}\right)^{2^{s}}$$

input	meaning	condition
`A`	a square matrix with $\lvert A\rvert_1 \leq 50$	

output	meaning
`exponential`	the matrix exp(A)

- **Method:** scaling and squaring: a Taylor series for exp(A / 2^s) with $\lvert A/2^{s}\rvert_1 \leq 1/2$, then s squarings
- **Assumes:** A is square, at most 10 x 10; $\lvert A\rvert_1 \leq 50$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); exponential (returned).

```
linear_algebra_ai999x.solve('matrix_exponential', A='[[0, 1], [0, 0]]')
# -> exponential = [[1.0, 1.0], [0.0, 1.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_exponential', A=[[0, -1], [1, 0]])
# -> exponential = [[0.5403023058681398, -0.8414709848078965], [0.8414709848078965,
0.5403023058681398]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_exponential', A=[[1, 0], [0, 2]])
# -> exponential = [[2.718281828459048, 0.0], [0.0, 7.389056098930642]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the matrix exponential of [[0, 1], [0, 0]].	SOLVED	exponential = [[1.0, 1.0], [0.0, 1.0]]
Compute the exponential of the matrix [0 -1; 1 0].	SOLVED	exponential = [[0.5403023058681398, -0.8414709848078965], [0.8414709848078965, 0.5403023058681398]]

Square root of a positive definite matrix — `matrix_square_root`

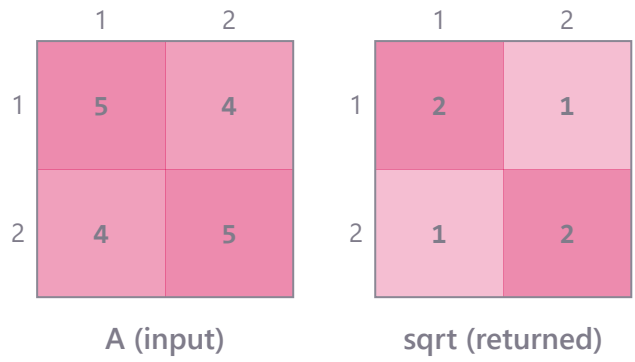
The unique symmetric positive definite X with $X^2 = A$, for a symmetric positive definite A.

$$A^{1/2} = Q\,\Lambda^{1/2} Q^T$$

input	meaning	condition
`A`	a symmetric positive definite matrix	

output	meaning
`sqrt`	the principal square root $A^{1/2}$

- **Method:** spectral decomposition by Jacobi rotations: $A^{1/2} = Q$ diag(sqrt(lambda)) Q^T
- **Assumes:** A is symmetric positive definite; at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); sqrt (returned).

```
linear_algebra_ai999x.solve('matrix_square_root', A='[[5, 4], [4, 5]]')
# -> sqrt = [[1.9999999999999991, 0.9999999999999996], [0.9999999999999996, 1.9999999999999991]]
(SOLVED)
```

```
linear_algebra_ai999x.solve('matrix_square_root', A=[[2, 0], [0, 9]])
# -> sqrt = [[1.4142135623730951, 0.0], [0.0, 3.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the square root of the matrix [[5, 4], [4, 5]].	SOLVED	sqrt = [[1.9999999999999991, 0.9999999999999996], [0.9999999999999996, 1.9999999999999991]]
Find the square root of the matrix [[1, 2], [2, 1]].	INVALID_INPUT	Square root of a positive definite matrix: A must be positive definite (a leading principal minor is not positive).

Polar decomposition — `polar_decomposition`

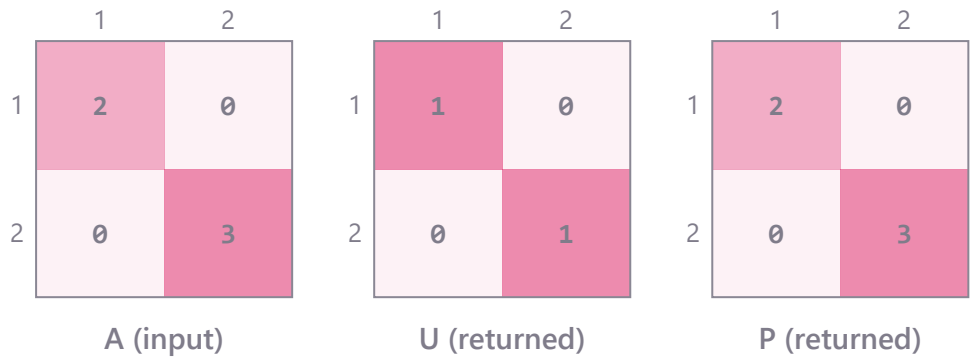
Write a nonsingular square matrix as $A = UP$ P: U orthogonal (a rotation or reflection), P symmetric positive definite (a stretch).

$$A = UP, \quad P = (A^T A)^{1/2}, \; U = AP^{-1}$$

input	meaning	condition
A	a nonsingular square matrix	

output	meaning
U	orthogonal factor
P	symmetric positive definite factor (A^T A)^(1/2)

- **Method:** from the one-sided Jacobi SVD $A = W$ S V^T: $U = W\,V^T$, $P = V$ S V^T
- **Assumes:** A is square and nonsingular; at most 10 x 10; right polar form $A = UP$ (scipy.linalg.polar's default)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); U (returned); P (returned).

```
linear_algebra_ai999x.solve('polar_decomposition', A='[[2, 0], [0, 3]]')
# -> U = [[1.0, 0.0], [0.0, 1.0]], P = [[2.0, 0.0], [0.0, 3.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('polar_decomposition', A=[[0, -2], [1, 0]])
# -> U = [[0.0, -1.0], [1.0, 0.0]], P = [[1.0, 0.0], [0.0, 2.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the polar decomposition of the matrix [0 -2; 1 0].	SOLVED	U = [[0.0, -1.0], [1.0, 0.0]], P = [[1.0, 0.0], [0.0, 2.0]]
Find the polar decomposition of [[1, 2], [2, 4]].	INVALID_INPUT	Polar decomposition: A is singular: its polar factor is not unique.

Jacobi iteration for Ax = b — `jacobi_iteration`

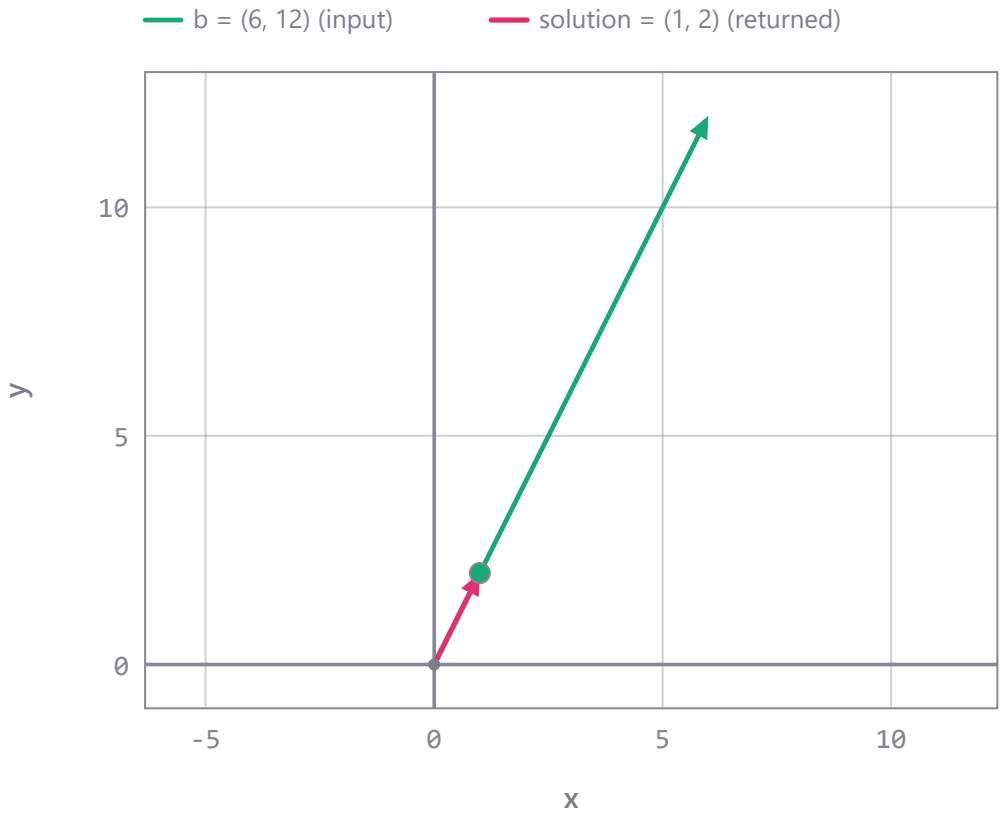
Solve $Ax = b$ for a strictly diagonally dominant A by the Jacobi method, iterated to convergence.

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}\right)$$

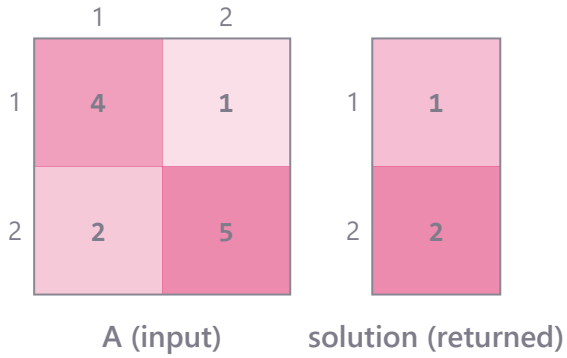
input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

output	meaning
`solution`	the converged solution x

- **Method:** x_i <- (b_i - sum_{$j \neq i$} a_ij x_j) / a_ii for all i at once, from $x = 0$, until the iterates stop changing
- **Assumes:** A is square and strictly diagonally dominant by rows (guarantees convergence); at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input b = (6, 12); returned by linear-algebra-ai999x: solution = (1, 2). Other inputs: A = [[4, 1], [2, 5]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); solution (returned). Other inputs: b = 6, 12.

```
linear_algebra_ai999x.solve('jacobi_iteration', A='[[4, 1], [2, 5]]', b='6, 12')
# -> solution = [1.0, 2.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('jacobi_iteration', A=[[10, -1, 2], [-1, 11, -1], [2, -1, 10]], b=[6,
25, -11])
# -> solution = [1.0432692307692308, 2.269230769230769, -1.0817307692307692]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use Jacobi iteration to solve Ax = b with A = [[4, 1], [2, 5]] and b = (6, 12).	SOLVED	solution = [1.0, 2.0]
Use Jacobi iteration to solve Ax = b with A = [[1, 2], [3, 4]] and b = (1, 1).	INVALID_INPUT	Jacobi iteration for Ax = b: A must be strictly diagonally dominant by rows (then the Jacobi iteration converges).

Gauss-Seidel iteration for Ax = b — `gauss_seidel`

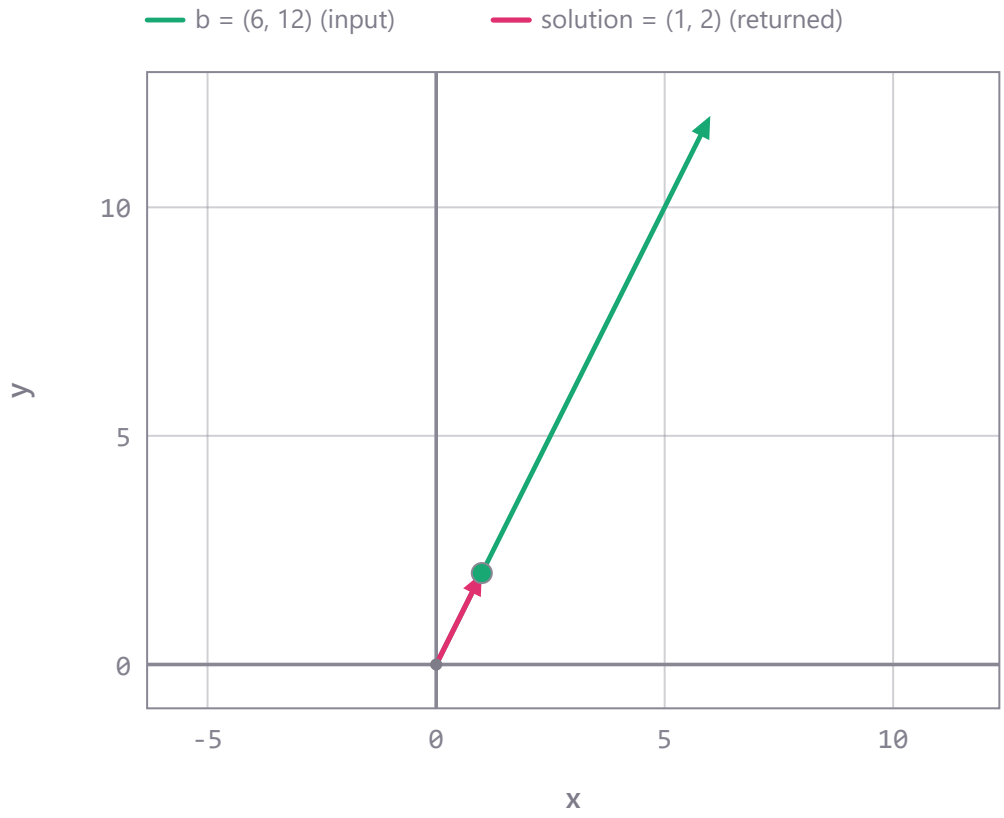
Solve $Ax = b$ by the Gauss-Seidel method (each new value used at once), iterated to convergence.

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right)$$

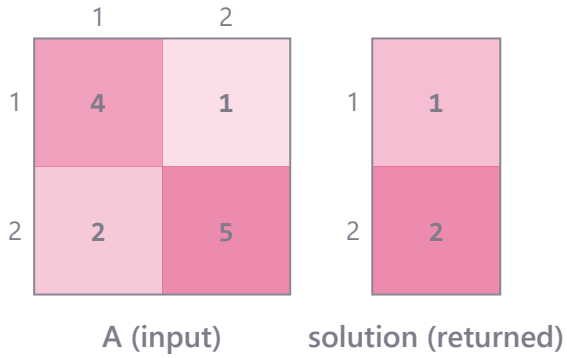
input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

output	meaning
`solution`	the converged solution x

- **Method:** x_i <- (b_i - sum_{$j < i$} a_ij x_j(new) - sum_{$j > i$} a_ij x_j(old)) / a_ii, from $x = 0$, until the iterates stop changing
- **Assumes:** A is square and strictly diagonally dominant, or symmetric positive definite (guarantees convergence); at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input b = (6, 12); returned by linear-algebra-ai999x: solution = (1, 2). Other inputs: A = [[4, 1], [2, 5]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); solution (returned). Other inputs: b = 6, 12.

```
linear_algebra_ai999x.solve('gauss_seidel', A='[[4, 1], [2, 5]]', b='6, 12')
# -> solution = [1.0, 2.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('gauss_seidel', A=[[4, -1, 0], [-1, 4, -1], [0, -1, 4]], b=[2, 4, 10])
# -> solution = [0.9999999999999999, 2.0, 3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use Gauss-Seidel to solve Ax = b with A = [[4, 1], [2, 5]] and b = (6, 12).	SOLVED	solution = [1.0, 2.0]

question	outcome	result
Use Gauss-Seidel to solve Ax = b with A = [[1, 3], [3, 1]] and b = (1, 1).	INVALID_INPUT	Gauss-Seidel iteration for Ax = b: A must be strictly diagonally dominant or symmetric positive definite (then Gauss-Sei

Conjugate gradient method for Ax = b — `conjugate_gradient`

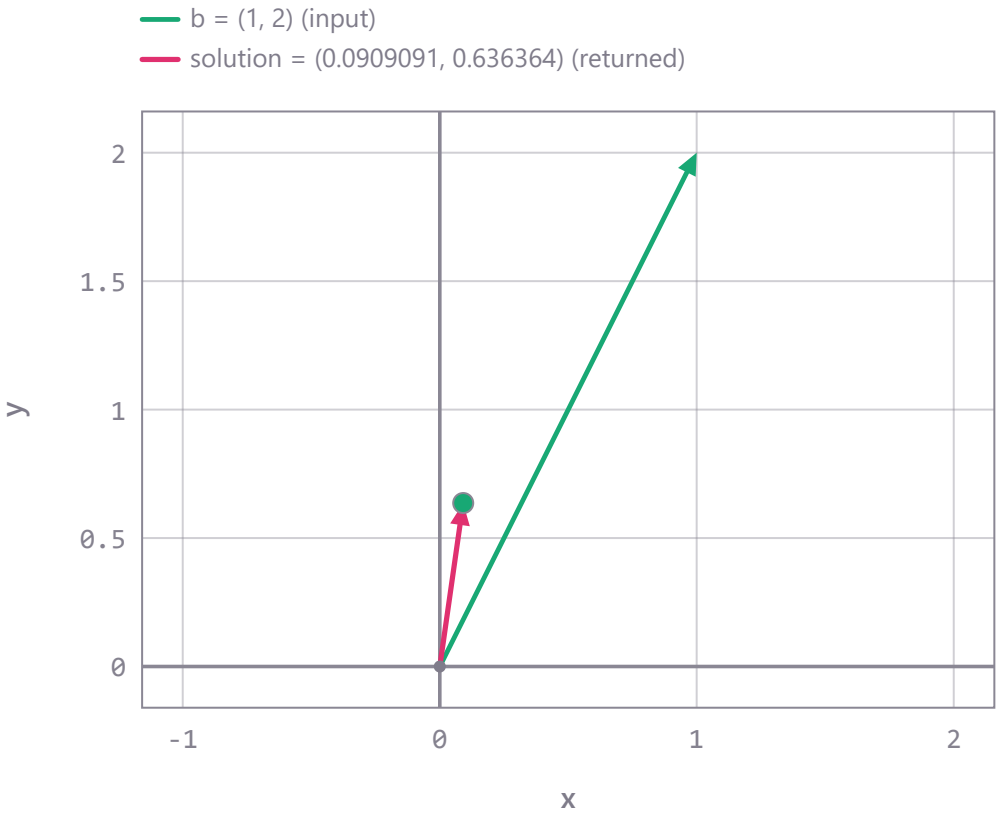
Solve $Ax = b$ for a symmetric positive definite A by the method of conjugate gradients.

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k},\ \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k,\ \mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k} \mathbf{p}_k$$

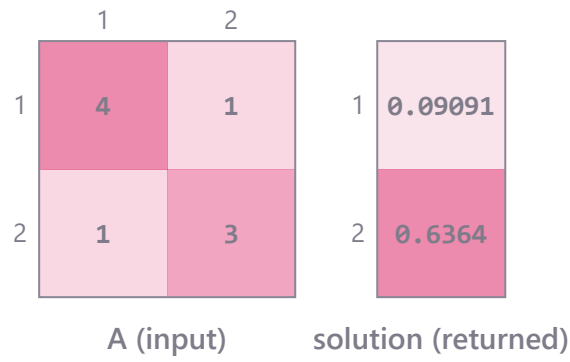
input	meaning	condition
`A`	coefficient matrix	
`b`	right-hand side	

output	meaning
`solution`	the solution x

- **Method:** conjugate gradients from $x = 0$ (at most n steps in exact arithmetic), restarted twice to remove rounding errors
- **Assumes:** A is symmetric positive definite (checked exactly); at most 10 x 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input b = (1, 2); returned by linear-algebra-ai999x: solution = (0.0909091, 0.636364). Other inputs: A = [[4, 1], [1, 3]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); solution (returned). Other inputs: b = 1, 2.

```
linear_algebra_ai999x.solve('conjugate_gradient', A='[[4, 1], [1, 3]]', b='1, 2')
# -> solution = [0.09090909090909091, 0.6363636363636364]   (SOLVED)
```

```
linear_algebra_ai999x.solve('conjugate_gradient', A=[[2, -1, 0], [-1, 2, -1], [0, -1, 2]], b=[1, 0, 1])
# -> solution = [1.0, 1.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use conjugate gradients to solve Ax = b with A = [[4, 1], [1, 3]] and b = (1, 2).	SOLVED	solution = [0.09090909090909091, 0.6363636363636364]
Use conjugate gradients to solve Ax = b with A = [[1, 2], [2, 1]] and b = (1, 2).	INVALID_INPUT	Conjugate gradient method for Ax = b: A must be symmetric positive definite.

Tridiagonal system (Thomas algorithm) — `tridiagonal_solve`

Solve a tridiagonal system given its three diagonals and right-hand side, in O(n) operations.

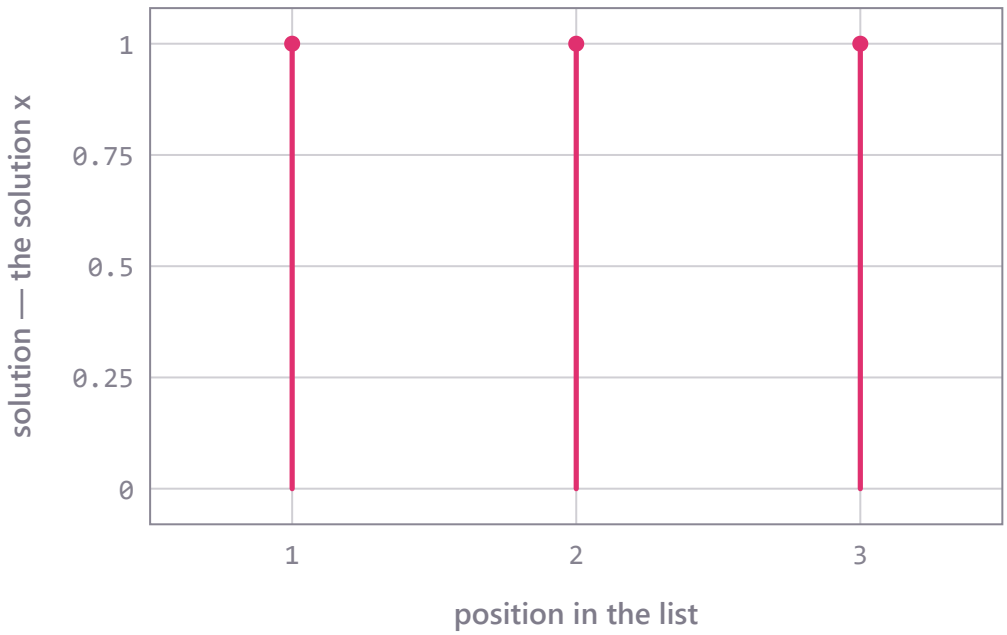
$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$$

input	meaning	condition
`sub`	sub-diagonal a_2..a_n (n-1 entries)	
`diag`	main diagonal b_1..b_n	
`sup`	super-diagonal c_1..c_(n-1) (n-1 entries)	
`d`	right-hand side (n entries)	

output	meaning
`solution`	the solution x

- **Method:** the Thomas algorithm (forward sweep, back substitution) in exact rational arithmetic
- **Assumes:** $n \geq 2$, at most 200; no zero pivot arises in the elimination (true for diagonally dominant matrices)

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the solution x (solution) returned by linear-algebra-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 1, largest 1). Inputs: sub = 1, 1, diag = 4, 4, 4, sup = 1, 1, d = 5, 6, 5.

```
linear_algebra_ai999x.solve('tridiagonal_solve', sub='1, 1', diag='4, 4, 4', sup='1, 1', d='5, 6, 5')
# -> solution = [1.0, 1.0, 1.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('tridiagonal_solve', sub=[-1, -1, -1], diag=[2, 2, 2, 2], sup=[-1, -1, -1], d=[1, 0, 0, 1])
# -> solution = [1.0, 1.0, 1.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the tridiagonal system with sub-diagonal (1, 1), diagonal (4, 4, 4), super-diagonal (1, 1) and right-hand side (5, 6, 5).	SOLVED	solution = [1.0, 1.0, 1.0]
Solve the tridiagonal system with sub-diagonal (1, 1), diagonal (4, 4, 4), super-diagonal (1, 1, 1) and right-hand side (5, 6, 5).	INVALID_INPUT	Tridiagonal system (Thomas algorithm): sup must have exactly 2 entries.

Compressed sparse row (CSR) form of a matrix — `csr_format`

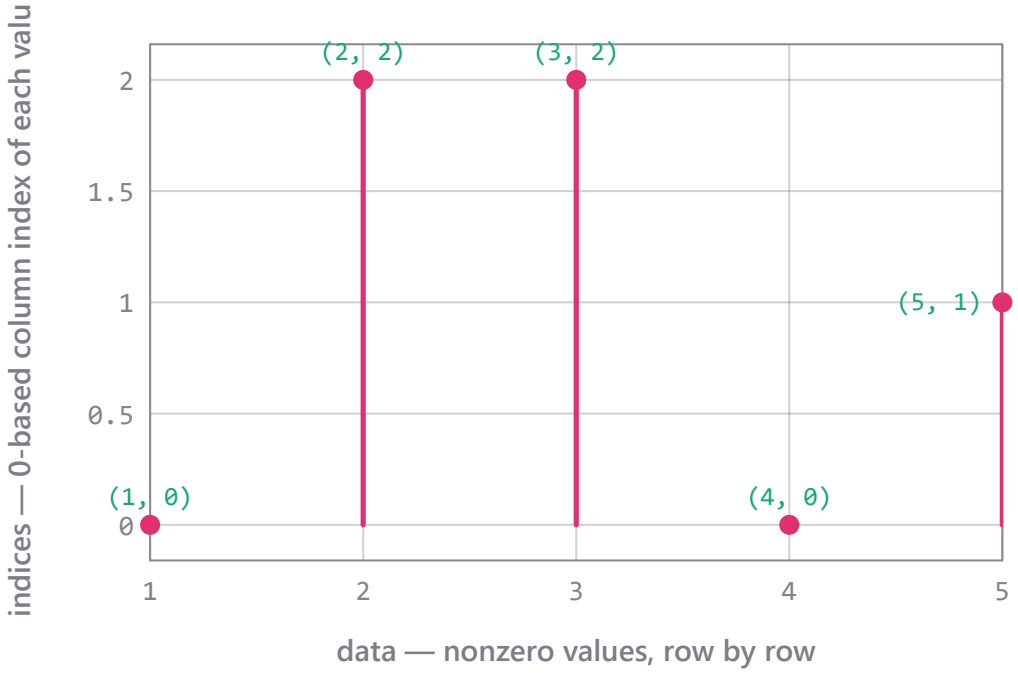
Store a matrix in compressed sparse row form: the nonzero values, their column indices and the row pointers.

$$\texttt{data}[\texttt{indptr}[k] : \texttt{indptr}[k+1]]$$

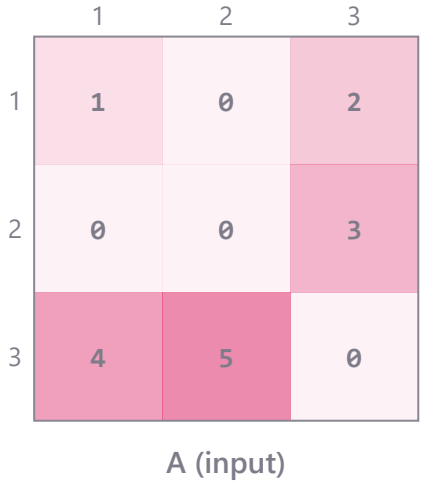
input	meaning	condition
`A`	the matrix (up to 50 x 50)	

output	meaning
`data`	nonzero values, row by row
`indices`	0-based column index of each value
`indptr`	row pointers: the values of row k are data[indptr[k]:indptr[$k+1$]]
`nnz`	number of stored nonzeros
`density`	nnz / (rows x columns)

- **Method:** scan row by row, keep the nonzeros
- **Assumes:** at most 50 x 50; zeros are not stored; indices sorted within each row (canonical form)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



0-based column index of each value (indices) at each nonzero values, row by row (data), both returned by linear-algebra-ai999x for the worked example (A = [[1, 0, 2], [0, 0, 3], [4, 5, 0]]): 1 → 0; 2 → 2; 3 → 2; 4 → 0; 5 → 1.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: nnz = 5, density = 0.555556.

```
linear_algebra_ai999x.solve('csr_format', A='[[1, 0, 2], [0, 0, 3], [4, 5, 0]]')
# -> data = [1.0, 2.0, 3.0, 4.0, 5.0], indices = [0, 2, 2, 0, 1], indptr = [0, 2, 3, 5], nnz = 5,
density = 0.555556   (SOLVED)
```

```
linear_algebra_ai999x.solve('csr_format', A=[[0, 7], [0, 0], [8, 0]])
# -> data = [7.0, 8.0], indices = [1, 0], indptr = [0, 1, 1, 2], nnz = 2, density = 0.333333
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert [[1, 0, 2], [0, 0, 3], [4, 5, 0]] to CSR format.	SOLVED	data = [1.0, 2.0, 3.0, 4.0, 5.0], indices = [0, 2, 2, 0, 1], indptr = [0, 2, 3, 5], nnz = 5, density = 0.555556

Compressed sparse column (CSC) form of a matrix — `csc_format`

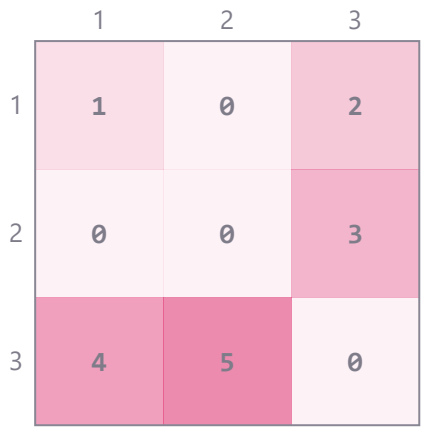
Store a matrix in compressed sparse column form: the nonzero values, their row indices and the column pointers.

$$\mathtt{data}[\mathtt{indptr}[k] : \mathtt{indptr}[k+1]]$$

input	meaning	condition
`A`	the matrix (up to 50 x 50)	

output	meaning
`data`	nonzero values, column by column
`indices`	0-based row index of each value
`indptr`	column pointers: the values of column k are data[indptr[k]:indptr[$k+1$]]
`nnz`	number of stored nonzeros
`density`	nnz / (rows x columns)

- **Method:** scan column by column, keep the nonzeros
- **Assumes:** at most 50 x 50; zeros are not stored; indices sorted within each column (canonical form)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by linear-algebra-ai999x: nnz = 5, density = 0.555556.

```
linear_algebra_ai999x.solve('csc_format', A='[[1, 0, 2], [0, 0, 3], [4, 5, 0]]')
# -> data = [1.0, 4.0, 5.0, 2.0, 3.0], indices = [0, 2, 2, 0, 1], indptr = [0, 2, 3, 5], nnz = 5,
density = 0.555556   (SOLVED)
```

```
linear_algebra_ai999x.solve('csc_format', A=[[0, 7], [0, 0], [8, 0]])
# -> data = [8.0, 7.0], indices = [2, 0], indptr = [0, 1, 2], nnz = 2, density = 0.333333
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert [[1, 0, 2], [0, 0, 3], [4, 5, 0]] to CSC format.	SOLVED	data = [1.0, 4.0, 5.0, 2.0, 3.0], indices = [0, 2, 2, 0, 1], indptr = [0, 2, 3, 5], nnz = 5, density = 0.555556

Dense matrix from CSR arrays — `csr_to_dense`

Rebuild the full matrix from compressed sparse row arrays (data, indices, indptr) and the number of columns.

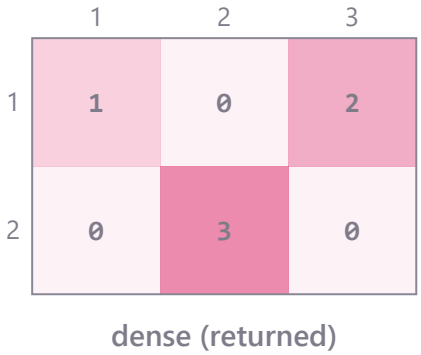
$$A_{r,\,\mathtt{indices}[k]} \mathrel{+}= \mathtt{data}[k],\ \mathtt{indptr}[r] \le k < \mathtt{indptr}[r+1]$$

input	meaning	condition
`data`	stored values	
`indices`	0-based column index of each value	
`indptr`	row pointers (rows + 1 entries)	
`ncols`	number of columns	

output	meaning
`dense`	the full matrix

- **Method:** row r holds data[indptr[r]:indptr[$r+1$]] at columns indices[indptr[r]:indptr[$r+1$]]

- **Assumes:** indptr starts at 0, never decreases, ends at len(data); indices within 0..ncols-1; duplicate entries are summed; at most 50 x 50
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dense (returned)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): dense (returned). Other inputs: data = 1, 2, 3, indices = 0, 2, 1, indptr = 0, 2, 3, ncols = 3.

```
linear_algebra_ai999x.solve('csr_to_dense', data='1, 2, 3', indices='0, 2, 1', indptr='0, 2, 3',
ncols=3)
# -> dense = [[1.0, 0.0, 2.0], [0.0, 3.0, 0.0]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('csr_to_dense', data=[5], indices=[1], indptr=[0, 0, 1], ncols=2)
# -> dense = [[0.0, 0.0], [0.0, 5.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Build the dense matrix with 3 columns from the CSR arrays data = [1, 2, 3], indices = [0, 2, 1], indptr = [0, 2, 3].	SOLVED	dense = [[1.0, 0.0, 2.0], [0.0, 3.0, 0.0]]
Build the dense matrix with 2 columns from the CSR arrays data = [1, 2, 3], indices = [0, 2, 1], indptr = [0, 2, 3].	INVALID_INPUT	Dense matrix from CSR arrays: every index must be a whole number from 0 to ncols - 1 (at most 50 rows).

Sparse matrix times vector (CSR) — `sparse_matvec`

The product $y = A$ x for a matrix A given in compressed sparse row form.

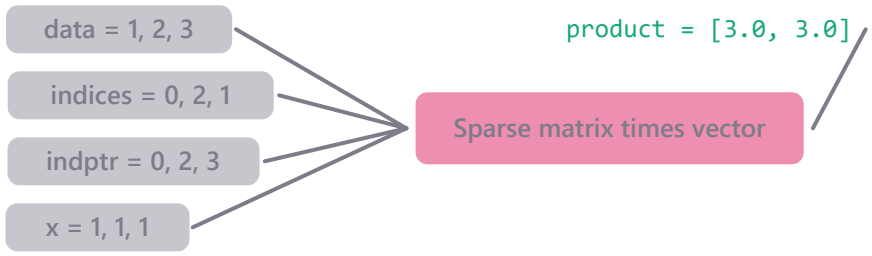
$$y_r = \sum_{k=\mathtt{indptr}[r]}^{\mathtt{indptr}[r+1]-1} \mathtt{data}[k]\, x_{\mathtt{indices}[k]}$$

input	meaning	condition
`data`	stored values	
`indices`	0-based column index of each value	
`indptr`	row pointers (rows + 1 entries)	

input	meaning	condition
`x`	the vector (one entry per column)	

output	meaning
`product`	the vector A x

- **Method:** y_r = sum over the stored entries of row r of data[k] x[indices[k]] (exact)
- **Assumes:** consistent CSR arrays; column indices below len(x); at most 50 x 50
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Sparse matrix times vector (CSR) as inputs and results: stated data = 1, 2, 3, indices = 0, 2, 1, indptr = 0, 2, 3, x = 1, 1, 1; returned by linear-algebra-ai999x: product = [3.0, 3.0].

```
linear_algebra_ai999x.solve('sparse_matvec', data='1, 2, 3', indices='0, 2, 1', indptr='0, 2, 3',
x='1, 1, 1')
# -> product = [3.0, 3.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('sparse_matvec', data=[2, -1], indices=[0, 1], indptr=[0, 1, 2], x=[3,
4])
# -> product = [6.0, -4.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiply the CSR matrix data = [1, 2, 3], indices = [0, 2, 1], indptr = [0, 2, 3] by x = [1, 1, 1].	SOLVED	product = [3.0, 3.0]
Multiply the CSR matrix data = [1, 2, 3], indices = [0, 2, 1], indptr = [0, 2, 3] by x = [1, 1].	INVALID_INPUT	Sparse matrix times vector (CSR): every index must be a whole number from 0 to ncols - 1 (at most 50 rows).

1-D Poisson equation by finite differences — `poisson_1d`

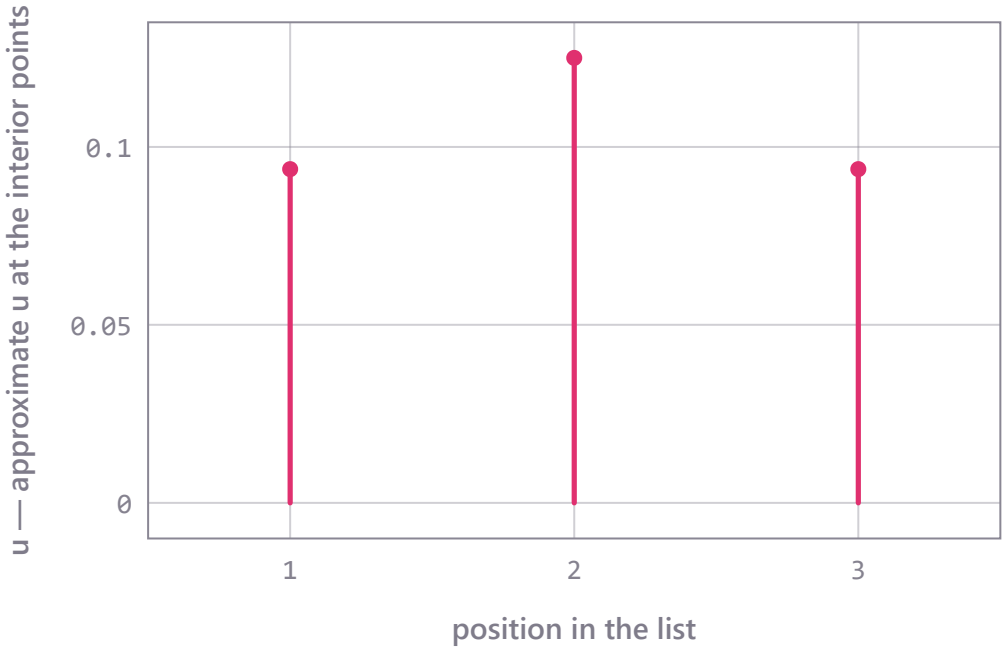
Solve -u'' = f on (0, L) with u(0) = u(L) = 0 on n equally spaced interior points (second-order finite differences).

$$\frac{-u_{k-1} + 2u_k - u_{k+1}}{h^2} = f_k, \quad u_0 = u_{n+1} = 0$$

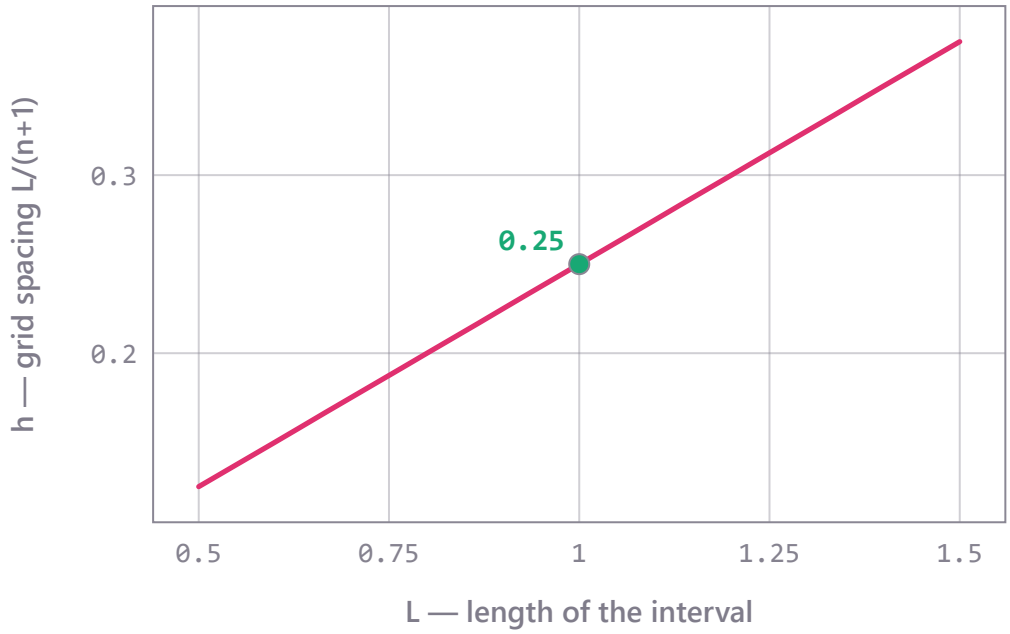
input	meaning	condition
`f`	values of f at the n interior grid points $x_k = k\,L/(n+1)$	
`L`	length of the interval	

output	meaning
`u`	approximate u at the interior points
`h`	grid spacing $L/(n+1)$

- **Method:** the tridiagonal system (-u_(k-1) + 2u_k - u_(k+1))/$h^2 = f_k$ solved exactly by the Thomas algorithm
- **Assumes:** $L > 0$; $u = 0$ at both ends (homogeneous Dirichlet conditions); at most 200 interior points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



approximate u at the interior points (u) returned by linear-algebra-ai999x for the worked example, one value for each position in the list (3 values, first 0.09375, last 0.09375, largest 0.125). Inputs: f = 1, 1, 1, L = 1.



h (grid spacing L/(n+1)) against L (length of the interval) from 0.5 to 1.5. Other inputs as in the worked example: f = 1, 1, 1. 31 points computed by linear-algebra-ai999x; marked: the worked example, L = 1, h = 0.25.

```
linear_algebra_ai999x.solve('poisson_1d', f='1, 1, 1', L=1)
# -> u = [0.09375, 0.125, 0.09375], h = 0.25   (SOLVED)
```

```
linear_algebra_ai999x.solve('poisson_1d', f=[2, 2, 2], L=4)
# -> u = [3.0, 4.0, 3.0], h = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the 1-D Poisson equation -u'' = f with zero boundary values, L = 1 and f = (1, 1, 1) at the interior points.	SOLVED	u = [0.09375, 0.125, 0.09375], h = 0.25
Solve the one-dimensional Poisson problem with u(0) = u(L) = 0, L = -4 and f = [2, 2, 2].	INVALID_INPUT	1-D Poisson equation by finite differences: the interval length L must be positive.

2-D Poisson equation on a grid (5-point stencil) — `poisson_2d`

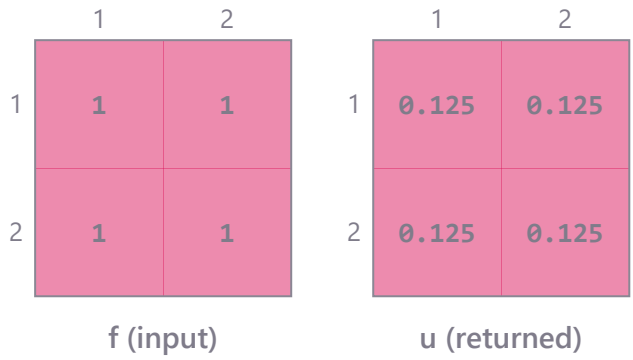
Solve -(u_xx + u_yy) = f on a rectangle with $u = 0$ on the boundary, on an n x m grid of interior points with spacing h.

$$\frac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2} = f_{i,j}$$

input	meaning	condition
`f`	values of f at the interior grid points (row i, column j)	
`h`	grid spacing (equal in x and y)	

output	meaning
`u`	approximate u at the interior grid points

- **Method:** the 5-point finite-difference system (4u_ij - neighbours)/h^2 = f_ij, solved by banded Gaussian elimination
- **Assumes:** $h > 0$; $u = 0$ on the boundary; at most 12 x 12 interior points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): f (input); u (returned). Other inputs: h = 0.5.

```
linear_algebra_ai999x.solve('poisson_2d', f='[[1, 1], [1, 1]]', h=0.5)
# -> u = [[0.125, 0.125], [0.12499999999999999, 0.12499999999999999]]   (SOLVED)
```

```
linear_algebra_ai999x.solve('poisson_2d', f=[[4]], h=1)
# -> u = [[1.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve the 2-D Poisson equation with zero boundary values on the grid with h = 0.5 and f = [[1, 1], [1, 1]].	SOLVED	u = [[0.125, 0.125], [0.12499999999999999, 0.12499999999999999]]

Root of a polynomial by bisection — `bisection_root`

The root of a polynomial inside an interval [a, b] where it changes sign, by repeated halving.

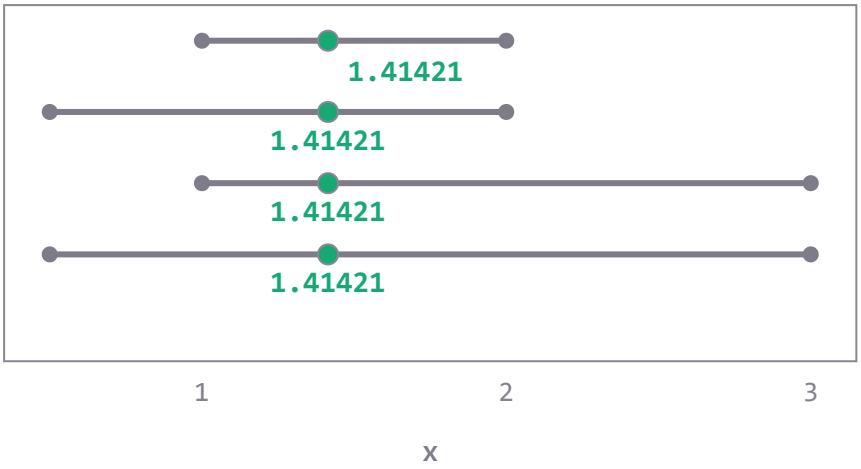
$$m = \tfrac{a+b}{2};\ [a,b] \leftarrow [a,m] \text{ if } p(a)p(m) < 0 \text{ else } [m,b]$$

input	meaning	condition
`coefficients`	polynomial coefficients, highest power first (written out in x in a question)	
`a`	left end of the interval	
`b`	right end of the interval	

output	meaning
`root`	the root in [a, b]

- **Method:** bisection until the interval cannot be halved in floating point (about 52 halvings per unit)
- **Assumes:** $a < b$; p(a) and p(b) have opposite signs; exactly one root in (a, b) (checked exactly by a Sturm sequence); the root is simple and well conditioned

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Brackets [a, b] for the root of the polynomial with coefficients 1, 0, -2 (highest power first): the worked example's [1, 2] (top) and three wider ones chosen here; on each, the root the package returned is marked: [1, 2] → 1.41421; [0.5, 2] → 1.41421; [1, 3] → 1.41421; [0.5, 3] → 1.41421.

```
linear_algebra_ai999x.solve('bisection_root', coefficients='1, 0, -2', a=1, b=2)
# -> root = 1.41421   (SOLVED)
```

```
linear_algebra_ai999x.solve('bisection_root', coefficients=[1, -1, -2], a=0, b=5)
# -> root = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the root of f(x) = x^2 - 2 between 1 and 2 by the bisection method.	SOLVED	root = 1.41421
Use bisection to find the root of x^3 - x - 2 on [1, 2].	SOLVED	root = 1.52138
Use bisection to find the root of x^2 - 2 on [2, 3].	INVALID_INPUT	Root of a polynomial by bisection: p(a) and p(b) must have opposite signs.
Use bisection to find the root of x^3 - x on [-2, 2].	INVALID_INPUT	Root of a polynomial by bisection: the interval must contain exactly one root.

Root of a polynomial by Newton's method — `newton_root`

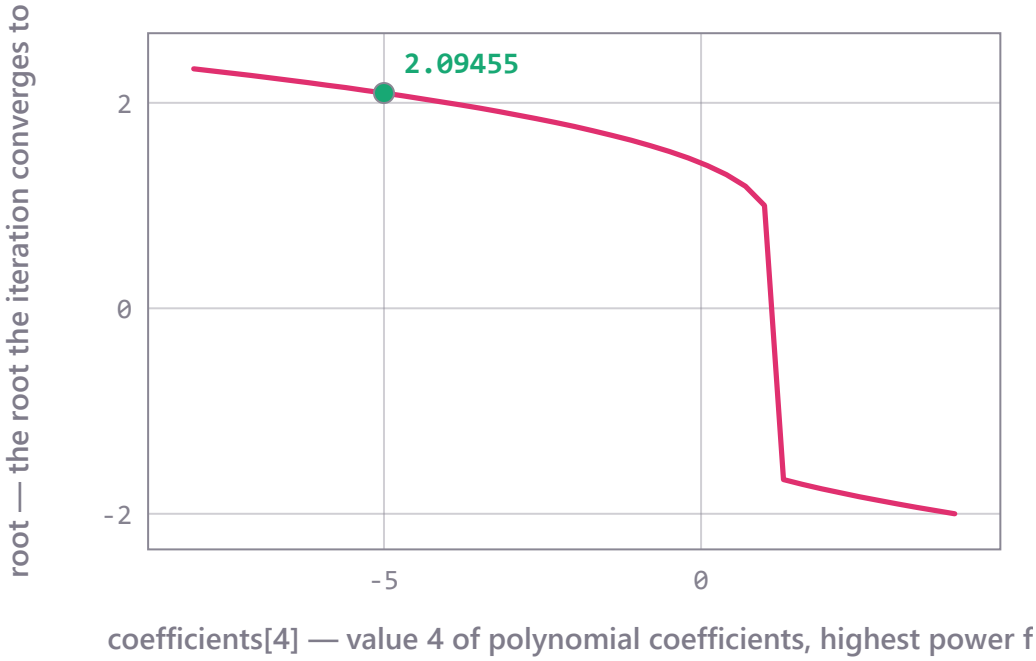
A root of a polynomial by Newton's method x <- x - p(x)/p'(x) from a starting value x0.

$$x_{k+1} = x_k - \frac{p(x_k)}{p'(x_k)}$$

input	meaning	condition
`coefficients`	polynomial coefficients, highest power first (written out in x in a question)	
`x0`	starting value	

output	meaning
`root`	the root the iteration converges to

- **Method:** Newton-Raphson iteration until the step is below the last bit of x
- **Assumes:** the iteration converges (within 200 steps) to a simple, well-conditioned root; p'(x) never vanishes at an iterate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



root (the root the iteration converges to) as value 4 of coefficients (polynomial coefficients, highest power first (written out in x in a question)) moves from -8 to 4, the other values as given (1, 0, -2, -5); 41 runs of linear-algebra-ai999x. Other inputs: x0 = 2. Marked: the worked example, coefficients[4] = -5, root = 2.09455.

```
linear_algebra_ai999x.solve('newton_root', coefficients='1, 0, -2, -5', x0=2)
# -> root = 2.09455   (SOLVED)
```

```
linear_algebra_ai999x.solve('newton_root', coefficients=[1, 0, -2], x0=1)
# -> root = 1.41421   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use Newton's method to find a root of x^3 - 2x - 5 starting from x0 = 2.	SOLVED	root = 2.09455
Find the root of f(x) = x^2 - 2 by Newton's method with x0 = 1.	SOLVED	root = 1.41421
Find the root of f(x) = x^2 + 1 by Newton's method with x0 = 1.	INVALID_INPUT	Root of a polynomial by Newton's method: p'(x) = 0 at an iterate: Newton's method breaks down.

Root of a polynomial by the secant method — `secant_root`

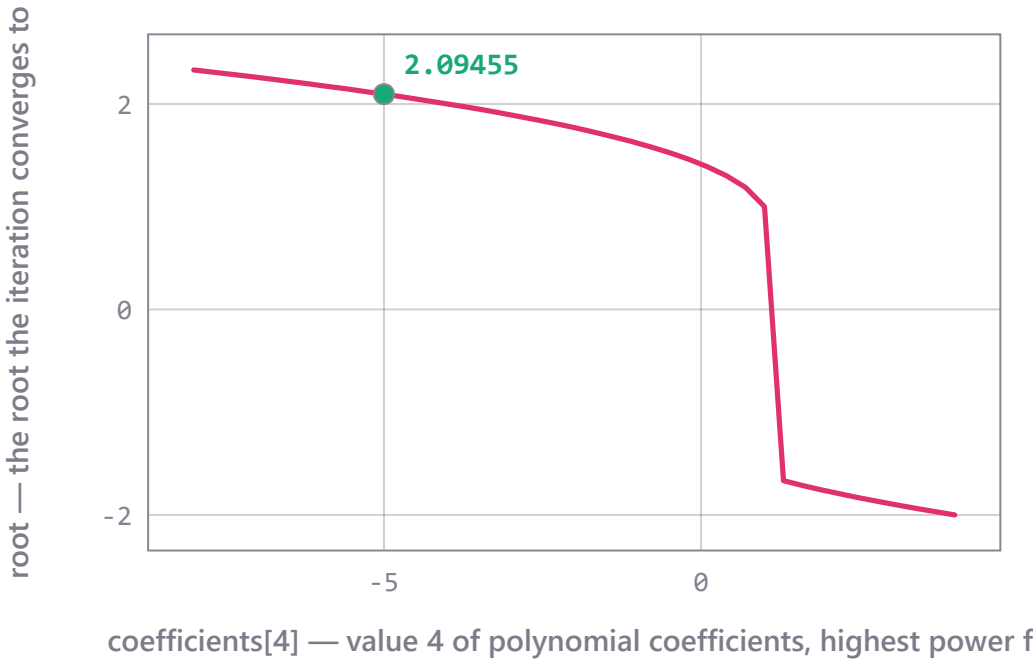
A root of a polynomial by the secant method from two starting values x0 and x1.

$$x_{k+1} = x_k - p(x_k)\,\frac{x_k - x_{k-1}}{p(x_k) - p(x_{k-1})}$$

input	meaning	condition
`coefficients`	polynomial coefficients, highest power first (written out in x in a question)	
`x0`	first starting value	
`x1`	second starting value	

output	meaning
`root`	the root the iteration converges to

- **Method:** secant iteration x_(k+1) = x_k - p(x_k)(x_k - x_(k-1))/(p(x_k) - p(x_(k-1)))
- **Assumes:** $x_0 \neq x_1$; the iteration converges (within 200 steps) to a simple, well-conditioned root
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



root (the root the iteration converges to) as value 4 of coefficients (polynomial coefficients, highest power first (written out in x in a question)) moves from -8 to 4, the other values as given (1, 0, -2, -5); 41 runs of linear-algebra-ai999x. Other inputs: x0 = 2, x1 = 3. Marked: the worked example, coefficients[4] = -5, root = 2.09455.

```
linear_algebra_ai999x.solve('secant_root', coefficients='1, 0, -2, -5', x0=2, x1=3)
# -> root = 2.09455   (SOLVED)
```

```
linear_algebra_ai999x.solve('secant_root', coefficients=[1, 0, -2], x0=1, x1=2)
# -> root = 1.41421   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the secant method to find a root of x^3 - 2x - 5 with x0 = 2 and x1 = 3.	SOLVED	root = 2.09455

question	outcome	result
Find the root of f(x) = x^2 - 2 by the secant method with x0 = 1, x1 = 1.	INVALID_INPUT	Root of a polynomial by the secant method: the two starting points must differ.

Evaluate a polynomial and its derivative (Horner's method) — `polynomial_evaluation`

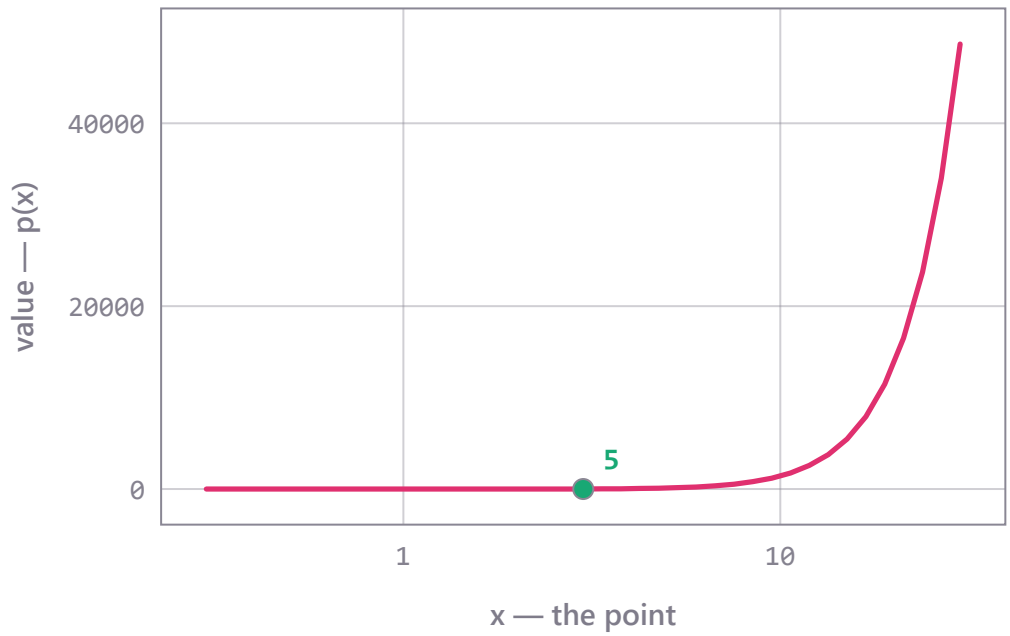
The value p(x) and the derivative p'(x) of a polynomial at a point.

$$p(x) = (\cdots((a_n x + a_{n-1})x + a_{n-2})x + \cdots)x + a_0$$

input	meaning	condition
`coefficients`	polynomial coefficients, highest power first (written out in x in a question)	
`x`	the point	

output	meaning
`value`	p(x)
`derivative`	p'(x)

- **Method:** Horner's scheme with the derivative carried along (exact decimal arithmetic)
- **Assumes:** at most degree 12
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



value (p(x)) against x (the point) from 0.3 to 30 on a logarithmic axis. Other inputs as in the worked example: coefficients = 2, -6, 2, -1. 41 points computed by linear-algebra-ai999x; marked: the worked example, x = 3, value = 5.

```
linear_algebra_ai999x.solve('polynomial_evaluation', coefficients='2, -6, 2, -1', x=3)
# -> value = 5, derivative = 20   (SOLVED)
```

```
linear_algebra_ai999x.solve('polynomial_evaluation', coefficients=[1, 0, 0, -3, 1], x=2)
# -> value = 11, derivative = 29   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate p(x) = 2x^3 - 6x^2 + 2x - 1 at x = 3.	SOLVED	value = 5, derivative = 20
Use Horner's method to evaluate x^4 - 3x + 1 at x = 2.	SOLVED	value = 11, derivative = 29

Lagrange interpolation — `lagrange_interpolation`

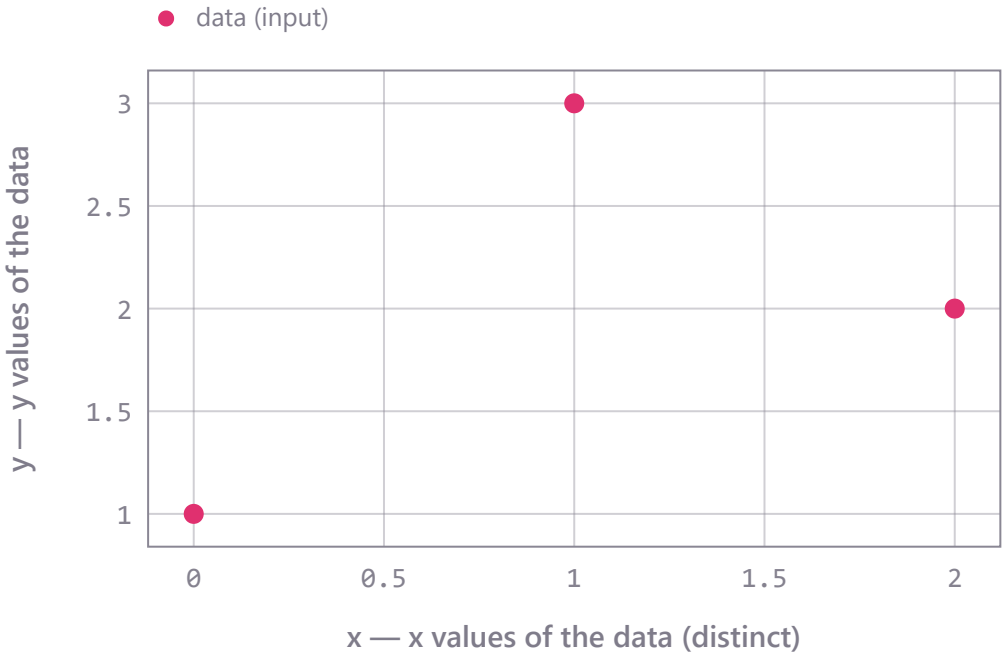
The value at a point of the polynomial of lowest degree through given data points (Lagrange form).

$$p(t) = \sum_i y_i \prod_{j \neq i} \frac{t - x_j}{x_i - x_j}$$

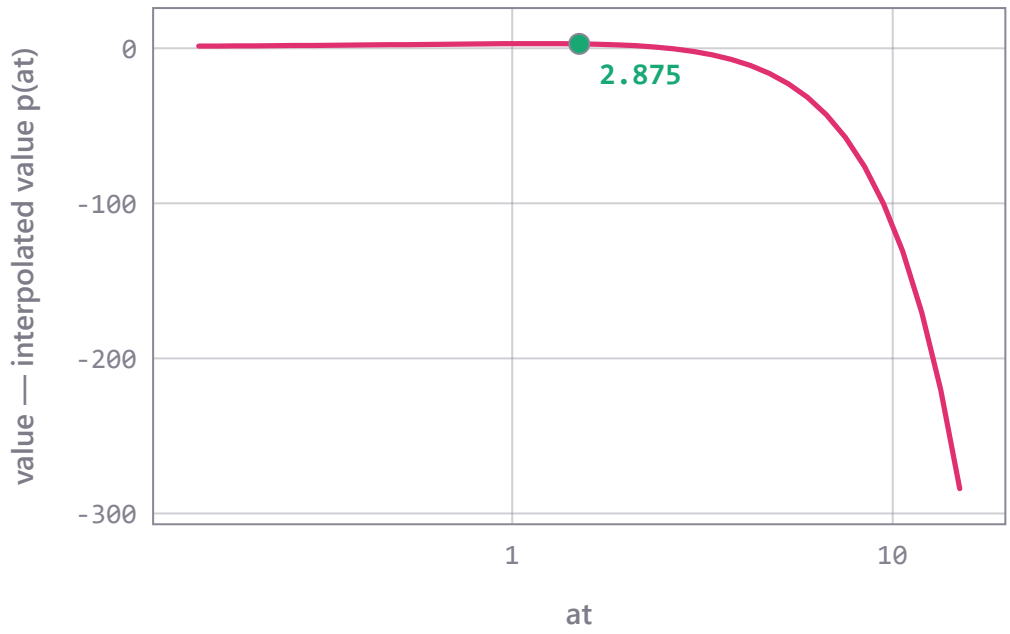
input	meaning	condition
`x`	x values of the data (distinct)	
`y`	y values of the data	
`at`	the point where the interpolant is evaluated	

output	meaning
`value`	interpolated value p(at)

- **Method:** the Lagrange formula sum y_i prod (t - x_j)/(x_i - x_j), exact decimal arithmetic
- **Assumes:** distinct x values; x and y of equal length (at most 60)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y values of the data) against x (x values of the data (distinct)) as given in the worked example (3 points). Also returned: value = 2.875. Other inputs: at = 1.5.



value (interpolated value p(at)) against at (the point where the interpolant is evaluated) from 0.15 to 15 on a logarithmic axis. Other inputs as in the worked example: x = 0, 1, 2, y = 1, 3, 2. 41 points computed by linear-algebra-ai999x; marked: the worked example, at = 1.5, value = 2.875.

```
linear_algebra_ai999x.solve('lagrange_interpolation', x='0, 1, 2', y='1, 3, 2', at=1.5)
# -> value = 2.875   (SOLVED)
```

```
linear_algebra_ai999x.solve('lagrange_interpolation', x=[1, 2, 4], y=[1, 4, 16], at=3)
# -> value = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use Lagrange interpolation through the points (0, 1), (1, 3), (2, 2) to estimate the value at x = 1.5.	SOLVED	value = 2.875
Interpolate the data x = (1, 2, 4), y = (1, 4, 16) with the Lagrange polynomial at 3.	SOLVED	value = 9
Use Lagrange interpolation through the points (1, 1), (1, 3), (2, 2) to estimate the value at x = 1.5.	INVALID_INPUT	Lagrange interpolation: the x values must be distinct.

Newton divided differences — `newton_divided_differences`

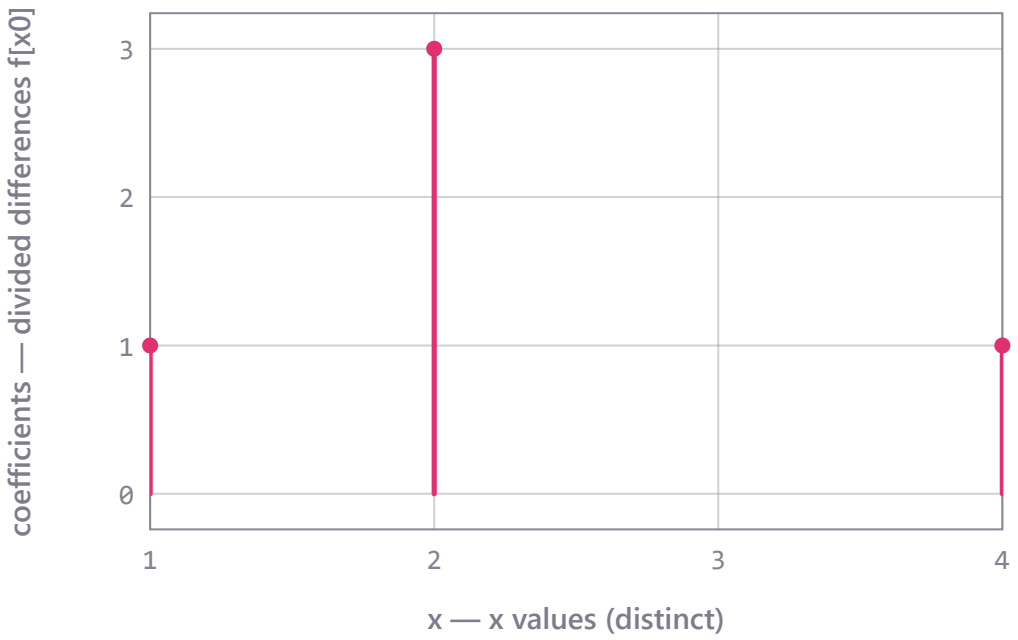
The coefficients f[x0], f[x0,x1], ..., f[x0..xn] of the Newton form of the interpolating polynomial.

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$$

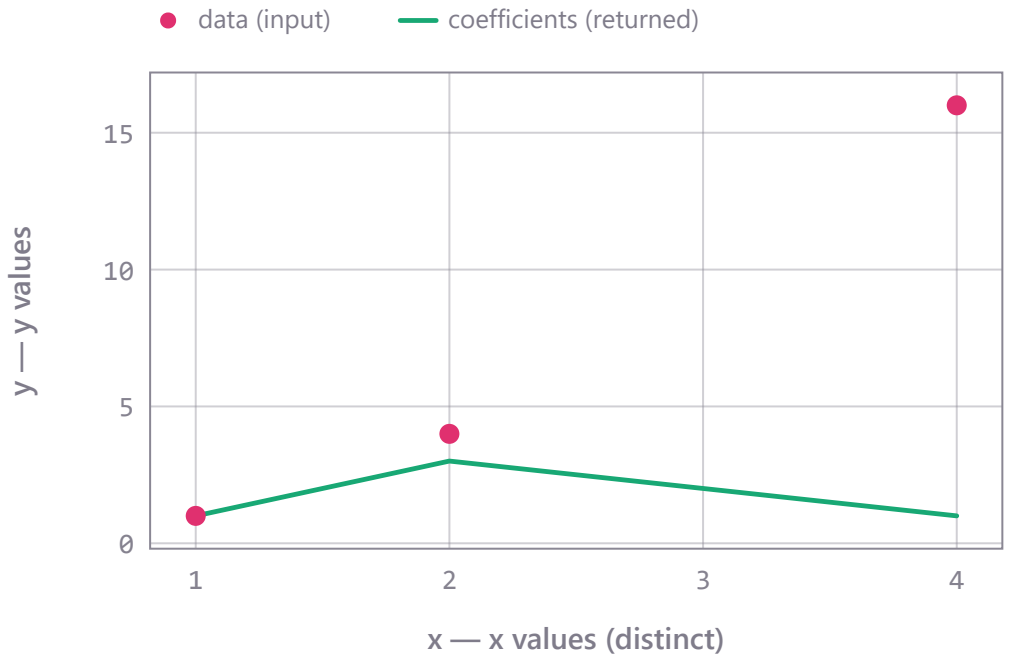
input	meaning	condition
`x`	x values (distinct)	
`y`	y values	

output	meaning
`coefficients`	divided differences f[x0], f[x0,x1], ...: p(t) = c0 + c1 $(t - x_0) + c_2 (t - x_0)(t - x_1) + \ldots$

- **Method:** the divided-difference table, exact decimal arithmetic
- **Assumes:** distinct x values; x and y of equal length (at most 60)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



divided differences f[x0], f[x0,x1], ...: p(t) = c0 + c1 (t - x0) + c2 (t - x0)(t - x1) + ... (coefficients) returned by linear-algebra-ai999x for the worked example, one value for each x (x values (distinct)) (3 values, first 1, last 1, largest 3). Inputs: x = 1, 2, 4, y = 1, 4, 16.



The data y (y values) against x (x values (distinct)) as given in the worked example (3 points); drawn on it from what linear-algebra-ai999x returned: the returned coefficients.

```
linear_algebra_ai999x.solve('newton_divided_differences', x='1, 2, 4', y='1, 4, 16')
# -> coefficients = [1.0, 3.0, 1.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('newton_divided_differences', x=[0, 1, 3], y=[1, 2, 10])
# -> coefficients = [1.0, 1.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Newton divided differences for the points (1, 1), (2, 4), (4, 16).	SOLVED	coefficients = [1.0, 3.0, 1.0]
Compute the divided differences of the data x = (0, 1, 3), y = (1, 2, 10).	SOLVED	coefficients = [1.0, 1.0, 1.0]

Coefficients of the interpolating polynomial — `interpolating_polynomial`

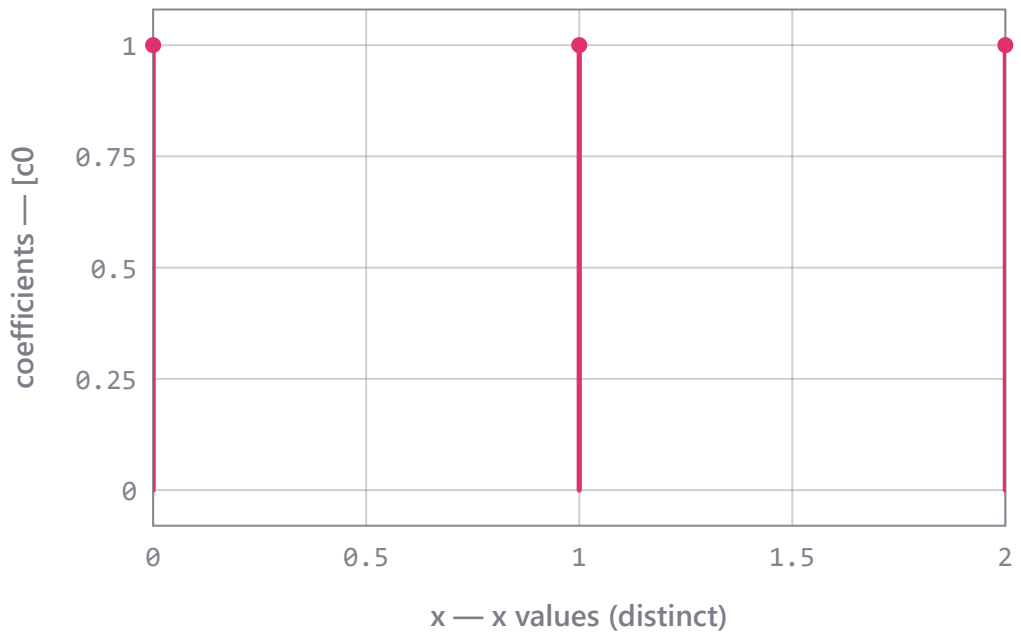
The polynomial $c_0 + c_1$ x + ... + c(n-1) x^{n-1} passing exactly through n data points.

$$\sum_k c_k x_i^k = y_i$$

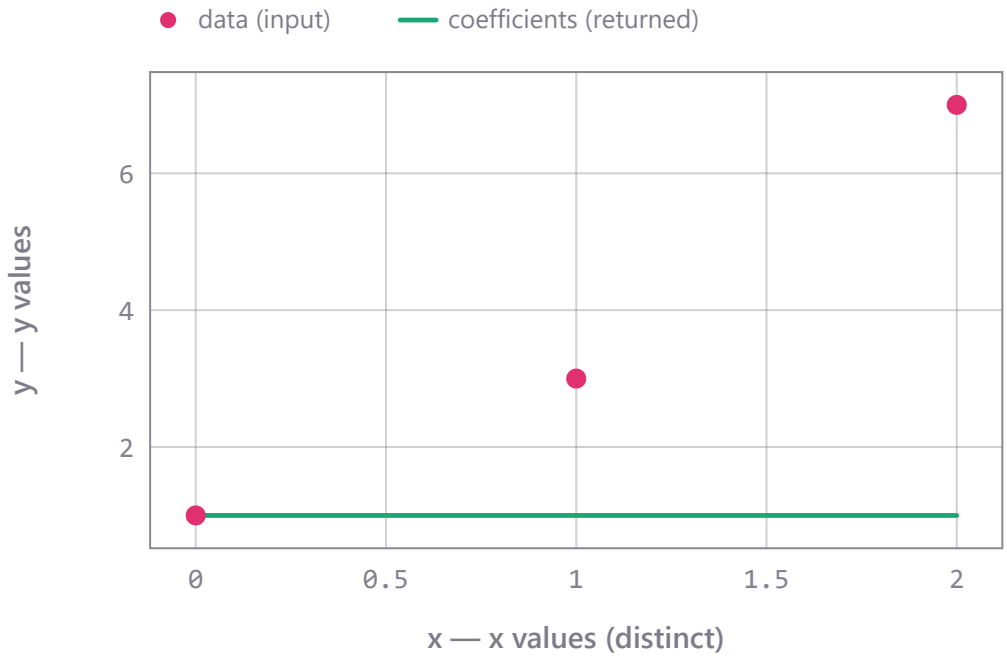
input	meaning	condition
`x`	x values (distinct)	
`y`	y values	

output	meaning
`coefficients`	[c0, c1, ...], lowest power first

- **Method:** solve the Vandermonde system V $c = y$ in exact rational arithmetic
- **Assumes:** distinct x values; at most 15 points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



[c0, c1, ...], lowest power first (coefficients) returned by linear-algebra-ai999x for the worked example, one value for each x (x values (distinct)) (3 values, first 1, last 1, largest 1). Inputs: x = 0, 1, 2, y = 1, 3, 7.



The data y (y values) against x (x values (distinct)) as given in the worked example (3 points); drawn on it from what linear-algebra-ai999x returned: the returned coefficients.

```
linear_algebra_ai999x.solve('interpolating_polynomial', x='0, 1, 2', y='1, 3, 7')
# -> coefficients = [1.0, 1.0, 1.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('interpolating_polynomial', x=[-1, 0, 1], y=[2, 1, 2])
# -> coefficients = [1.0, 0.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the interpolating polynomial through the points (0, 1), (1, 3), (2, 7).	SOLVED	coefficients = [1.0, 1.0, 1.0]
Find the polynomial passing through the points (-1, 2), (0, 1), (1, 2).	SOLVED	coefficients = [1.0, 0.0, 1.0]

Piecewise linear interpolation — `linear_interpolation`

The value at a point of the broken line joining consecutive data points.

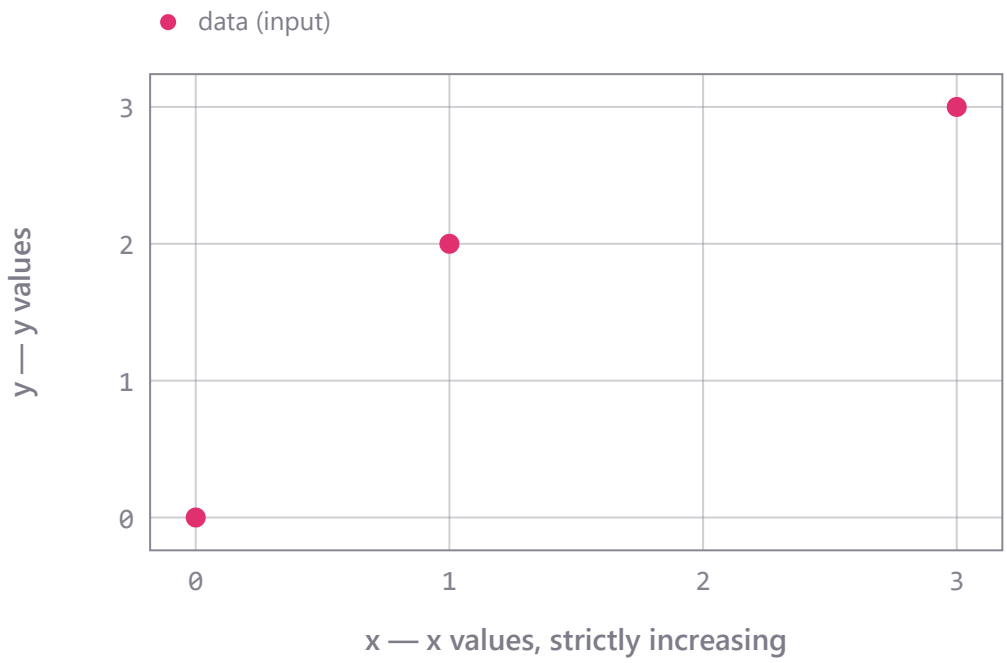
$$y = y_k + (y_{k+1} - y_k)\frac{t - x_k}{x_{k+1} - x_k}$$

input	meaning	condition
`x`	x values, strictly increasing	
`y`	y values	
`at`	the point (within the data range)	

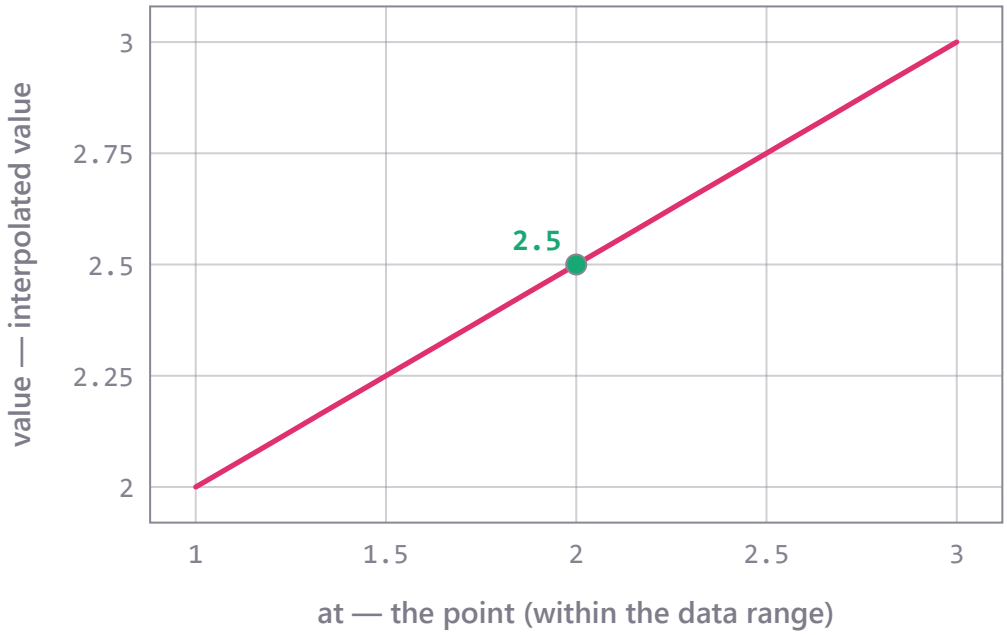
output	meaning
`value`	interpolated value

- **Method:** locate the interval containing the point and interpolate linearly (exact)

- **Assumes:** x strictly increasing; x[0] <= $at \leq x$[-1] (no extrapolation)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y values) against x (x values, strictly increasing) as given in the worked example (3 points). Also returned: value = 2.5. Other inputs: at = 2.



value (interpolated value) against at (the point (within the data range)) from 1 to 3. Other inputs as in the worked example: x = 0, 1, 3, y = 0, 2, 3. 31 points computed by linear-algebra-ai999x; marked: the worked example, at = 2, value = 2.5.

```
linear_algebra_ai999x.solve('linear_interpolation', x='0, 1, 3', y='0, 2, 3', at=2)
# -> value = 2.5   (SOLVED)
```

```
linear_algebra_ai999x.solve('linear_interpolation', x=[10, 20, 30], y=[5, 9, 4], at=25)
# -> value = 6.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use linear interpolation on the points (0, 0), (1, 2), (3, 3) to estimate the value at x = 2.	SOLVED	value = 2.5
Interpolate linearly in the table x = (10, 20, 30), y = (5, 9, 4) at 25.	SOLVED	value = 6.5
Interpolate linearly in the table x = (10, 20, 30), y = (5, 9, 4) at 35.	INVALID_INPUT	Piecewise linear interpolation: the point must lie within the range of the x values (no extrapolation).

Natural cubic spline interpolation — `natural_cubic_spline`

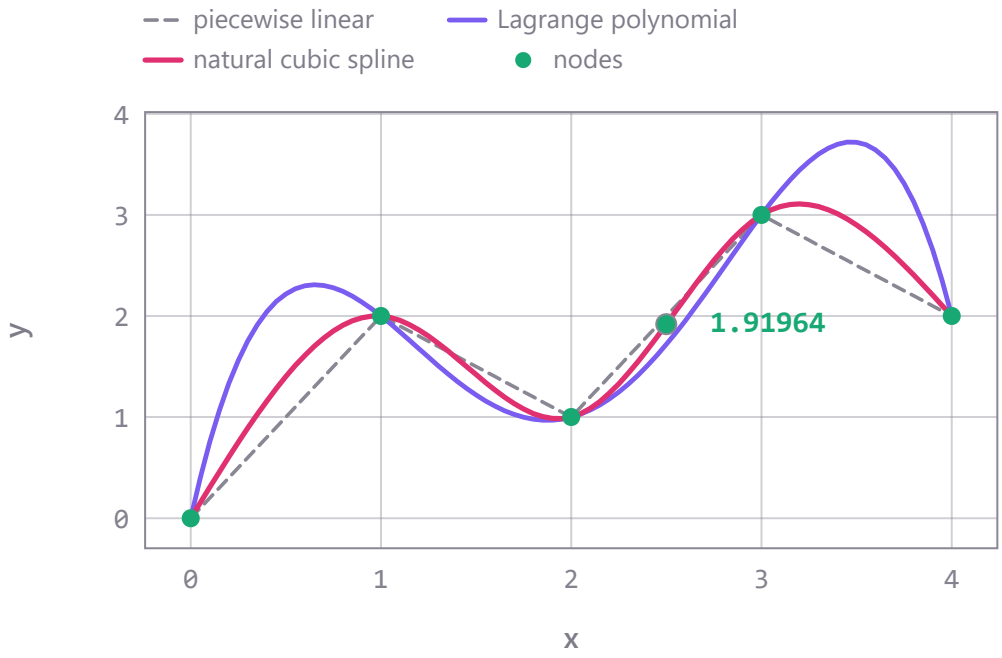
The value at a point of the natural cubic spline through the data (second derivative zero at both ends).

$$h_{i-1}M_{i-1} + 2(h_{i-1}+h_i)M_i + h_iM_{i+1} = 6\Big(\frac{y_{i+1}-y_i}{h_i} - \frac{y_i-y_{i-1}}{h_{i-1}}\Big),\ M_0 = M_n = 0$$

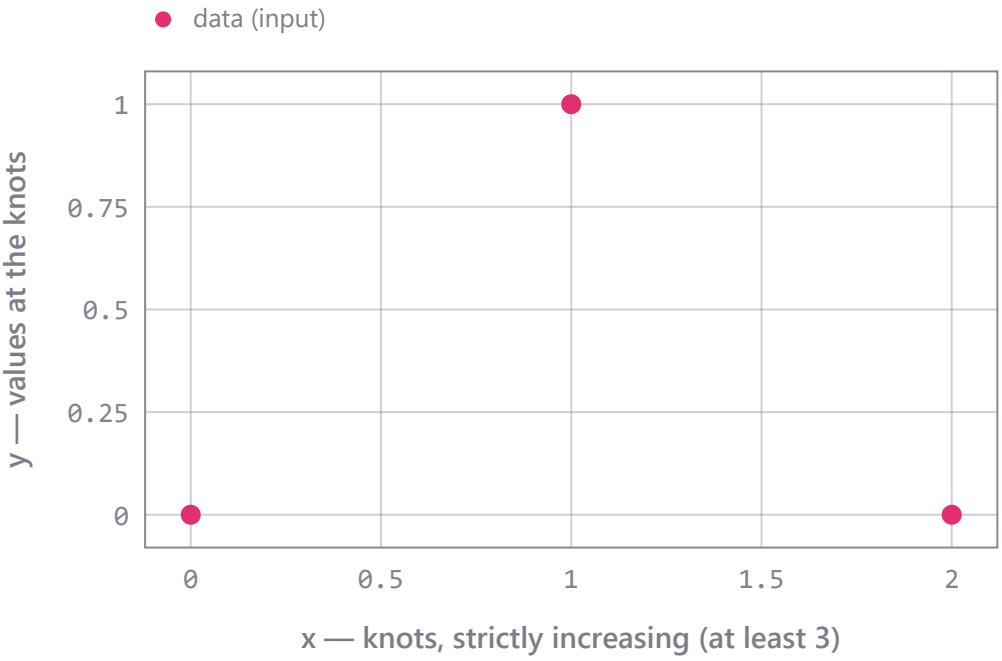
input	meaning	condition
`x`	knots, strictly increasing (at least 3)	
`y`	values at the knots	
`at`	the point (within the knots)	

output	meaning
`value`	spline value S(at)

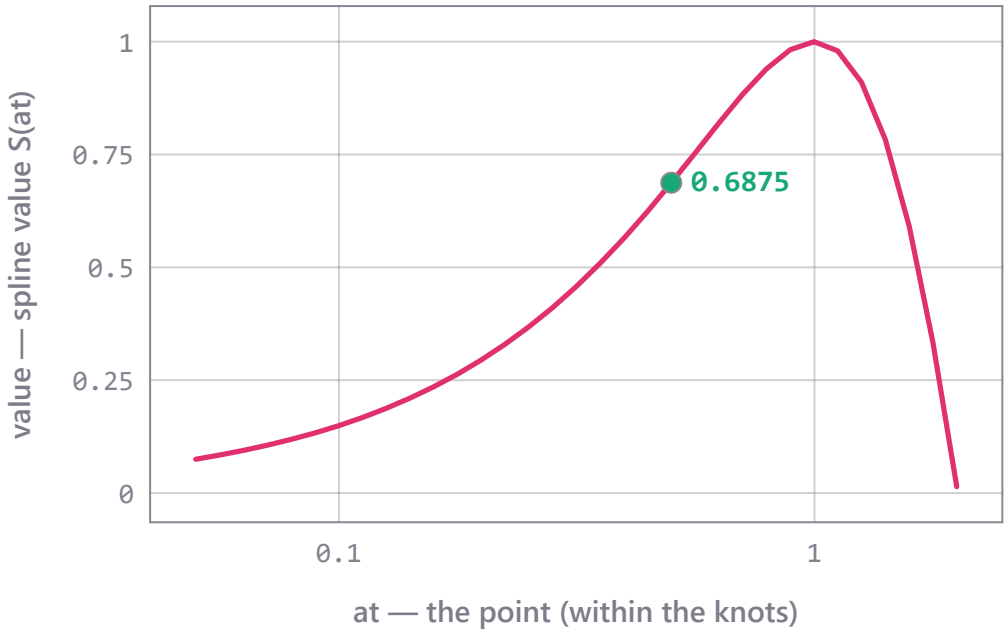
- **Method:** solve the tridiagonal system for the knot second derivatives exactly, then evaluate the cubic piece
- **Assumes:** x strictly increasing, at least 3 knots (at most 60); x[0] <= $at \le x$[-1]; natural end conditions S''$(x_0) = S''(xn) = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Three interpolants through the nodes x = 0, 1, 2, 3, 4, y = 0, 2, 1, 3, 2, each evaluated by the package at 81 points: the natural cubic spline, the degree-4 Lagrange polynomial and piecewise-linear interpolation; marked: x = 2.5, spline value = 1.91964.



The data y (values at the knots) against x (knots, strictly increasing (at least 3)) as given in the worked example (3 points). Also returned: value = 0.6875. Other inputs: at = 0.5.



value (spline value S(at)) against at (the point (within the knots)) from 0.05 to 5 on a logarithmic axis. Other inputs as in the worked example: x = 0, 1, 2, y = 0, 1, 0. 33 points computed by linear-algebra-ai999x; marked: the worked example, at = 0.5, value = 0.6875.

```
linear_algebra_ai999x.solve('natural_cubic_spline', x='0, 1, 2', y='0, 1, 0', at=0.5)
# -> value = 0.6875   (SOLVED)
```

```
linear_algebra_ai999x.solve('natural_cubic_spline', x=[0, 1, 2, 3], y=[0, 1, 8, 27], at=1.5)
# -> value = 3.15   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use a natural cubic spline through the points (0, 0), (1, 1), (2, 0) to estimate the value at x = 0.5.	SOLVED	value = 0.6875
Evaluate the natural cubic spline of the data x = (0, 1, 2, 3), y = (0, 1, 8, 27) at 1.5.	SOLVED	value = 3.15

Clamped cubic spline interpolation — `clamped_cubic_spline`

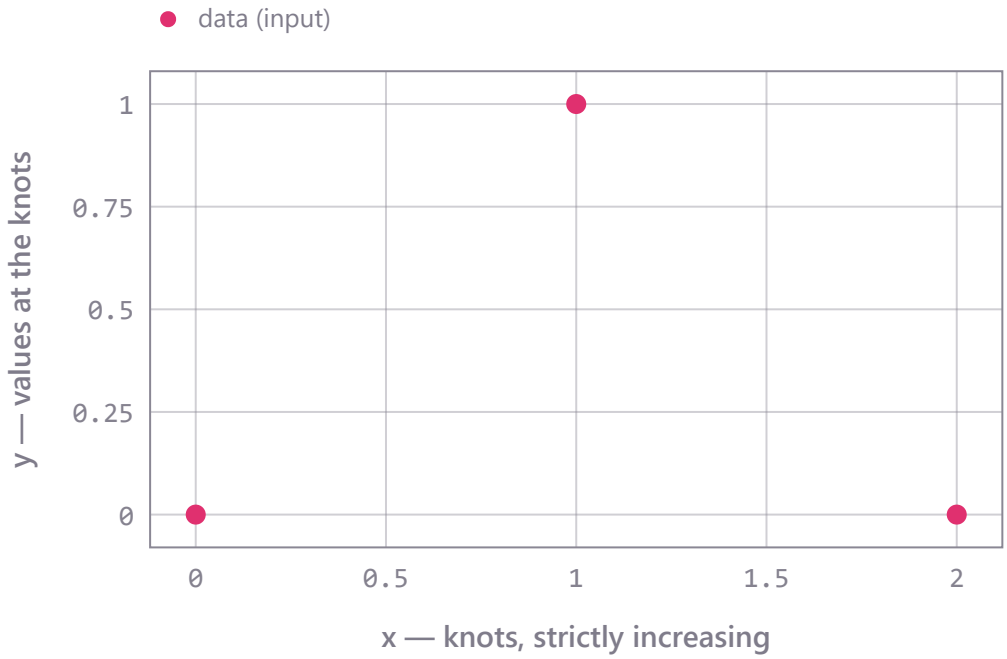
The value at a point of the cubic spline through the data with prescribed slopes at the two ends.

$$2h_0M_0 + h_0M_1 = 6\left(\frac{y_1 - y_0}{h_0} - s_0\right),\ h_{n-1}M_{n-1} + 2h_{n-1}M_n = 6\left(s_n - \frac{y_n - y_{n-1}}{h_{n-1}}\right)$$

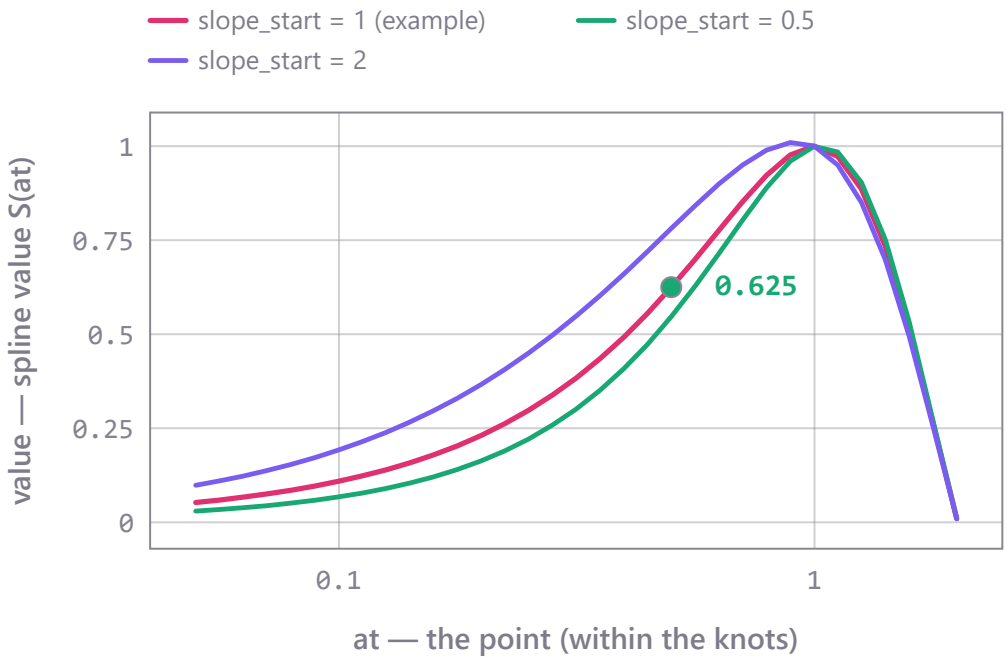
input	meaning	condition
`x`	knots, strictly increasing	
`y`	values at the knots	
`slope_start`	slope S'(x0)	
`slope_end`	slope S'(xn)	
`at`	the point (within the knots)	

output	meaning
`value`	spline value S(at)

- **Method:** solve the tridiagonal system with clamped end equations for the knot second derivatives exactly, then evaluate the cubic piece
- **Assumes:** x strictly increasing, at least 2 knots (at most 60); x[0] <= $at \leq x$[-1]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (values at the knots) against x (knots, strictly increasing) as given in the worked example (3 points). Also returned: value = 0.625. Other inputs: slope_start = 1, slope_end = -1, at = 0.5.



value (spline value S(at)) against at (the point (within the knots)) from 0.05 to 5 on a logarithmic axis. One curve for each value of slope_start (slope S'(x0)): 1, 0.5, 2. Other inputs as in the worked example: x = 0, 1, 2, y = 0, 1, 0, slope_end = -1. 33 points per curve computed by linear-algebra-ai999x; marked: the worked example, at = 0.5, value = 0.625.

```
linear_algebra_ai999x.solve('clamped_cubic_spline', x='0, 1, 2', y='0, 1, 0', slope_start=1,
slope_end=-1, at=0.5)
# -> value = 0.625   (SOLVED)
```

```
linear_algebra_ai999x.solve('clamped_cubic_spline', x=[0, 1, 2], y=[0, 1, 8], slope_start=0,
slope_end=12, at=1.5)
# -> value = 3.375   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use a clamped cubic spline through the points (0, 0), (1, 1), (2, 0) with end slopes 1 and -1 to estimate the value at x = 0.5.	SOLVED	value = 0.625

Trapezoidal rule for tabulated data — `trapezoid_rule`

The integral of tabulated data by the composite trapezoidal rule (any spacing).

$$\int y\,dx \approx \sum_i \frac{x_{i+1} - x_i}{2}(y_i + y_{i+1})$$

input	meaning	condition
`x`	x values, strictly increasing	
`y`	y values	

output	meaning
`integral`	trapezoidal estimate of the integral

- **Method:** sum of (x_(i+1) - x_i)(y_i + y_(i+1))/2, exact decimal arithmetic
- **Assumes:** x strictly increasing; at least 2 points (at most 60)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y values) against x (x values, strictly increasing) as given in the worked example (4 points). Also returned: integral = 6.5.

```
linear_algebra_ai999x.solve('trapezoid_rule', x='0, 1, 2, 4', y='1, 3, 2, 0')
# -> integral = 6.5   (SOLVED)
```

```
linear_algebra_ai999x.solve('trapezoid_rule', x=[0, 0.5, 1], y=[1, 2, 4])
# -> integral = 2.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the trapezoidal rule to integrate the data (0, 1), (1, 3), (2, 2), (4, 0).	SOLVED	integral = 6.5
Estimate the integral of the table x = (0, 0.5, 1), y = (1, 2, 4) by the trapezoid rule.	SOLVED	integral = 2.25

Simpson's rule for tabulated data — `simpson_rule`

The integral of equally spaced tabulated data by the composite Simpson $1/3$ rule.

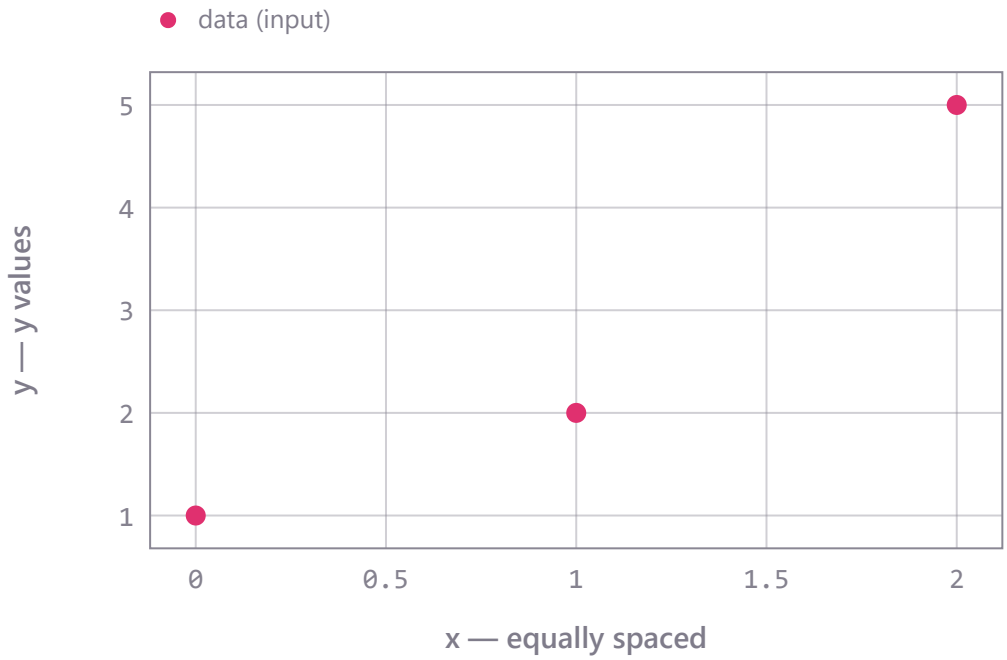
$$\int y\,dx \approx \frac{h}{3}\left(y_0 + 4\sum_{\text{odd}} y_i + 2\sum_{\text{even}} y_i + y_n\right)$$

input	meaning	condition
`x`	equally spaced, increasing x values (odd count)	
`y`	y values	

output	meaning
`integral`	Simpson estimate of the integral

- **Method:** $h/3$ (y_0 + 4 $y_1 + 2\,y_2 + 4$ y_3 + ... + y_n), exact decimal arithmetic

- **Assumes:** x equally spaced and increasing; an odd number of points, at least 3 (an even number of intervals)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y values) against x (equally spaced, increasing x values (odd count)) as given in the worked example (3 points). Also returned: integral = 4.66667.

```
linear_algebra_ai999x.solve('simpson_rule', x='0, 1, 2', y='1, 2, 5')
# -> integral = 4.66667   (SOLVED)
```

```
linear_algebra_ai999x.solve('simpson_rule', x=[0, 0.5, 1, 1.5, 2], y=[0, 0.25, 1, 2.25, 4])
# -> integral = 2.66667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use Simpson's rule to integrate the data (0, 1), (1, 2), (2, 5).	SOLVED	integral = 4.66667
Use Simpson's rule to integrate the data (0, 1), (1, 2), (2, 5), (3, 1).	INVALID_INPUT	Simpson's rule for tabulated data: Simpson's rule needs an odd number of points (an even number of intervals).

Composite midpoint rule for a polynomial — `midpoint_rule`

Approximate the integral of a polynomial over [a, b] by the composite midpoint rule with n subintervals.

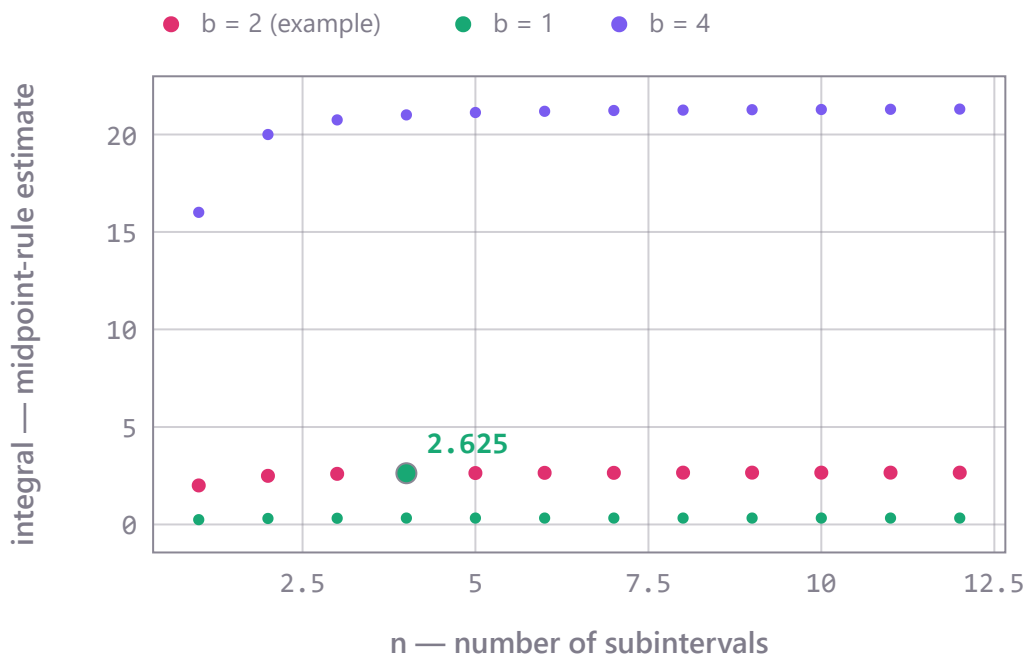
$$\int_a^b p \approx h \sum_{k=0}^{n-1} p\left(a + (k + \tfrac{1}{2})h\right)$$

input	meaning	condition
`coefficients`	polynomial coefficients, highest power first (written out in x in a question)	

input	meaning	condition
`a`	lower limit	
`b`	upper limit	
`n`	number of subintervals	

output	meaning
`integral`	midpoint-rule estimate

- **Method:** h sum p($a + (k + 1/2)$ h), $h = (b - a)/n$, exact decimal arithmetic
- **Assumes:** $a < b$; $1 \le n \le 1000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



integral (midpoint-rule estimate) against n (number of subintervals) from 1 to 12. One curve for each value of b (upper limit): 2, 1, 4. Other inputs as in the worked example: coefficients = 1, 0, 0, a = 0. 12 points per curve computed by linear-algebra-ai999x; marked: the worked example, n = 4, integral = 2.625.

```
linear_algebra_ai999x.solve('midpoint_rule', coefficients='1, 0, 0', a=0, b=2, n=4)
# -> integral = 2.625   (SOLVED)
```

```
linear_algebra_ai999x.solve('midpoint_rule', coefficients=[3, 0, 1], a=0, b=1, n=2)
# -> integral = 1.9375   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the midpoint rule with 4 subintervals to integrate x^2 from 0 to 2.	SOLVED	integral = 2.625
Approximate the integral of 3x^2 + 1 on [0, 1] by the midpoint rule with n = 2.	SOLVED	integral = 1.9375

Gauss-Legendre quadrature nodes and weights — `gauss_legendre_rule`

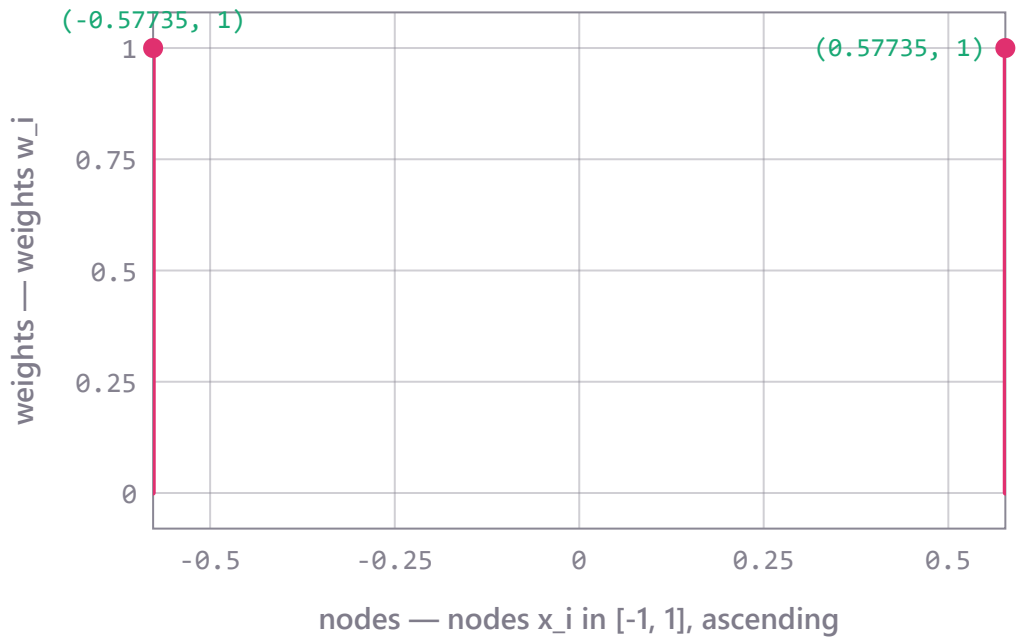
The nodes and weights of the n-point Gauss-Legendre rule on [-1, 1] (exact for polynomials of degree up to 2n - 1).

$$\int_{-1}^{1} f(x)\,dx \approx \sum_{i=1}^{n} w_i f(x_i), \quad P_n(x_i) = 0$$

input	meaning	condition
`n`	number of points (1 to 40)	

output	meaning
`nodes`	nodes x_i in [-1, 1], ascending
`weights`	weights w_i

- **Method:** Newton's method on the Legendre polynomial P_n (three-term recurrence); $w_i = 2$ / ($(1 - x_i^2)$ P_n'(x_i)^2)
- **Assumes:** $1 \le n \le 40$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



weights w_i (weights) at each nodes x_i in [-1, 1], ascending (nodes), both returned by linear-algebra-ai999x for the worked example (n = 2): -0.57735 → 1; 0.57735 → 1.

```
linear_algebra_ai999x.solve('gauss_legendre_rule', n=2)
# -> nodes = [-0.5773502691896257, 0.5773502691896257], weights = [1.0000000000000002,
1.0000000000000002]   (SOLVED)
```

```
linear_algebra_ai999x.solve('gauss_legendre_rule', n=3)
# -> nodes = [-0.7745966692414834, 0.0, 0.7745966692414834], weights = [0.5555555555555552,
0.8888888888888888, 0.5555555555555552]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the nodes and weights of the 3-point Gauss-Legendre rule.	SOLVED	nodes = [-0.7745966692414834, 0.0, 0.7745966692414834], weights = [0.5555555555555552, 0.8888888888888888, 0.5555555555555552]
What are the Gauss-Legendre nodes and weights for n = 2?	SOLVED	nodes = [-0.5773502691896257, 0.5773502691896257], weights = [1.0000000000000002, 1.0000000000000002]
What are the Gauss-Legendre nodes and weights for n = 50?	INVALID_INPUT	n: must be at most 40, got 50.

Gauss-Legendre quadrature of a polynomial — `gauss_legendre_integral`

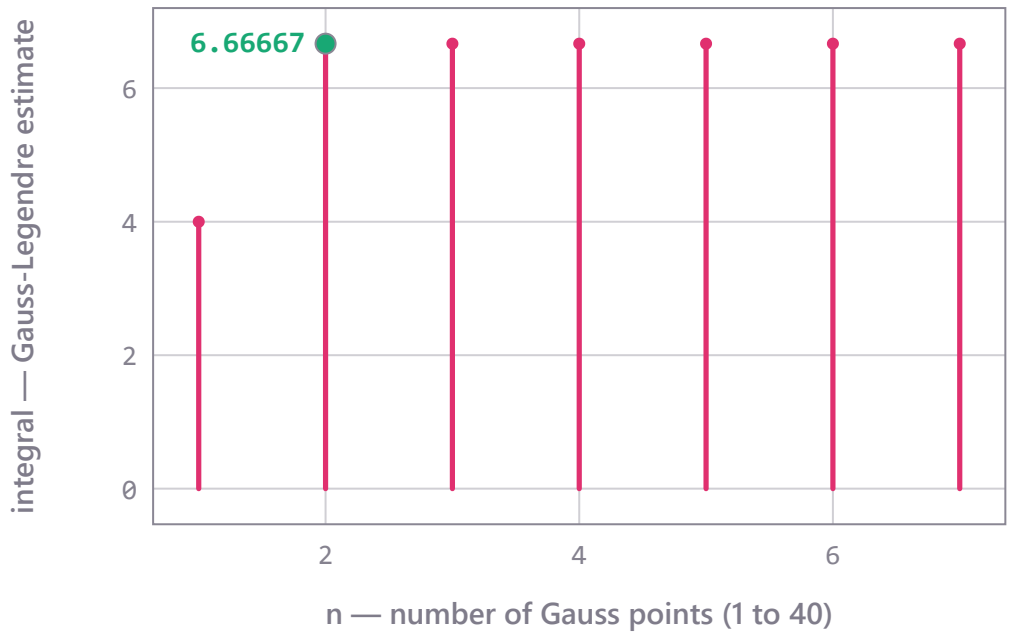
The n-point Gauss-Legendre approximation of the integral of a polynomial over [a, b] (exact up to degree 2n - 1).

$$\int_a^b p(x)\,dx \approx \frac{b-a}{2}\sum_i w_i\,p\Big(\frac{a+b}{2}+\frac{b-a}{2}x_i\Big)$$

input	meaning	condition
`coefficients`	polynomial coefficients, highest power first (written out in x in a question)	
`a`	lower limit	
`b`	upper limit	
`n`	number of Gauss points (1 to 40)	

output	meaning
`integral`	Gauss-Legendre estimate

- **Method:** map the n Gauss-Legendre nodes to [a, b]: $(b-a)/2$ sum w_i p($(a+b)/2 + (b-a)/2$ x_i)
- **Assumes:** $a < b$; $1 \le n \le 40$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



integral (Gauss-Legendre estimate) against n (number of Gauss points (1 to 40)) from 1 to 7. Other inputs as in the worked example: coefficients = 1, 1, 0, 0, a = 0, b = 2. 7 points computed by linear-algebra-ai999x; marked: the worked example, n = 2, integral = 6.66667.

```
linear_algebra_ai999x.solve('gauss_legendre_integral', coefficients='1, 1, 0, 0', a=0, b=2, n=2)
# -> integral = 6.66667   (SOLVED)
```

```
linear_algebra_ai999x.solve('gauss_legendre_integral', coefficients=[1, 0, 0, 0, 0], a=-1, b=1,
n=3)
# -> integral = 0.4   (SOLVED)
```

```
linear_algebra_ai999x.solve('gauss_legendre_integral', coefficients=[1, 0, 0, 0, 0], a=-1, b=1,
n=2)
# -> integral = 0.222222   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use 2-point Gauss-Legendre quadrature to integrate x^3 + x^2 from 0 to 2.	SOLVED	integral = 6.66667
Integrate x^4 on [-1, 1] by Gaussian quadrature with n = 3.	SOLVED	integral = 0.4

Derivative of equally spaced data (finite differences) — `numerical_derivative`

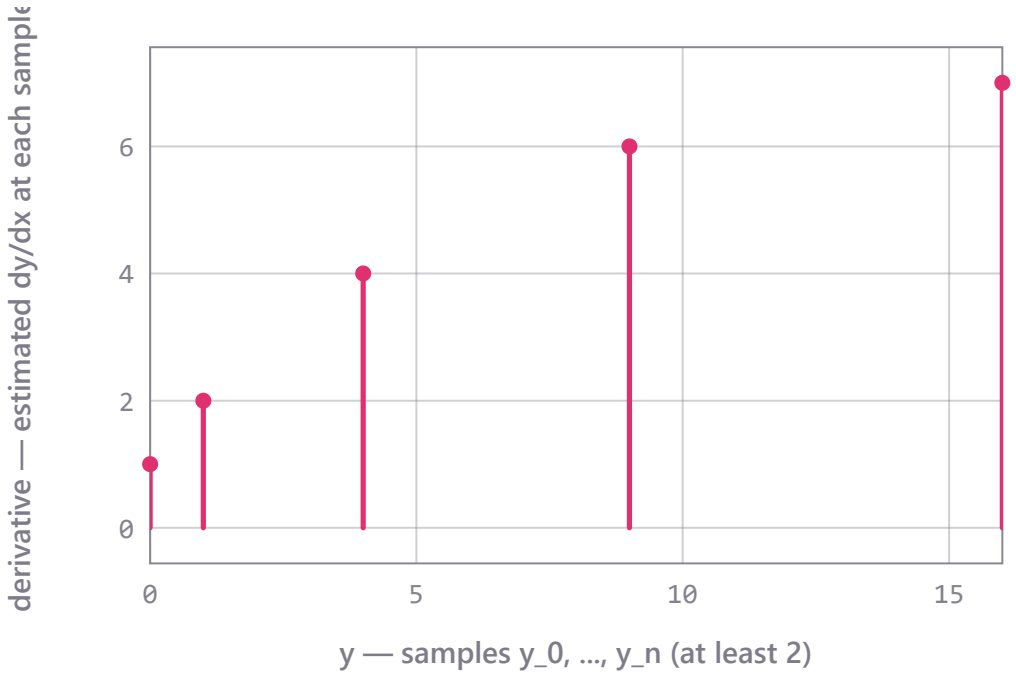
Estimate dy/dx at every sample of equally spaced data: central differences inside, one-sided differences at the ends.

$$y'_i \approx \frac{y_{i+1} - y_{i-1}}{2h}$$

input	meaning	condition
`y`	samples y_0, ..., y_n (at least 2)	
`h`	spacing between samples	

output	meaning
`derivative`	estimated dy/dx at each sample

- **Method:** (y_(i+1) - y_(i-1))/(2h) inside; $(y_1 - y_0)/h$ and (y_n - y_(n-1))/h at the ends (exact decimal arithmetic)
- **Assumes:** $h > 0$; at least 2 samples (at most 200)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



estimated dy/dx at each sample (derivative) returned by linear-algebra-ai999x for the worked example, one value for each y (samples y_0, ..., y_n (at least 2)) (5 values, first 1, last 7, largest 7). Inputs: y = 0, 1, 4, 9, 16, h = 1.

```
linear_algebra_ai999x.solve('numerical_derivative', y='0, 1, 4, 9, 16', h=1)
# -> derivative = [1.0, 2.0, 4.0, 6.0, 7.0]   (SOLVED)
```

```
linear_algebra_ai999x.solve('numerical_derivative', y=[1, 2, 4, 8], h=0.5)
# -> derivative = [2.0, 3.0, 6.0, 8.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Estimate the derivative of the samples y = (0, 1, 4, 9, 16) with spacing h = 1.	SOLVED	derivative = [1.0, 2.0, 4.0, 6.0, 7.0]
Differentiate the data [1, 2, 4, 8] numerically with h = 0.	INVALID_INPUT	Derivative of equally spaced data (finite differences): the spacing h must be positive.